Covering Problems in Network Design and Geometric Optimization

A THESIS

SUBMITTED FOR THE DEGREE OF

Doctor of Philosophy

IN THE

Faculty of Engineering

BY

Aditya Subramanian



Computer Science and Automation
Indian Institute of Science
Bangalore – 560 012 (INDIA)

July, 2026

Declaration of Originality

I, **Aditya Subramanian**, with SR No. **04-04-00-10-12-20-1-18510** hereby declare that the material presented in the thesis titled

Covering Problems in Network Design and Geometric Optimization

represents original work carried out by me in the **Department of Computer Science and Automation** at **Indian Institute of Science** during the years **2020-2025**.
With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: Student Signature

In my capacity as supervisor of the above-mentioned work, I certify that the above statements are true to the best of my knowledge, and I have carried out due diligence to ensure the originality of the report.

Advisor Name: Advisor Signature

© Aditya Subramanian
July, 2026
All rights reserved

DEDICATED TO

Keerthan

who started this path at my side. This work is for both of us.

Acknowledgements

First and foremost, I would like to express my deepest gratitude to my advisor, Prof. Arindam Khan. This thesis would not have been possible without his constant guidance, his immense patience, and the freedom he granted me to find my own path. I am incredibly fortunate to have had the opportunity to learn from him.

I am also deeply grateful to Prof. Rahul Saladi for our collaborations and his insightful feedback. Rahul has a unique ability to find the core of what makes a result work and simplify it with remarkable clarity. Beyond technical discussions, I truly value the career and life advice he has shared over the years.

A special thanks goes to Prof. Seeun William Umboh at the University of Melbourne and Prof. Manoj Gupta at IIT Gandhinagar. My research visits to their respective institutions were very productive and enriching periods of my doctoral journey. These experiences were instrumental in helping me transition toward working more independently, and I am sincerely thankful for their hospitality and the many stimulating discussions we shared.

Life at IISc would have been incomplete without Ajay and Pratheek, who have been my constant companions over the last five years. From the department to the mess and everywhere in between, they have been my primary support system. I am also thankful to KVN, Atasi, Swetha, and Nishanth for the countless hours spent on everything from technical discussions to questioning our decision to pursue a PhD. Back at the hostel, there was never a dull moment thanks to the company of Dillip, Santosh, Darshit, and Gagan. I cherish our time spent jamming, cooking together, playing corridor cricket, watching movies, and the late-night chats that stretched into the morning. I also value the much-needed breaks provided by my friends from the tennis courts, the music rooms, the cubing community, and various trekking expeditions. These escapes were vital for my well-being and sanity.

I owe a special debt of gratitude to my friends from my undergraduate days, Nishank and Praneet. Their excellence and drive originally motivated me to pursue a career in academia. And a special thanks to Praneet for hosting us more times than I can count; those weekends filled with puzzle contests, games, and great food provided the perfect reset whenever I needed

a break from research.

I would like to thank the CSA office staff, especially Ms. Kushael and Ms. Shubha, for their administrative support, which allowed me to focus on research rather than paperwork. I am also grateful to the Walmart Center for Tech Excellence and the Kotak AI-ML Centre for the financial support that made this doctoral study possible.

Finally, I want to thank my parents. They showed me the beauty of mathematics and sparked the curiosity that ultimately led me toward research. They have stood by me in every decision I've made, and their unwavering support is the foundation of everything I achieve.

Abstract

Covering problems represent a fundamental paradigm in combinatorial optimization: how can we efficiently select a minimum-cost collection of resources to satisfy a set of requirements? These problems lie at the heart of many real-world applications, ranging from the design of fault-tolerant internet architectures to resource allocation in heterogeneous computing environments. This thesis explores this question across two domains: *Fault-Tolerant Network Design* and *Computational Geometry*. By addressing inputs characterized by uncertainty and dynamic changes, we contribute new algorithms and hardness results across online, dynamic, and approximation models. A recurring technical theme is the development of an "interval flattening" procedure, which allows us to decompose complex weighted structures into manageable, bounded-frequency ones. This technique proves robust across both graph-theoretic and geometric settings.

The first part of the thesis focuses on making networks resilient to failures via the Connectivity Augmentation Problem (CAP). We consider CAP in the *online setting*: given a k-edge-connected graph G with n vertices and a set $\mathcal{L}$ of additional edges over the vertices of G, called links, online requests arrive one by one, each specifying two vertices that need to be $(k+1)$-edge-connected. We start with the graph G, and upon the arrival of a request $\{u, v\}$, we must *immediately and irrevocably* add links from $\mathcal{L}$ to the graph so that u and v are $(k+1)$-edge-connected in the resulting augmented graph, while trying to minimize the number of links added. We present a deterministic $O(\log n)$-competitive algorithm, improving upon the previously known randomized $\tilde{O}(k \log^3 n)$ bound by Gupta, Krishnaswamy, and Ravi (SIAM J. Comput. '12). Further, for the online Tree Augmentation Problem (TAP), the special case where the underlying graph is a tree, we develop simpler, self-contained combinatorial algorithms for both the unweighted and weighted variants that achieve tight competitive ratios.

In the second part, we shift focus to geometric optimization in dynamic environments. We address the weighted dynamic Geometric Set Cover problem. In this problem, we are given a collection of weighted d-dimensional hyperrectangles, and we have to maintain a low-cost subset that covers a set of points undergoing insertions and deletions. We provide the first

algorithms to achieve both polylogarithmic approximation ratios and polylogarithmic update times for weighted d-dimensional hyperrectangles, partially resolving an open question posed by Chan, He, Suri, and Xue (SODA '22). Notably, our bounds remain independent of the range of input weights, and also extend to the Geometric Hitting Set problem.

Finally, we settle the approximability of the Rectangle Stabbing problem. In this setting, one must select a set of horizontal segments of minimum total length, to stab a given collection of axis-aligned rectangles. A segment is said to stab a rectangle if it intersects opposite sides of the rectangle. We provide a Polynomial-Time Approximation Scheme (PTAS) for this problem, resolving its approximability. We further generalize our study to rectilinear polygons, and characterize the conditions under which the problem admits a PTAS. We show that there exists a PTAS if each input polygon is composed of a stack of rectangles with a bounded range of widths. On the other hand, if the input polygons do not satisfy these conditions, we prove that the problem is APX-hard.

We complement these algorithmic contributions with conditional lower bounds for the Batched Maximum Range Sum (MaxRS) problem, which seeks to position a geometric shape to maximize the weight of enclosed points. MaxRS is a heavily used primitive in geographic information systems (GIS) and spatial databases for optimal facility placement and hotspot detection. By leveraging the conditional hardness of $(\min, +)$-convolution, we demonstrate that subquadratic algorithms for MaxRS are unlikely, effectively proving that processing queries independently is optimal for weighted instances.

Publications Based on This Thesis

- *Online Connectivity Augmentation.*
  Mohit Garg, Aditya Subramanian.
  37th ACM-SIAM Symposium on Discrete Algorithms **(SODA)**: 85-104, 2026.

- *A Bouquet of Results on Maximum Range Sum: General Techniques and Hardness Reductions.*
  Rachana Gusain, Saladi Rahul, Aditya Subramanian.
  To appear in 45th Symposium on Principles of Database Systems **(PODS)**, 2026.

- *A PTAS for the Horizontal Rectangle Stabbing Problem.*
  Arindam Khan, Aditya Subramanian, Andreas Wiese.
  Mathematical Programming, 206(1):607–630, 2024.
  23rd Conference on Integer Programming and Combinatorial Optimization (**IPCO**): 361-374, 2022.

- *On Approximation Schemes for Stabbing Rectilinear Polygons.*
  Arindam Khan, Aditya Subramanian, Tobias Widmann, Andreas Wiese.
  44th Conference on Foundations of Software Technology and Theoretical Computer Science **(FSTTCS)**: 27:1-27:18, 2024.

- *Online and Dynamic Algorithms for Geometric Set Cover and Hitting Set.*
  Arindam Khan, Aditya Lonkar, Saladi Rahul, Aditya Subramanian, Andreas Wiese.
  39th International Symposium on Computational Geometry (**SoCG**): 46:1-46:17, 2023.

Contents

Acknowledgements i

Abstract iii

Publications Based on This Thesis v

Contents vi

List of Figures xi

List of Tables xiii

1 Introduction 1

1.1 A Unifying Thread: Interval Set Cover . . . 2
1.2 Covering Deficient Cuts with Edges . . . 3
1.3 Covering Dynamic Point Sets with Hyperrectangles . . . 4
1.4 Covering Rectangular Spans with Segments . . . 4
1.5 Maximizing Coverage of Points with Geometric Objects . . . 5
1.6 Thesis Organization . . . 5

2 A Snapshot of Covering Paradigms 7

2.1 Models of Computation . . . 7
2.2 Problem Families . . . 11

3 Preliminaries 14

3.1 General Notation and Analysis . . . 14
3.1.1 Asymptotic Notation . . . 14
3.1.2 Approximation Algorithms (Offline Setting) . . . 15
3.1.3 Online Algorithms . . . 16

3.1.4 Dynamic Algorithms . . . 17
3.2 The Anatomy of Covering Problems . . . 18
3.2.1 Generic Covering Formulation . . . 18
3.2.2 The Covering LP and its Dual . . . 18
3.3 Graph Theory and Connectivity . . . 19
3.3.1 Connectivity and Cuts . . . 19
3.3.2 Cactus Graphs . . . 20
3.4 Geometric Optimization Fundamentals . . . 20
3.4.1 Geometric Objects . . . 20
3.4.2 Geometric Set Cover and Geometric Hitting Set . . . 21
3.4.3 Geometric Stabbing . . . 21
3.5 Geometric Data Structures . . . 22
3.5.1 Interval Trees . . . 22
3.5.2 Segment Trees . . . 22
3.6 Fine-Grained Complexity . . . 23
3.6.1 The $(\min, +)$-Convolution Hypothesis . . . 24

4 Algorithms for Connectivity Augmentation 25
4.1 Introduction . . . 25
4.1.1 Our Results . . . 27
4.1.2 Our Techniques . . . 27
4.1.3 Further Related Work . . . 28
4.2 Reduction from Online CAP to Online CacAP . . . 29
4.3 Properties of Links and Cacti . . . 31
4.4 An $O(\log n)$-competitive Algorithm for Online CAP . . . 37
4.4.1 Structuring the Input Instance . . . 38
4.4.2 Solving Structured Instances . . . 40
4.4.3 Analysis via a Charging Argument . . . 42
4.4.3.1 Bound for a Cycle Instance . . . 43
4.4.3.2 Bound for the Tree Instance . . . 45
4.5 Online Algorithms for Weighted CAP and Online CycAP . . . 45
4.5.1 Reduction to Online NWSF . . . 46
4.5.2 Approximating CycAP using EWSF . . . 48

5 Algorithms for Tree Augmentation **50**
5.1 Introduction . . . 50
5.2 Simple $O(\log n)$-competitive Algorithm for TAP . . . 51
5.2.1 Algorithm . . . 52
5.2.2 Analysis . . . 52
5.3 Combinatorial Algorithm for Online WTAP . . . 55
5.3.1 Reduction from Online WTAP to Nice Online WPAP . . . 55
5.3.2 Preprocessing Links . . . 57
5.3.2.1 Geometric Pruning for Rooted Links . . . 57
5.3.2.2 Frequency Reduction for Unrooted Links . . . 57
5.3.3 The Algorithm . . . 60
5.3.4 Analysis . . . 62
5.4 An $\Omega(\log n)$ Lower Bound for TAP under Random-Order Arrivals . . . 64

6 Dynamic Weighted Set Cover for Hyperrectangles **66**
6.1 Introduction . . . 66
6.1.1 Dynamic Geometric Set Cover and Geometric Hitting Set . . . 67
6.1.2 Our Results and Techniques . . . 68
6.1.3 Related Work . . . 69
6.2 Weighted Dynamic Set Cover on Hyperrectangles . . . 70
6.2.1 An Overview of the Algorithm . . . 71
6.2.2 Construction of the New Set System on the Real Line . . . 72
6.2.3 Construction of New Set System in Higher Dimensions . . . 73
6.3 Weighted Dynamic Hitting Set on Hyperrectangles . . . 75

7 Stabbing Rectangles and Rectilinear Polygons **77**
7.1 Introduction . . . 77
7.1.1 Prior Work . . . 78
7.1.2 Our Results and Techniques . . . 78
7.2 Preliminaries . . . 79
7.2.1 Structural Properties . . . 80
7.2.2 Discretization . . . 81
7.3 PTAS for Horizontal Rectangle Stabbing . . . 84
7.3.1 Dynamic Program . . . 84
7.3.2 DP-decision Trees . . . 86

7.3.3 Analysis of the DP-Decision Tree . . . 91
7.4 Extensions to Similar Problems . . . 96
7.4.1 δ-large Rectangles . . . 96
7.4.1.1 Guessing Long Segments in the Solution . . . 96
7.4.1.2 Computing Short Segments in the Solution . . . 99
7.4.2 Generalized Minimum Manhattan Network . . . 100
7.5 PTAS for k-shapes with Bounded Width Variance . . . 101
7.5.1 Dynamic Program . . . 101
7.5.2 Analysis . . . 102
7.6 Lower Bound for the General Case . . . 107
7.6.1 APX-Hardness for 3-Shapes . . . 107
7.6.2 Logarithmic Hardness for General k-Shapes . . . 109

8 Lower Bounds for Batched MaxRS **112**
8.1 Introduction . . . 112
8.1.1 The Batched MaxRS Problem . . . 112
8.1.2 Our Results and Techniques . . . 113
8.1.3 Related Work . . . 113
8.2 Reductions for Convolution Variants . . . 114
8.2.1 From (min,+)-Convolution to (min,+,M)-Convolution . . . 114
8.2.2 From (min,+,M)-Convolution to (max,+,M)-Convolution . . . 115
8.2.3 From (max,+,M)-Convolution to Positive (max,+,M)-Convolution . . . 116
8.3 Lower Bound for Batched MaxRS . . . 116
8.4 Lower Bound for Batched Smallest k-Enclosing Interval . . . 118
8.4.1 Reduction from (min,+)-Convolution to Monotone (min,+)-Convolution 119
8.4.2 Reduction from Monotone (min,+)-Convolution to BSEI . . . 119

9 Conclusion and Future Directions **121**
9.1 Summary of Contributions . . . 121
9.1.1 Part I: Connectivity Augmentation . . . 121
9.1.2 Part II: Geometric Optimization . . . 122
9.2 Future Research Directions . . . 122
9.2.1 Online Network Design . . . 122
9.2.2 Geometric Set Cover and Geometric Hitting Set . . . 123

9.2.3 Rectangle Stabbing 123
9.2.4 Maximum Range Sum Queries 123

Bibliography **125**

List of Figures

1.1 Reduction from Interval Covering to Path Augmentation. A point request on the real line corresponds to an edge request on the discretized path graph. . . . 2

3.1 Example cactus, and the tree obtained on articulation vertices. 20
3.2 Geometric Set Cover and Hitting Set. 21
3.3 Horizontal Rectangle Stabbing. 22
3.4 Structure of interval tree. 23
3.5 Structure of segment tree 24

4.1 Link projections and crossing. 33
4.2 Different ways in which sections of colored vertices can interact. 35
4.3 Example instance of Cactus Augmentation Problem. 40
4.4 CacAP to NWSF reduction. 47

5.1 w_i is the branching vertex for the bottommost link b_i. x_i and y_i are its two children. 54
5.2 Path Decomposition 56
5.3 Interval flattening procedure. 59

6.1 Frequency reduction for 2D rectangles. 73
6.2 Hitting Set to Set Cover, dual transformation. 76

7.1 A solution to an instance of Rectangle Stabbing and HV-Stabbing. 80
7.2 Examples of k-shapes satisfying the hourglass condition. 81
7.3 A 3-shape not satisfying the hourglass condition (left), and a stack of rectangles that does not form a k-shape (right). 81
7.4 For $\delta = 1/2$, a k-shape that spans a width greater than $k/2 = O(k)$ times the narrowest part. 84
7.5 Horizontal line operations. 90

7.6 Grid lines and well-aligned segments across levels. 104
7.7 Reduction from Vertex Cover to k-stabbing. 108
7.8 Reduction from Hitting Set to k-stabbing. 110

8.1 Series of reductions leading to conditional hardness of Batched MaxRS. . . . 116
8.2 Position of constructed (weighted) points on real line. 117
8.3 Values of newly constructed sequence as points on the real line. 119

List of Tables

6.1 Dynamic algorithms for Geometric Set Cover and Hitting Set. 69

Chapter 1

Introduction

Combinatorial optimization is one of the staples of any introduction to algorithm design. Yet, despite being a foundational topic, the tools developed within this field remain highly relevant, continually pushing the frontiers of knowledge in resource allocation, logistics, and network stability. Within this vast field, covering problems represent a significant class of challenges. In the most abstract sense, a covering problem asks: how can we select a minimum-cost collection of resources to satisfy a set of requirements? Whether the resources are edges in a graph, transmitters in a wireless network, or traces on a printed circuit board, the fundamental objective remains to *cover* the demands efficiently.

This thesis, titled *Covering Problems in Network Design and Geometric Optimization*, explores the landscape of covering problems across two distinct but mathematically connected domains: *fault-tolerant network design* and *computational geometry*. While the specific structures differ, graphs and edges in the former, Euclidean spaces and geometric shapes in the latter, the underlying algorithmic challenge remains the same: how to achieve coverage efficiently under constraints of online arrivals, dynamic updates, and computational intractability.

The theoretical abstractions studied in this thesis are deeply rooted in modern technological applications. In the domain of network design, as telecommunication and fiber-optic infrastructures scale, they become increasingly vulnerable to physical link failures. Algorithms for connectivity augmentation directly translate to minimizing the cost of laying redundant physical cables to guarantee fault tolerance. Similarly, in the geometric domain, covering problems such as geometric stabbing have direct applications in VLSI circuit design and bandwidth allocation. Moreover, computing the optimal placement of a shape to maximize the weight of enclosed points, known as the Maximum Range Sum problem, is a fundamental primitive in spatial databases. For instance, such queries are used for hotspot detection, enabling authorities to optimally allocate emergency resources by pinpointing areas with the highest density of

reported cases during a pandemic.

1.1 A Unifying Thread: Interval Set Cover

Before delving into the specific domains of graphs and geometry, we look at a common thread between these two fields, which motivated our study of these diverse topics: the Interval Set Cover problem. Here, the universe consists of points on a line, and the resources are intervals; the goal is to select the minimum number of intervals to cover a target set of points.

This simple problem serves as the "common ancestor" for some of the problems addressed in this thesis, branching into our two main areas of focus:

- Network Design: In the context of augmentation problems, Interval Set Cover is equivalent to the Path Augmentation Problem (PAP). Here, the "line" is a path graph, the "points" are the tree edges (1-cuts on the path), and the "intervals" are candidate links that are available to be picked to cover some of the deficient cuts. Our work on the Tree Augmentation Problem (TAP) and Connectivity Augmentation Problem (CAP) can be viewed as generalizing PAP to arbitrary trees and general graphs, respectively.

- Geometric Optimization: In the geometric setting, Interval Set Cover is simply the 1-dimensional case of Geometric Set Cover. Our work on covering points with hyperrectangles extends this logic to d-dimensions, where the interplay between coordinates adds significant complexity.

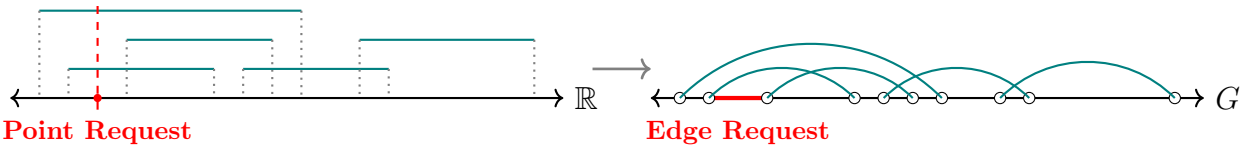


Figure 1.1: Reduction from Interval Covering to Path Augmentation. A point request on the real line corresponds to an edge request on the discretized path graph.

While we study these generalizations under different paradigms (*online* arrival for network design and *dynamic* updates for geometry), a core technique bridges the gap: a novel *interval flattening procedure*. At a high level, this procedure decomposes a given set of weighted intervals, by breaking them into smaller pieces and selecting an appropriate subset, such that they still cover the same space (i.e., union of the intervals remains the same), but now any point is covered by only a few intervals, i.e., the frequency of the system is now bounded. This technique

proves robust enough to serve as a cornerstone for our $O(\log n)$-competitive algorithms for online weighted Tree Augmentation Problem (WTAP, discussed in Chapter 5) and our polylogarithmic update times for dynamic Geometric Set Cover (discussed in Chapter 6). By identifying this shared lineage, we are able to port algorithmic insights from the simple structure of 1D intervals to more complex higher-dimensional Euclidean spaces.

1.2 Covering Deficient Cuts with Edges

The first part of this thesis addresses the problem of making networks resilient to failures. In the Connectivity Augmentation Problem (CAP), we are given a graph that is currently k-edge-connected, and a set of candidate *links* (additional edges). The goal is to select a minimum-cost subset of links such that the resulting graph becomes $(k+1)$-edge-connected.

While formulated as a graph property, CAP is fundamentally a covering problem. By Menger's Theorem [109, 60], increasing connectivity is equivalent to ensuring that there are no *cuts* (partitions of the vertices) of insufficient size. To increase connectivity from k to $k+1$, the algorithm must select links to *cover* (cross) all cuts that currently have only k edges crossing them. Thus, we are effectively solving a Set Cover instance where the universe consists of these *deficient cuts*, and the available sets are the candidate edges (links) that can cross them.

In Chapter 4, we study CAP in the online setting. In this model, the connectivity requirements are revealed over time. A sequence of requests (deficient cuts) arrives, and the algorithm must irrevocably select links to cover these cuts as they appear. Prior to this work, the best competitive ratio for online CAP was $\tilde{O}(k \log^3 n)$, achieved by a randomized algorithm. We present a deterministic algorithm that achieves an $O(\log n)$ competitive ratio, matching the theoretical lower bound up to constant factors. We also improve the competitive ratio for the weighted case (online WCAP) to $O(\log^2 n)$.

Following this, Chapter 5 is dedicated to the online Tree Augmentation Problem (TAP) and its weighted variant (online WTAP). TAP restricts the underlying graph to a tree, focusing the problem on covering the minimal deficient cuts (the edges of the tree) using non-tree links. This is the natural generalization of the Path Augmentation Problem described in Section 1.1. By stripping away the structural complexity of general graphs, we are able to deploy the interval flattening procedure directly. For online TAP, we present a self-contained $O(\log n)$-competitive algorithm. For the weighted case, we develop a randomized combinatorial algorithm, also achieving an $O(\log n)$-competitive ratio. Finally, we investigate the limits of these algorithms by proving an $\Omega(\log n)$ lower bound even when the requests arrive in random-order.

1.3 Covering Dynamic Point Sets with Hyperrectangles

Moving from graphs to geometry, the second part of the thesis examines how to maintain covering solutions when the input changes dynamically. In Chapter 6, we focus on Geometric Set Cover and Hitting Set for d-dimensional hyperrectangles.

In geometric settings, covering problems take on a spatial form: given a collection of geometric objects, such as rectangles or disks, one seeks the smallest (or minimum-weight) subset whose union covers all given points. As noted earlier, this is the high-dimensional generalization of Interval Set Cover. The Geometric Set Cover problem has been a cornerstone of computational geometry, yielding rich algorithmic techniques such as ε-nets, quasi-uniform sampling, and PTAS *(Polynomial-Time Approximation Scheme)* frameworks.

While the static versions of these problems are well understood, much less is known about their behavior in dynamic environments, where points can be inserted or deleted. In such settings, one seeks algorithms that maintain an approximately optimal cover with small update time. A fundamental open question, highlighted by Chan, He, Suri, and Xue [39], is whether polylogarithmic approximation ratios can be maintained with polylogarithmic update times.

We answer this affirmatively for hyperrectangles, presenting an algorithm that achieves a $(\log n)^{O(d)}$ approximation and update time. Our result is built upon the same technical foundation as our network design results: we adapt and generalize the interval flattening procedure. By combining the flattening technique with nested segment trees, we develop a powerful frequency reduction method that reduces complex high-dimensional geometric instances to general Set Cover instances with bounded frequency.

1.4 Covering Rectangular Spans with Segments

The next part of the thesis turns to *geometric stabbing problems*, which can be viewed as structured instances of weighted Set Cover.

Chapter 7 considers the Rectangle Stabbing problem, where the universe consists of axis-aligned rectangles, and the available resources are horizontal line segments. The goal is to select a set of segments with minimum total length such that every rectangle is *stabbed* (opposite sides are intersected) by at least one segment. In this context, a segment acts as a geometric set covering the "rectangular spans" it intersects.

Unlike the general Set Cover problem, the geometric constraints here allow for much stronger algorithmic guarantees. We develop a PTAS for the case where segments are horizontal, resolving an open problem posed in prior work [40]. Our approach combines hierarchical grid decomposition with recursive dynamic programming. The technique extends to obtain improved

results for several variants, including the case where both horizontal and vertical segments are allowed, as well as for δ-large rectangles.

In later sections of the chapter, we generalize this to study *rectilinear polygons*, specifically k-shapes (which are stacks of k rectangles). We show that while a PTAS is achievable for shapes with *bounded width variance*, the problem becomes APX-hard for general k-shapes ($k \geq 3$). This establishes a sharp complexity boundary: while simple rectangular geometry permits near-optimal approximations, slightly more complex shapes reintroduce the logarithmic hardness of general Set Cover.

1.5 Maximizing Coverage of Points with Geometric Objects

Finally, Chapter 8 explores the limits of efficient computation in geometric optimization through the Maximum Range Sum (MaxRS) problem. Given a set of weighted points, the goal is to position a geometric shape (such as a box or ball) to maximize the weight of the enclosed points.

While MaxRS is technically a maximization problem (akin to "packing"), it is intimately tied to covering via geometric duality. Finding a location for a shape Q to cover the maximum number of points is equivalent to the dual problem of finding a point p in the plane that is covered by the maximum number of "dual" shapes. We focus specifically on lower bounds for the Batched MaxRS problem. We establish that for a set of n points and m intervals, solving the problem for each interval requires $\Omega(mn)$ time, conditioned on the hardness of the $(\min, +)$-Convolution problem. Our result rules out the possibility of subquadratic algorithms for Batched MaxRS and related problems such as the Smallest k-Enclosing Interval problem, thereby establishing tight complexity barriers for this class of geometric optimization problems.

1.6 Thesis Organization

The remainder of this thesis is organized as follows:

- Chapter 2 presents a brief snapshot of the different paradigms in which covering problems have been studied.
- Chapter 3 provides the necessary preliminaries, notation, and definitions for covering LPs, graph theory, competitive analysis, and computational geometry.

- Chapter 4 details the tight deterministic algorithms for online Connectivity Augmentation Problem.
- Chapter 5 focuses on designing simple algorithms for Tree Augmentation Problem.
- Chapter 6 describes the dynamic algorithms for Geometric Set Cover and Hitting Set.
- Chapter 7 presents the PTAS for the Rectangle Stabbing problem, and further generalizes the results to rectilinear polygons.
- Chapter 8 establishes the conditional lower bounds for Batched MaxRS.
- Chapter 9 concludes with a summary of contributions and directions for future research.

Chapter 2

A Snapshot of Covering Paradigms

The fundamental Set Cover problem (finding the minimum-cost collection of sets to cover a universe of elements) is one of Karp's 21 NP-complete problems [93], and its study has involved many critical tools such as linear programming relaxations, primal-dual schemas, and probabilistic sampling.

As it is one of the central problems in theoretical computer science, over the years, the study of covering problems has expanded far beyond the search for static, polynomial-time approximation algorithms. The algorithmic landscape is now defined not just by *what* needs to be covered, but by many additional constraints: *how* the input is revealed, *when* decisions must be made, and *what* computational resources (such as time, space, or randomness) are available. This chapter provides a broad overview of these paradigms. While the subsequent chapters of this thesis will formally define and analyze specific problems within the online and dynamic regimes, the goal here is to contextualize those results within the wider spectrum of algorithmic models.

2.1 Models of Computation

We begin by examining how the availability of information and computational resources dictates the design of algorithms.

Offline, Exact, and Parameterized Models. In the classical *offline* setting, the entire input, the universe of elements and the family of available sets, is known to the algorithm upfront. Since most covering problems of interest are NP-hard, the primary research focus is often on designing *approximation algorithms* that run in polynomial time [132, 133]. The performance of these algorithms is measured by the *approximation ratio*, the worst-case ratio between the cost of the algorithm's solution and that of an optimal solution.

The approximability of covering problems varies significantly based on structure. The general Set Cover problem cannot be approximated to a factor better than $(1 - o(1)) \ln n$ unless $P = NP$ [62]. Conversely, geometric variants often admit constant-factor approximations or even *Polynomial-Time Approximation Schemes (PTASs)* (for instance Rectangle Stabbing, a topic of study of this thesis). Parallel to approximation is the study of *exact algorithms* [68] and *fixed-parameter tractability (FPT)* [51]. In scenarios where strict optimality is required, or where specific parameters are small, we can bypass general NP-hardness. For instance, while Vertex Cover is NP-hard, it is fixed-parameter tractable with respect to the solution size k, i.e., it can be solved exactly in time $O(1.253^k + kn)$ [86]. This lens allows us to classify covering problems not just by their worst-case hardness, but by which structural parameters make them tractable.

Online Algorithms. In many real-world applications, such as assigning servers to requests or routing in telecommunication networks, data arrives sequentially, and decisions must be made immediately. In the online model, the input is revealed piece-by-piece. Upon receiving a new request (an element to cover), the algorithm must irrevocably select resources (sets) to satisfy it. The standard measure of performance is the *competitive ratio*, defined as the maximum ratio between the cost of the online algorithm's solution and that of an optimal offline solution that knows the entire sequence in advance [24, 131].

The difficulty in this setting arises from the *information gap*: the algorithm must hedge against future, unknown requests. To analyze this rigorously, the online setting is often modeled as a game between the algorithm and an *adversary* who constructs the input sequence to maximize the algorithm's cost relative to the offline optimum. The power of this adversary is categorized by its knowledge of the algorithm's behavior: an *oblivious adversary* knows the algorithm's logic but must fix the entire sequence of requests before the game begins. In contrast, an *adaptive adversary* can observe the algorithm's past decisions and issue subsequent requests accordingly. This distinction is fundamental to the study of randomized algorithms, which can often leverage uncertainty to achieve significantly better performance against an oblivious adversary than is possible in the deterministic case.

From a technical standpoint, the *online primal-dual* framework [30] has emerged as an important design paradigm. This method has yielded tight polylogarithmic competitive ratios for general Set Cover and its variants [106].

Dynamic Algorithms. Distinct from the online model, the dynamic setting allows for both the insertion and deletion of elements (or sets). Here, the central challenge is not the irrevocability of decisions, as algorithms are permitted to modify their chosen cover, but rather the

computational cost of maintaining a valid solution. We seek to avoid recomputing the solution from scratch after every change.

The goal in dynamic algorithms is to optimize the trade-off between the *update time* (the time required to modify the solution after an input change) and the approximation ratio. Ideally, we seek algorithms with *polylogarithmic* update times that maintain a solution close to the optimum. This field distinguishes between *fully dynamic* algorithms (handling both insertions and deletions) and *partially dynamic* ones (handling only insertions or only deletions). Additionally, a secondary metric often considered is *recourse*, which measures the number of changes made to the solution set per update. Recent research has focused on obtaining fully dynamic f-approximate set cover, with polylogarithmic update times, while also optimizing the worst-case recourse [1, 17, 123].

Algorithms with Advice. Standard online analysis is often criticized for being overly pessimistic, as it assumes a worst-case adversarial input that rarely occurs in practice. To bridge the gap between theoretical hardness and practical performance, the *advice model* [110] augments the online algorithm with information regarding the future request sequence.

This paradigm can be studied through different lenses. One is that of *Advice Complexity*, which treats information as a computational resource. Here, the advice is generated by an all-knowing oracle, and the goal is to determine the minimum number of advice bits required to achieve a specific competitive ratio [27]. This information-theoretic perspective quantifies exactly how much "knowledge" is missing in the online setting.

Another, more recent lens is that of *learning-augmented algorithms*. Motivated by advances in machine learning, this model assumes the algorithm has access to a prediction of the future input (e.g., a predicted cover). Unlike the oracle model, these predictions may be erroneous. Research here focuses on designing algorithms that benefit from good predictions while remaining resilient to bad ones. The performance is measured by two competing metrics: *consistency*, the approximation ratio when predictions are perfect, and *robustness*, the worst-case bound when predictions are arbitrarily wrong [50]. Recent work has successfully extended the primal-dual framework to this setting, allowing for a smooth interpolation between these two extremes [13].

Streaming Models. In the era of "Big Data," inputs often exceed the capacity of available working memory. *Streaming algorithms* [114, 135] address this by processing the input as a continuous sequence of items, with the constraint that the algorithm can store only a tiny fraction of the data at any given time. Unlike the online model, where decisions are irrevocable, the challenge here is *space complexity*. The algorithm typically aims to compute an approximate solution in a single pass (or a small constant number of passes) over the data.

For numerical statistics (such as frequency moments), algorithms often strive for space *polylogarithmic* in the input size. However, for many problems, like the ones studied in this thesis, such strict bounds are often provably impossible. This has led to the formalization of the *semi-streaming model*, particularly for graph problems [107]. Here, if the graph has n vertices and m edges (where m can be $O(n^2)$), the algorithm is permitted $\tilde{O}(n)$ space, which is enough to store the vertices and a sparse subgraph but insufficient to store all edges.

Key techniques in this domain include *linear sketching*, which compresses high-dimensional vectors into small projections to estimate properties like cut sizes, and *coresets* [66], which extract small representative samples that preserve the covering properties of the full dataset. Research in this area explores the tight trade-offs between the number of passes, the space allocated, and the achievable approximation ratio.

Stochastic Model. In the stochastic model, uncertainty is addressed by assuming the input is not adversarial, but rather a realization of a random variable drawn from a known probability distribution. Unlike the online model, which must hedge against any possible sequence, the goal here is to leverage prior statistical knowledge to minimize the *expected cost* over all possible outcomes.

A common framework within this paradigm is the two-stage recourse model [124, 83]. Here, the algorithm must construct an initial partial solution (Stage I) based only on the distribution of potential inputs. Once the actual scenario is realized (drawing specific demands from the distribution) the algorithm may augment the solution (Stage II) to satisfy the covering constraints. The objective is to balance the deterministic cost of the first-stage elements against the expected cost of the second-stage recourse actions, which are typically modeled as being more expensive than planning ahead.

Partial Covering and Prize-Collecting. Classic covering problems impose a strict requirement: every single demand must be satisfied, regardless of the cost. In practice, however, data may contain outliers, which are isolated requests that are disproportionately expensive to serve. To address this, we relax the covering mandate through two related frameworks.

In *partial covering*, the strict "all-or-nothing" constraint is replaced by a quota: the algorithm must cover at least k elements (or a specified fraction of the total weight). This introduces a "packing" aspect to the covering problem; the algorithm must identify the most "profitable" subset of elements to cover while minimizing resource cost. Note that such variants can be significantly harder than the standard problem; for instance, while Vertex Cover is FPT, Partial Vertex Cover is $W[1]$-complete [78].

Closely related is the *prize-collecting* framework, which internalizes the decision to cover or

discard. Here, every element is associated with a penalty. The algorithm may choose to leave an element uncovered by paying its penalty, and the objective becomes minimizing the sum of the resource costs plus the penalties of uncovered elements [74, 69, 100]. This model captures the reality of domains like logistics and network design, providing a formal way to "bypass" requirements that are too expensive to satisfy (such as connecting a highly remote terminal) by instead paying a representative penalty.

Fairness and Coloring Constraints. Standard combinatorial optimization typically seeks to minimize the *total* cost, ignoring how that cost or service quality is distributed among individual participants. Motivated by algorithmic bias and equity concerns, modern research imposes *fairness* constraints. In covering problems, this often entails partitioning the universe into distinct groups (e.g., demographic categories) and requiring a minimum level of coverage for *each* group, rather than just the aggregate. This prevents solutions that efficiently cover the majority while neglecting minority groups.

A closely related domain is the study of *coloring constraints*. Here, the resources are not uniform; instead, they are associated with specific "colors" representing different types, origins, or properties. In variants such as Colorful Vertex Cover, the algorithm is tasked with finding a cover that simultaneously satisfies specific quotas for each color class. This framework is essential for modeling multi-objective covering tasks, where one must satisfy various requirements simultaneously, such as ensuring a network uses equipment from a diverse set of vendors to maintain reliability. Because these constraints force the algorithm to choose potentially sub-optimal resources to satisfy color quotas, these problems are often computationally more complex than their standard counterparts [108, 33].

2.2 Problem Families

We now look at some specific problem families, and how they relate to covering problems.

Graph Covers (Vertex and Edge Cover). Some simple restrictions of Set Cover arise in graph theory, where the incidence structure is fixed by the underlying graph. The two most prominent variants illustrate the sharp contrast in complexity often found in this field. Vertex Cover asks for a subset of vertices to "cover" (be incident to) all edges. It is a classic NP-hard problem used extensively for reductions. Conversely, Edge Cover asks for a subset of edges to cover all vertices. Unlike its sibling, Edge Cover can be solved exactly in polynomial time via maximum matching algorithms. These problems serve as the foundation for more complex variants such as Capacitated Vertex Cover, where vertices have limits on how many edges they can cover, adding a layer of resource allocation to the problem [54, 53].

Network Design Variants. Network design transforms covering from a local constraint (covering a specific vertex) to a global connectivity requirement. The universe here is often the set of all *cuts* in a graph. Since a graph with n vertices has 2^{n-1} cuts, the universe is exponentially large, distinguishing these problems from standard Set Cover where the universe is polynomial.

The Survivable Network Design Problem (SNDP) generalizes this by requiring that every pair of nodes (u, v) be separated by at least r_{uv} edge-disjoint paths. By Menger's Theorem, this is equivalent to covering all u-v cuts with at least r_{uv} edges. Other classic variants include Steiner Tree and Steiner Forest, which seek to connect specific subsets of terminals. These problems are central to the first part of this thesis, as problems like TAP and CAP are essentially specialized instances of covering cuts to increase connectivity [94, 79].

Geometric Covering. In computational geometry, the input is defined by coordinates and shapes rather than arbitrary adjacency lists. This spatial structure is a powerful asset: while general Set Cover is hard to approximate, geometric instances often possess bounded VC-dimension. This property allows for the use of powerful sampling techniques, such as ε-nets and quasi-uniform sampling, to achieve constant-factor approximations or even PTASs [29, 49, 113, 118].

A sub-family within this domain involves intersection and stabbing problems. In Rectangle Stabbing, for instance, the elements to be covered are axis-aligned rectangles, and the resources are line segments. A rectangle is considered "covered" if it is stabbed by a selected segment, i.e., the segment intersects opposite sides of the rectangle. This problem illustrates how geometric structure can be exploited; the grid-like intersection patterns of axis-aligned objects allow for the use of recursive decomposition and dynamic programming, which often yield much tighter approximation bounds than are possible for general sets.

Geometric covering also extends to continuous domains through visibility-based problems. In the Art Gallery and Terrain Guarding paradigms, the goal is to place a minimum number of guards to visually cover a polygon or terrain. These problems bridge the gap between discrete optimization and continuous visibility analysis, often requiring discretization arguments to reduce the infinite search space to a finite Set Cover instance [2, 129].

Bin Packing and Configuration LPs. While colloquially framed as a packing task, Bin Packing and its generalizations are fundamentally covering problems: the objective is to *cover* a set of items using a minimum number of bins. This perspective is formalized through the *Configuration LP*, a seminal formulation dating back to Gilmore and Gomory [73], where each variable represents a feasible subset of items that can fit within a single bin. While

the 1-dimensional case admits near-optimal approximations, the problem becomes significantly more complex in multidimensional settings. A landmark result by Bansal, Caprara, and Sviridenko [14] established a general framework for solving such set-covering problems called the "round-and-approx" framework. Their approach successfully broke some long-standing barriers like the 2-approximation barrier in 2D vector packing, and also showed that their method can be generalized to solve other geometric set-covering problems.

Facility Location and Clustering. The Facility Location problem can be thought of as a bridge between pure covering problems and clustering. In its classical uncapacitated form (UFLP), the goal is to select a set of facilities to open (incurring an opening cost) such that every client is connected to an open facility. Unlike Set Cover, where a client is either "covered" or not, Facility Location introduces a *connection cost* proportional to the distance between the client and the facility.

This formulation can be viewed as a modified version of Set Cover: rather than enforcing a hard constraint that a client must be within a specific radius of a facility (as in the Set Covering Location Problem), the algorithm pays a continuous penalty for distance [92, 104].

Paging and k-Server. While the problems in this thesis focus on *accumulation*, a setting where resources (such as edges) are purchased permanently, a distinct class of covering problems deals with *movement* and *eviction*. The most fundamental of these is Paging (or Caching), where a cache of size k must "cover" a sequence of page requests. When a requested page is not in the cache, the algorithm must incur a cost to retrieve it, potentially evicting another page to make room [134].

This concept generalizes to metric spaces in the form of the k-Server Problem. Here, k mobile servers reside in a metric space, and requests appear at various locations. To cover a request, a server must move to that location, incurring a cost equal to the distance traveled. These problems are central to the study of online algorithms because they model reversible decisions and resource constraints [103].

Chapter 3

Preliminaries

In this chapter, we define the standard notation and fundamental concepts used throughout this thesis. We begin with general asymptotic notation and the formal definitions of the computational models studied, specifically the online and dynamic settings. We then introduce the formal structure of generic covering problems, including their Linear Programming (LP) formulations and duals. Following this, we introduce the specific graph-theoretic structures required for the network design chapters, and the geometric definitions and data structures required for the geometric optimization chapters.

3.1 General Notation and Analysis

3.1.1 Asymptotic Notation

We use standard asymptotic notation to describe the resource consumption of algorithms. Let $f(n)$ and $g(n)$ be non-negative functions.

- $f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Equivalently, $\limsup_{n\to\infty} \frac{f(n)}{g(n)} < c$.
- $f(n) = \Omega(g(n))$ if $g(n) = O(f(n))$.
- $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.
- $f(n) = o(g(n))$ if for every constant $c > 0$, there exists $n_0 \geq 0$ such that $f(n) < c \cdot g(n)$ for all $n \geq n_0$. Equivalently, $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.
- $f(n) = \omega(g(n))$ if $g(n) = o(f(n))$.

Common Complexity Bounds. Throughout this thesis, we frequently characterize the running times and space requirements of our algorithms using the following standard classes of functions, defined with respect to the input size n:

- *Polylogarithmic:* A function bounded by $O(\log^c n)$ for some constant $c > 0$. We denote this class of functions as $\text{polylog}(n)$.
- *Polynomial:* A function bounded by $O(n^c)$ for some constant $c > 0$. Algorithms running in polynomial time are generally considered efficient.
- *Quasipolynomial:* A function bounded by $2^{O(\log^c n)}$ for some constant $c > 1$. Equivalently, this can be written as $n^{\text{polylog}(n)}$. While strictly slower than polynomial-time, quasipolynomial-time is significantly faster than purely exponential time (e.g., $2^{O(n)}$).
- *Tilde O notation:* The notation $\tilde{O}(f(n))$ is used to suppress polylogarithmic factors, i.e., $\tilde{O}(f(n)) = O(f(n) \cdot \text{polylog}(n))$.

3.1.2 Approximation Algorithms (Offline Setting)

For offline minimization problems, where the entire input is known in advance, we measure the performance of an algorithm by its *approximation ratio*.

Definition 3.1 (Approximation Ratio)**.** *Let Π be a minimization problem and let $\mathcal{A}$ be an algorithm for Π. Let $\mathsf{ALG}(I)$ denote the cost of the solution produced by $\mathcal{A}$ on input instance I, and let $\mathsf{OPT}(I)$ denote the cost of an optimal solution for I. We say that $\mathcal{A}$ is an* α-approximation algorithm *(or achieves an approximation ratio of α) if for all instances I:*

$$\mathsf{ALG}(I) \leq \alpha \cdot \mathsf{OPT}(I)$$

where $\alpha \geq 1$. For maximization problems, the inequality is reversed ($\mathsf{ALG}(I) \geq \frac{1}{\alpha}\mathsf{OPT}(I)$) or defined as $\mathsf{OPT}(I) \leq \alpha \cdot \mathsf{ALG}(I)$.

Polynomial-Time Approximation Scheme (PTAS). Some NP-hard optimization problems allow for approximation factors arbitrarily close to 1.

Definition 3.2 (PTAS)**.** *A problem admits a* Polynomial-Time Approximation Scheme (PTAS) *if there exists an algorithm $\mathcal{A}$ taking an instance I and a parameter $\epsilon > 0$ as input, such that $\mathcal{A}$ produces a $(1+\epsilon)$-approximation, and for every fixed ϵ, the running time of $\mathcal{A}$ is polynomial in $n = |I|$ (e.g., $O(n^{1/\epsilon})$).*

If the running time is also polynomial in $1/\epsilon$ (e.g., $O(n^c(1/\epsilon)^k)$), the scheme is called a *Fully Polynomial-Time Approximation Scheme (FPTAS)*. If the running time is quasipolynomial in n (e.g., $n^{(\log(n)/\varepsilon)^c}$), it is a *Quasi-Polynomial-Time Approximation Scheme (QPTAS)*.

APX-Hardness. The class APX (Approximable) consists of NP optimization problems that admit polynomial-time approximation algorithms with some constant approximation ratio. A problem is said to be APX-hard if every problem in APX can be reduced to it (via an approximation-preserving reduction, such as an L-reduction). If a problem is APX-hard, it implies that there exists a constant $c > 1$ such that it is NP-hard to approximate the problem within a factor better than c. Consequently, unless P = NP, an APX-hard problem does not admit a PTAS. For more information on these classes, and reductions between them, one can refer to the standard textbooks on approximation algorithms [132, 133].

3.1.3 Online Algorithms

For online problems, the input is not known in advance but arrives in a sequence $\sigma = \langle \sigma_1, \sigma_2, \dots \rangle$. Upon the arrival of each request σ_i, an online algorithm ALG must make an irrevocable decision to satisfy the request without knowledge of future requests. Performance of online algorithms is measured using the *competitive ratio*.

Definition 3.3 (Competitive Ratio)**.** *Let* ALG *be an online algorithm and* OPT *be the optimal offline algorithm. Let* $\mathsf{ALG}(\sigma)$ *and* $\mathsf{OPT}(\sigma)$ *denote the costs of the solutions produced by* ALG *and* OPT *respectively on input sequence* σ*. We say* ALG *is* c-competitive *if for all valid input sequences* σ*:*

$$\mathsf{ALG}(\sigma) \leq c \cdot \mathsf{OPT}(\sigma) + \alpha$$

where α *is a constant independent of the input size. For randomized algorithms, we require that the expected cost satisfies the bound:*

$$\mathbb{E}[\mathsf{ALG}(\sigma)] \leq c \cdot \mathsf{OPT}(\sigma) + \alpha$$

where the expectation is taken over the internal random choices of the algorithm.

The Adversarial Model. In the standard online model, we assume the input sequence is generated by an *adversary* that knows the algorithm's strategy and aims to maximize the ratio between the algorithm's cost and the optimal cost. For randomized algorithms, we typically assume an *oblivious adversary*, who fixes the entire input sequence in advance based on the algorithm's code but not its random outcomes, rather than an *adaptive adversary*, who can react to the algorithm's random choices during execution.

The Random-Order Model. In certain scenarios, the worst-case assumption of the adversarial model is too pessimistic. In the *random-order model*, the adversary selects the set of requests, but they are presented to the algorithm in a uniformly random permutation. Here, the competitive ratio is defined in expectation: $\mathbb{E}[\mathsf{ALG}(\sigma)] \leq c \cdot \mathsf{OPT}(\sigma) + \alpha$, where the expectation is taken over the $n!$ possible permutations of the input sequence.

Other Models. Beyond these settings, the literature also studies the *stochastic model*, where requests are drawn i.i.d. from a probability distribution, and the model of *algorithms with predictions*, where the algorithm has access to imperfect information about the future sequence. Further, the model of *online algorithms with recourse* is considered, where the algorithm is permitted to modify or update a limited number of previous decisions upon the arrival of a new request.

3.1.4 Dynamic Algorithms

Distinct from the online setting, *dynamic algorithms* handle problems where the input data changes over time through updates, such as insertions and deletions. The goal is to maintain a valid solution (or data structure) that approximates the optimum at all times while minimizing the computational overhead per update.

Fully vs. Partially Dynamic. An algorithm is *fully dynamic* if it supports both insertions and deletions of elements. It is *incremental* if it supports only insertions, and *decremental* if it supports only deletions.

Preprocessing Time. Before any updates are processed, the algorithm typically constructs an initial representation of the problem from a starting input instance of size n. The time required for this setup is the *preprocessing time*. Ideally, we require the preprocessing time to be polynomial in n, ensuring that the initial construction is computationally feasible.

Update Time. The primary performance metric in this setting is the *update time*, which is the time required to modify the solution after an input change.

- *Worst-case update time:* The maximum time required for any single update.
- *Amortized update time:* The average time per update over a worst-case sequence of k operations. If a sequence of k operations takes $T(k)$ total time, the amortized time is $T(k)/k$.

In this thesis, we primarily seek algorithms with *polylogarithmic* update times while maintaining a polylogarithmic approximation ratio.

3.2 The Anatomy of Covering Problems

A large portion of this thesis deals with problems that can be framed as Integer Linear Programs (ILP) of a specific form.

3.2.1 Generic Covering Formulation

In a generic covering problem, we are given a universe of elements $U = \{e_1, \ldots, e_n\}$ and a family of sets $\mathcal{S} = \{S_1, \ldots, S_m\}$ where each $S \in \mathcal{S}$ has an associated cost c_S. The objective is to minimize the cost of sets selected such that every element e_i is covered at least k_i times (in the standard Set Cover problem, $k_i = 1$).

This can be formulated as an Integer Linear Program. Let $x_S \in \{0, 1\}$ indicate whether set S is selected.

$$\begin{aligned} \text{Minimize} \quad & \sum_{S \in \mathcal{S}} c_S x_S \\ \text{Subject to} \quad & \sum_{S : e \in S} x_S \geq k_e \quad \forall e \in U \\ & x_S \in \{0, 1\} \quad \forall S \in \mathcal{S} \end{aligned}$$

3.2.2 The Covering LP and its Dual

The Linear Programming relaxation allows x_S to take fractional values in $[0, 1]$. This relaxation is crucial for the design of approximation algorithms.

Primal (Covering) LP:

$$\begin{aligned} \text{Minimize} \quad & \sum_{S \in \mathcal{S}} c_S x_S \\ \text{Subject to} \quad & \sum_{S : e \in S} x_S \geq k_e \quad \forall e \in U \\ & x_S \geq 0 \quad \forall S \in \mathcal{S} \end{aligned}$$

Associated with every Linear Program is a *dual LP*. For covering problems, the dual is a *packing LP*. Here, we assign a variable y_e to every element $e \in U$.

Dual (Packing) LP:

$$\text{Maximize} \quad \sum_{e \in U} k_e y_e$$

$$\text{Subject to} \quad \sum_{e \in S} y_e \leq c_S \quad \forall S \in \mathcal{S}$$
$$y_e \geq 0 \quad \forall e \in U$$

Intuitively, the dual problem asks us to assign a "price" y_e to each element such that the total price of elements contained in any set S does not exceed the cost of that set. By weak duality, any feasible solution to the Dual LP provides a lower bound on the cost of the Primal LP (and thus the optimal integral solution).

3.3 Graph Theory and Connectivity

We denote a graph by $G = (V, E)$, where V is the set of vertices and E is the set of edges. Given a graph G, we will occasionally use $V(G)$ to refer to its vertex set, and $E(G)$ to refer to its edge set. For a set of edges L, we denote $G \cup L = (V, E \cup L)$.

3.3.1 Connectivity and Cuts

Two vertices $u, v \in V$ are *k-edge-connected* if the removal of any set of fewer than k edges does not disconnect them. The fundamental relationship between edge connectivity and paths is established by Menger's Theorem.

Theorem 3.1 (Menger's Theorem)**.** *Let $G = (V, E)$ be an undirected graph and let $u, v \in V$. The maximum number of edge-disjoint paths between u and v is equal to the minimum number of edges whose removal disconnects u and v.*

By Menger's Theorem, two vertices are k-edge-connected if and only if there exist k edge-disjoint paths between them. A graph G is k-edge-connected if every pair of vertices in G is k-edge-connected.

Formally, a *cut* is a partition of the vertex set V into two non-empty disjoint sets $(S, V \setminus S)$. For any subset $S \subseteq V$, we denote the set of edges with exactly one endpoint in S and the other in $V \setminus S$ by $\delta(S)$. The *size* of a cut is the number of crossing edges, $|\delta(S)|$. In a k-edge-connected graph, any cut of size k is called a *minimum cut* (or min-cut).

Throughout this thesis, we adopt the following standard terminology and conventions. We define a *p-cut* as a cut of size exactly p (i.e., $|\delta(S)| = p$); for example, a 2-cut is a cut where exactly two edges cross the partition. Furthermore, we frequently employ common abuses of notation by identifying a cut with either its *shore* $S \subseteq V$ or its set of crossing edges $\delta(S)$. Thus, we may write "let $C \subseteq V$ be a cut" to refer to the partition $(C, V \setminus C)$, and similarly, we may refer to the set of edges $\delta(S)$ simply as "the 2-cut $\{e_1, e_2\}$".

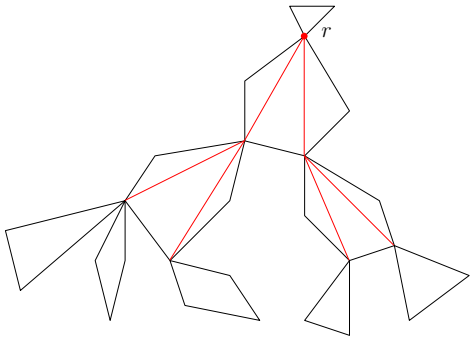


Figure 3.1: Example cactus, and the tree obtained on articulation vertices.

A set of edges L is said to *cover* a cut S if adding L to the graph increases the size of the cut.

3.3.2 Cactus Graphs

A central structure in our analysis of Connectivity Augmentation is the cactus graph.

Definition 3.4 (Cactus Graph and Articulation Vertex)**.** *A* cactus *is a connected graph in which every edge lies in exactly one cycle. A vertex that lies on two or more cycles of the cactus is called an* articulation vertex*.*

A cactus can be viewed as a tree of cycles connected at articulation vertices. This tree-like structure of cycles allows for efficient decomposition of connectivity problems. In Figure 3.1, we can see an example cactus graph, with the tree structure on the articulation vertices highlighted in red.

Remark 3.1. *We note a slight deviation from the definition of cactus graph in some of the existing literature. A cactus graph is frequently defined more broadly as a connected graph where every edge belongs to* at most *one simple cycle, which also includes structures like trees. In this work, we adopt the more restricted definition where every edge must lie in* exactly *one cycle. Consequently, all cactus graphs considered in this thesis are inherently 2-edge-connected and contain no bridge edges.*

3.4 Geometric Optimization Fundamentals

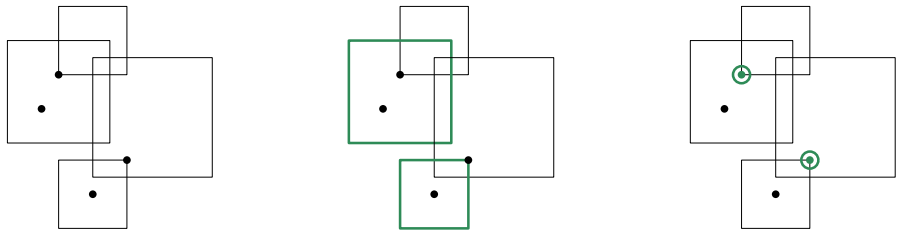
3.4.1 Geometric Objects

We operate in $\mathbb{R}^d$.

- *Hyperrectangles:* An axis-aligned hyperrectangle is the Cartesian product of d intervals $[a_1, b_1] \times \cdots \times [a_d, b_d]$. For $d = 2$, these are rectangles.
- *d-Balls:* A d-dimensional ball (or d-ball) with center $c \in \mathbb{R}^d$ and radius $r \geq 0$ is the set of all points at a distance at most r from c, i.e., $\{p \in \mathbb{R}^d \mid \|p - c\|_2 \leq r\}$.

3.4.2 Geometric Set Cover and Geometric Hitting Set

- Geometric Set Cover: Given a set of points P and a family of geometric shapes $\mathcal{S}$, select a minimum cardinality subset $\mathcal{S}' \subseteq \mathcal{S}$ such that every point in P is contained in at least one shape in $\mathcal{S}'$ (see Figure 3.2 for an illustration).
- Geometric Hitting Set: Given a set of geometric shapes $\mathcal{S}$ and a set of candidate points P, select a minimum cardinality subset $P' \subseteq P$ such that every shape in $\mathcal{S}$ contains at least one point from P'.



(a) (b) (c)

Figure 3.2: (a) A set of squares $\mathcal{S}$ and a set of points P, (b) A set cover (in green) $\mathcal{S}' \subseteq \mathcal{S}$ covering P, (c) A hitting set (green points) $P' \subseteq P$ for $\mathcal{S}$.

3.4.3 Geometric Stabbing

We now look at the notion of *geometric stabbing*, which can be viewed as structured instances of the general weighted Set Cover problem.

Definition 3.5 (Stabbing a Rectangle)**.** *Let $R = [x_1, x_2] \times [y_1, y_2]$ be an axis-aligned rectangle in the plane. A horizontal line segment $\ell = [x_a, x_b] \times \{y\}$ is said to* stab *R if it completely traverses the rectangle from its left boundary to its right boundary. Formally, this requires that $y_1 \leq y \leq y_2$ and $x_a \leq x_1 \leq x_2 \leq x_b$.*

Definition 3.6 (Rectangle Stabbing)**.** *In the fundamental horizontal Rectangle Stabbing problem, we are given a set of axis-aligned rectangles $\mathcal{R}$ in the plane. The goal is to select a set of horizontal line segments $\mathcal{S}$ of minimum total length (sum of the lengths of all segments) such that every rectangle $R \in \mathcal{R}$ is stabbed by at least one segment in $\mathcal{S}$.*

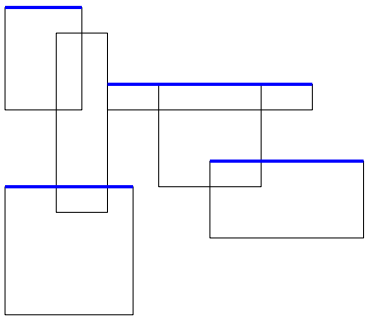


Figure 3.3: An instance of horizontal RECTANGLE STABBING. The horizontal segments (in blue) stab the rectangles by fully traversing their widths.

3.5 Geometric Data Structures

We introduce some classical geometric data structures, which are used to manage spatial data efficiently.

3.5.1 Interval Trees

An *interval tree* is a data structure used to store a set of intervals $\mathcal{I}$ on the real line to support efficient stabbing queries (reporting all intervals that contain a query point q). In this thesis, we utilize the structural properties of interval trees to decompose the set of intervals into a hierarchy based on containment depth.

Construction. The tree is built recursively: given a range, the root stores the median coordinate x_{mid} and all intervals in $\mathcal{I}$ that contain x_{mid}. The remaining intervals (those strictly to the left or right of x_{mid}) are stored recursively in the left and right subtrees. See Figure 3.4 for an example.

Query. Querying an interval tree for a point q takes $O(\log n + k)$ time, where n is the number of intervals and k is the output size.

3.5.2 Segment Trees

A *segment tree* is a data structure designed to store a set of intervals (or segments) over a fixed set of points (called elementary coordinates). Unlike interval trees, segment trees are based on a fixed partitioning of the underlying space.

Construction. We first sort the unique endpoints of the input intervals to partition the real line into a set of elementary intervals. The segment tree is a balanced binary tree built on top of these elementary intervals. Each leaf corresponds to an elementary interval, and each internal node represents the union of the intervals in its subtree, referred to as a *canonical interval*.

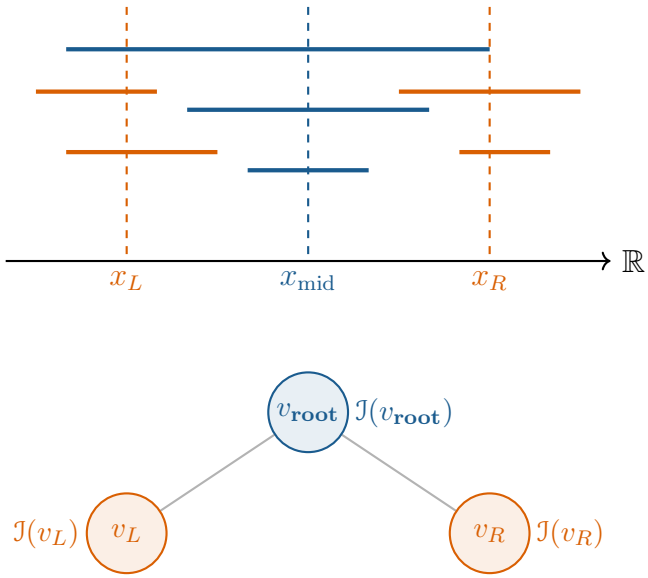


Figure 3.4: The hierarchical structure of an interval tree. Intervals that contain the median x_{mid} are stored in the root node (blue). Intervals entirely to the left or right are handled recursively by the children (orange), assigned to nodes based on the secondary medians (x_L, x_R) they intersect.

Canonical Decomposition. A key property of the segment tree is that any arbitrary input interval can be uniquely decomposed into a disjoint union of $O(\log N)$ canonical intervals (nodes of the tree), where N is the number of elementary coordinates. See Figure 3.5 for an example.

In our dynamic Set Cover algorithms (Chapter 6), we use nested segment trees (segment trees of segment trees) to generalize our results from 1-dimensional intervals to d-dimensional hyperrectangles, leveraging this canonical decomposition to handle weights efficiently.

3.6 Fine-Grained Complexity

Classical complexity theory, centered around the P vs. NP question, classifies problems broadly by whether they admit polynomial-time algorithms. Fine-grained complexity further attempts to pinpoint the exact asymptotic complexities of polynomial-time solvable problems, by establishing conditional lower bounds based on a few well-established hypotheses. The most famous of these include the Strong Exponential Time Hypothesis (SETH), the 3SUM conjecture, and the All-Pairs Shortest Path (APSP) conjecture. If any problem shown to be hard under these hypotheses is solved faster than its conjectured lower bound, it would lead to breakthrough algorithms for a massive class of other problems.

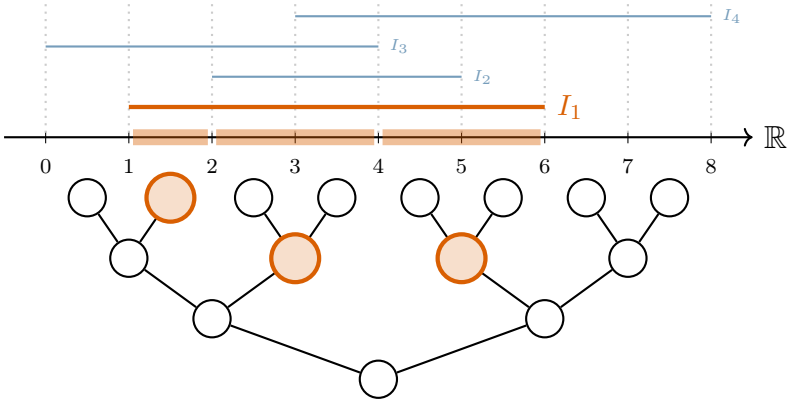


Figure 3.5: Structure of a segment tree. An input interval $I_1 = [1, 6]$ is decomposed into a set of canonical nodes (highlighted in orange).

3.6.1 The $(\min, +)$-Convolution Hypothesis

For some of our geometric lower bounds, we base our hardness on the $(\min, +)$-Convolution problem. This is another fundamental problem, and its relation with other major conjectures in the field has been studied by Cygan, Mucha, Wegrzycki, and Wlodarczyk [52].

Definition 3.7 ($(\min, +)$-Convolution)**.** *Given two sequences of integers $A = (a_0, \ldots, a_{n-1})$ and $B = (b_0, \ldots, b_{n-1})$, their $(\min, +)$-Convolution is a sequence C where:*

$$c_k = \min_{i+j=k} (a_i + b_j)$$

Computing this sequence trivially takes $O(n^2)$ time by evaluating all pairs. While slight improvements exist, such as the $O(n^2/\text{polylog}(n))$ time algorithm by Bremner, Chan, Demaine, Erickson, Hurtado, Iacono, Langerman, Pătraşcu, and Taslakian [28], it is widely believed that no algorithm can achieve a strictly subquadratic running time.

Conjecture 3.1 ($(\min, +)$-Convolution Conjecture)**.** *For any $\epsilon > 0$, there is no randomized algorithm that computes the $(\min, +)$-Convolution of two sequences of length n in $O(n^{2-\varepsilon})$ time.*

In Chapter 8, we rely on this conjecture to rule out $O(\text{polylog}(n))$ query-time data structures for batched geometric optimization problems by showing they would imply an $O(n^{2-\varepsilon})$ algorithm for $(\min, +)$-Convolution.

Chapter 4

Algorithms for Connectivity Augmentation

4.1 Introduction

Networks deployed in the real world are prone to failures. Fault-tolerant network design aims to construct networks that maintain a desired level of connectivity between nodes or groups of nodes, while being resilient to a certain number of node or edge failures. For example, if a base network represents a national fiber-optic backbone or a rural power grid, augmenting its connectivity ensures that no single severed cable or fallen tree can isolate a city. In a graph G, we say that two vertices u and v are k-edge-connected if deleting any set of fewer than k edges does not disconnect them. Equivalently, by Menger's theorem, u and v are k-edge-connected if and only if there exist at least k edge-disjoint paths between them. Furthermore, a graph is said to be k-edge-connected if every pair of distinct vertices in the graph is k-edge-connected. One of the most fundamental and extensively studied problems in fault-tolerant network design is the Connectivity Augmentation Problem (CAP). In CAP, the input consists of an undirected k-edge-connected graph G with n vertices and a set of additional edges (called links) $\mathcal{L} \subseteq \binom{V(G)}{2}$. The goal is to find a subset $\mathcal{L}' \subseteq \mathcal{L}$ of minimum cardinality such that every pair of distinct vertices is $(k+1)$-edge-connected in the augmented graph $G \cup \mathcal{L}'$; i.e., $G \cup \mathcal{L}'$ is $(k+1)$-edge-connected.

In this work, we initiate the study of CAP in the *online setting*, where we are given an undirected k-edge-connected graph G along with a set of links $\mathcal{L}$, and online requests arrive one by one, each specifying two vertices that need to be $(k+1)$-edge-connected. We start with the graph G and progressively add links to serve these requests. More specifically, upon the arrival of a request $\{u, v\}$, we must *immediately and irrevocably* add zero or more links from $\mathcal{L}$ to G

so that u and v are $(k+1)$-edge-connected in the resulting augmented graph. The goal is to minimize the total number of links added, and we evaluate an algorithm's performance by its *competitive ratio* relative to an optimal offline solution.

In contrast to the unweighted setting, the weighted version, denoted WCAP, assigns non-negative weights to links, and the objective is instead to minimize the total weight of the links added.

CAP, and consequently its weighted variant WCAP, are APX-hard even for $k = 1$, and have been extensively studied in the context of approximation algorithms. The special case of CAP when $k = 1$ is known as the Tree Augmentation Problem (TAP), since in this case, any cycle in the input graph can be safely contracted to a single vertex, as cycles are already 2-edge-connected; repeatedly contracting such cycles reduces the input graph to a tree. This case has also been extensively studied. The corresponding weighted variant is referred to as weighted TAP (WTAP). For all four problems: CAP, WCAP, TAP, and WTAP, multiple 2-approximations (see, for example, [91]) were long known, and improving this ratio was considered a significant challenge. In a long line of research, progressively better-than-2 approximation algorithms have been developed [3, 45, 44, 64, 67, 76, 102, 101, 115, 32, 34, 126]. The current best-known approximation ratio is 1.393 for both CAP and TAP [34], and $1.5 + \varepsilon$ for WCAP [128] and WTAP [127], for any constant $\varepsilon > 0$.

The online version of connectivity augmentation has previously been studied only for the special case $k = 1$, where Naor, Umboh, and Williamson [117] presented a deterministic $O(\log n)$-competitive algorithm for online WTAP. This guarantee is tight up to constant factors. For arbitrary values of k, bounds can be inferred from a more general problem studied by Gupta, Krishnaswamy, and Ravi [81]: the online Survivable Network Design Problem (SNDP). In their setting, the underlying graph is fixed with non-negative edge weights, and online requests arrive in the form of vertex pairs $\{s_i, t_i\}$, each with an associated edge-connectivity requirement r_i. The task is to maintain a minimum-cost subgraph that satisfies the required number of edge-disjoint paths for all requests seen so far. They provide a randomized $\tilde{O}(r_{\max} \log^3 n)$-competitive algorithm, where $r_{\max} = \max_i r_i$. Observe that WCAP can be embedded as a special case by treating the given k-edge-connected graph as a fixed subgraph with zero-weight edges and the links as additional weighted edges, with all edge-connectivity requirements equal to $k + 1$. To avoid parallel edges that could arise from superimposing the fixed graph and the links, we can subdivide each edge of the original graph into two zero-weight edges. Under this embedding, their result yields a $\tilde{O}(k \log^3 n)$-competitive algorithm for online WCAP.

4.1.1 Our Results

We obtain tight or near-tight bounds for several online variants of the connectivity augmentation problem, including its weighted version and the random-order model. Our main results are summarized below.

- **Online CAP.** We present a deterministic $O(\log n)$-competitive algorithm for online CAP. This improves upon the previously known randomized $\tilde{O}(k \log^3 n)$ bound and matches the known $\Omega(\log n)$ lower bound up to constant factors.

  Theorem 4.1. *There exists a deterministic $O(\log n)$-competitive algorithm for online CAP.*

- **Online WCAP.** We give a deterministic $O(\log^2 n)$-competitive algorithm for online WCAP, improving on the prior randomized $\tilde{O}(k \log^3 n)$ bound.

  Theorem 4.2. *There exists a deterministic $O(\log^2 n)$-competitive algorithm for online WCAP.*

It is worth noting that the previous competitive ratio for WCAP, $\tilde{O}(k \log^3 n)$, scaled linearly with k. Our results eliminate this dependence on k entirely. Moreover, our online algorithms run in polynomial time, unlike the earlier method for WCAP, whose running time was not guaranteed to be polynomial when $k = \omega(1)$.

4.1.2 Our Techniques

We obtain our upper bounds for online CAP and WCAP by first leveraging a classical result of Dinitz, Karzanov, and Lomonosov [61], which reduces any k-edge-connected graph to a cactus while preserving all global minimum cuts. This enables a reduction to the Cactus Augmentation Problem (CacAP). CacAP is a special case of CAP with $k = 2$ where the input graph is a cactus, i.e., a connected graph in which every edge lies on exactly one cycle. The reduction is known to be approximation-preserving in the offline setting; we show that it can be adapted to the online setting while preserving competitiveness.

To solve online CacAP and its weighted version (WCacAP), we draw inspiration from an offline reduction of CAP to the Steiner Tree problem [15, 32]. Our reduction follows a similar high-level approach but introduces new technical elements tailored to the online setting. Specifically, since connectivity must be augmented only for online-requested pairs (and not all vertex pairs), we reduce to the online Node-Weighted Steiner Forest (NWSF) problem.

Crucially, we include not only degree-2 vertices (as in the offline reduction) but also all articulation vertices (i.e., vertices that participate in multiple cycles of the cactus), as Steiner terminals. This refinement is essential for preserving feasibility and competitiveness.

By applying the known $O(\log^2 n)$-competitive algorithm for online NWSF [25], we obtain the same bound for online WCacAP, and consequently for online WCAP.

To obtain our $O(\log n)$-competitive algorithm for online CAP, we begin with the special case where the cactus consists of a single cycle, referred to as the online Cycle Augmentation Problem (CycAP). Here, we reduce to the online Edge-Weighted Steiner Forest (EWSF) problem, which admits a deterministic $O(\log n)$-competitive algorithm [16]. The reduction is facilitated by the fact that all Steiner nodes (i.e., links) have degree 2 and unit weight, allowing us to simulate node-weighted selection through edge-weighted selection with only a constant-factor loss in competitiveness.

Our key technical contribution lies in handling the general CacAP instance. Since a cactus graph can be viewed as a tree of cycles joined at articulation vertices, we reduce the full problem to (i) one instance of online TAP over a tree defined by these articulation vertices, and (ii) one instance of CycAP for each cycle in the cactus. These instances are solved independently using known algorithms for online TAP and online Steiner Forest. The idea is that the algorithm handling the cycle instances performs local optimizations within individual cycles, whereas the algorithm for the tree instance manages global, long-range optimizations across the cactus structure. We carefully analyze how each request decomposes into subrequests handled by the tree and the cycles, and show via a charging argument that the combined cost is within $O(\log n)$ of the offline optimum. This decomposition and analysis are the central innovations of our work and yield a tight bound for online CAP.

4.1.3 Further Related Work

Connectivity Augmentation Problem and Tree Augmentation Problem are closely related to several fundamental network design problems. In the 2-Edge-Connected Spanning Subgraph (2-ECSS) problem, the goal is to find a minimum-cardinality spanning subgraph that is 2-edge-connected. Numerous better-than-2 approximation algorithms are known [98, 46, 88, 120, 70, 99, 72], for this problem, with the current best approximation factor being $\frac{5}{4}$ [26]. However, obtaining a better-than-2 approximation for the weighted version of 2-ECSS remains a major open problem. WTAP can be viewed as a special case of the weighted 2-ECSS problem.

Other well-studied problems that generalize 2-ECSS and fall under the umbrella of weighted 2-ECSS include the Matching Augmentation Problem (MAP), studied in [12, 43, 42, 71]), and the Forest Augmentation Problem (FAP), studied in [75, 87]. In MAP, the input

includes a graph with a matching as a weight-zero subgraph; for FAP, the zero-weight subgraph is a forest. The objective in both cases is to augment the input with a minimum-cost set of additional edges to achieve 2-edge connectivity. These augmentation problems and their weighted variants also present natural and compelling directions for study in the online setting.

Connectivity in the online setting has also been extensively studied, especially in the context of Steiner Tree and Steiner Forest and their variants. See, for example, [85, 25] and the references therein.

Organization. The chapter is organized as follows. Section 4.2 presents the foundational reduction from online CAP to the online Cactus Augmentation Problem (CacAP), which underpins our approach. In Section 4.3 we establish some graph theoretic foundations and properties of cactus graphs. Following this, Section 4.4 details our deterministic $O(\log n)$-competitive algorithm for online CAP, which utilizes a decomposition framework that reduces the problem to cycle and tree augmentation; the tree augmentation component is further detailed in Chapter 5. In Section 4.5, we provide the $O(\log^2 n)$-competitive algorithm for the weighted setting (WCAP) and the specific reduction for cycle augmentation (CycAP).

4.2 Reduction from Online CAP to Online CacAP

In this section, we prove the following proposition.

Proposition 4.1. *If there exists a c-competitive online algorithm for the Cactus Augmentation Problem (CacAP or WCacAP), then there exists a c-competitive online algorithm for the Connectivity Augmentation Problem (CAP or WCAP, respectively).*

To establish the above proposition, we first recall a classical result of Dinitz, Karzanov, and Lomonosov [61], as stated in Theorem 3.1 of Traub and Zenklusen [128].

Theorem 4.3 ([61])**.** *For any undirected graph $G = (V, E)$, one can efficiently construct a cactus $\overline{G} = (\overline{V}, \overline{E})$ together with a mapping $\phi : V \to \overline{V}$ such that:*

1. *If $C \subseteq V$ is a minimum cut in G, then there exists a 2-cut $\overline{C} \subseteq \overline{V}$ in the cactus $\overline{G}$ satisfying*
$$C = \{v \in V : \phi(v) \in \overline{C}\}.$$
2. *Conversely, if $\overline{C} \subseteq \overline{V}$ is a 2-cut in the cactus $\overline{G}$, then*
$$C = \{v \in V : \phi(v) \in \overline{C}\}$$
is a minimum cut in G.

To prove the proposition, we assume the existence of a c-competitive online algorithm for cactus augmentation and show how to obtain a c-competitive online algorithm for connectivity augmentation.

Given a (weighted) CAP instance consisting of a k-edge-connected graph G (which is not $(k+1)$-edge-connected) and a set of links $\mathcal{L}$, we apply Theorem 4.3 to efficiently construct a corresponding (weighted) CacAP instance. In particular, since G is k-edge-connected but not $(k+1)$-edge-connected, every minimum cut in G has size exactly k. Specifically, we obtain a cactus graph $\overline{G}$ together with a mapping $\phi : V(G) \to V(\overline{G})$, and define the set of links

$$\overline{\mathcal{L}} := \{(\phi(u), \phi(v)) \mid (u, v) \in \mathcal{L}\}.$$

For each online request (s, t) in the CAP instance, we create a corresponding request $(\phi(s), \phi(t))$ in the CacAP instance.

We run the online algorithm for CacAP on the constructed instance. Whenever the algorithm selects a link $(\phi(u), \phi(v)) \in \overline{\mathcal{L}}$, our algorithm for CAP selects the corresponding link $(u, v) \in \mathcal{L}$. Since costs are preserved, it remains to argue that feasibility is preserved by this reduction.

Claim 4.1. *A set of links $A \subseteq \mathcal{L}$ serves a request (s, t) in G if and only if the corresponding set of links $\bar{A} := \{(\phi(u), \phi(v)) \mid (u, v) \in A\}$ serves the request $(\phi(s), \phi(t))$ in $\overline{G}$.*

Proof. We prove the claim in two directions.

($\Rightarrow$) Assume that A serves the request (s, t) in G. We show that $\bar{A}$ serves the request $(\phi(s), \phi(t))$ in $\overline{G}$. Suppose, for the sake of contradiction, that $\bar{A}$ does not serve the request $(\phi(s), \phi(t))$ in $\overline{G}$. Then there exists a 2-cut $\overline{M} \subseteq V(\overline{G})$ in the cactus $\overline{G}$ that separates $\phi(s)$ and $\phi(t)$ and is not covered by any link in $\bar{A}$.

By part (ii) of Theorem 4.3, the set

$$M := \{v \in V(G) \mid \phi(v) \in \overline{M}\}$$

is a k-cut in G. Moreover, since $\overline{M}$ separates $\phi(s)$ and $\phi(t)$ in $\overline{G}$, the cut M separates s and t in G.

Since A serves the request (s, t) in G, there is no k-cut separating s and t in $G \cup A$. Hence, there must exist a link $\ell = (u, v) \in A$ that covers the cut M. Note that a link (u, v) covers M if and only if the corresponding link $(\phi(u), \phi(v))$ covers $\overline{M}$. Therefore, the link $(\phi(u), \phi(v)) \in \bar{A}$ covers the 2-cut $\overline{M}$, contradicting the assumption that $\overline{M}$ is not covered by any link in $\bar{A}$. This contradiction shows that $\bar{A}$ serves the request $(\phi(s), \phi(t))$ in $\overline{G}$.

($\Leftarrow$) Assume that $\bar{A}$ serves the request $(\phi(s), \phi(t))$ in $\overline{G}$. We show that A serves the request (s, t) in G. Suppose, for the sake of contradiction, that A does not serve the request (s, t) in G. Then there exists a k-cut $M \subseteq V(G)$ in G that separates s and t and is not covered by any link in A.

By part (i) of Theorem 4.3, there exists a 2-cut $\overline{M} \subseteq V(\overline{G})$ in the cactus $\overline{G}$ such that

$$M = \{v \in V(G) \mid \phi(v) \in \overline{M}\}.$$

Since M separates s and t in G, the corresponding 2-cut $\overline{M}$ separates $\phi(s)$ and $\phi(t)$ in $\overline{G}$. Moreover, a link (u, v) covers M if and only if the corresponding link $(\phi(u), \phi(v))$ covers $\overline{M}$. Hence, since no link in A covers M, it follows that no link in $\bar{A}$ covers $\overline{M}$.

This contradicts the assumption that $\bar{A}$ serves $(\phi(s), \phi(t))$ in $\overline{G}$, since $\overline{M}$ is a 2-cut separating $\phi(s)$ and $\phi(t)$ that remains uncovered. Therefore, A must serve the request (s, t) in G. □

4.3 Properties of Links and Cacti

Now that we have established the reduction from CAP to the Cactus Augmentation Problem (CacAP), we delve into the specific structural properties of cacti that are central to our online algorithms.

Given a graph G, a cut is a partition of $V(G)$ into two parts. Given a cut (V_1, V_2) in a cactus graph, we say that a link $\ell = \{u, v\}$ *covers* the cut if $u \in V_1$ and $v \in V_2$, or vice versa. A key component of our approach for cactus graphs is to decompose any given link based on the cycles traversed by it. This decomposition gives rise to the notions of a link's projection vertex set and its projections onto cycles. Sometimes it will be useful to decompose a pair of distinct vertices which need not be a link, so the following definition is stated more generally.

Definition 4.1 (Link Decomposition)**.** *Let G be a cactus and let $\ell = (u, v)$ be an arbitrary vertex pair consisting of two distinct vertices. Note that although links are inherently unordered pairs, we will, for technical convenience, sometimes treat them as ordered.*

1. ***Projection Vertex Set:*** *Consider all simple paths between u and v in the cactus graph. Let $(c_1, c_2, \ldots, c_k)$ be the sequence of articulation vertices (excluding u and v) that appear on every such path from u to v, in order. The* projection vertex set *of ℓ is the ordered set of vertices $\mathcal{V}(\ell) = (u, c_1, \ldots, c_k, v)$.*

2. ***Projection on a Cycle:*** *The Projection Vertex Set $\mathcal{V}(\ell)$ decomposes $\ell = (u, v)$ into a set of segments. Each segment (x, y), where x, y are consecutive vertices in $\mathcal{V}(\ell)$, lies on*

a unique cycle C. We call such a segment (x, y) the projection *of ℓ onto the cycle C, denoted* $\texttt{proj}_C(\ell)$.

The projection vertex set of a link $\mathcal{V}(\ell)$ characterizes which vertex pairs become 3-edge-connected when ℓ is added to G. See Figure 4.1 for an illustration. We have the following two simple propositions.

Proposition 4.2. *Suppose a set of links is added to the cactus G. Two distinct vertices u and v become 3-edge-connected if and only if every pair of distinct vertices $(x, y) \in \mathcal{V}((u, v)) \times \mathcal{V}((u, v))$ is 3-edge-connected.*

Proof. Let the projection vertex set be denoted as $\mathcal{V}((u, v)) = (c_0, c_1, \ldots, c_m)$, where $c_0 = u$ and $c_m = v$. We prove the equivalence in two directions.

($\Rightarrow$) Consider any pair of vertices $(x, y) \in \mathcal{V}((u, v)) \times \mathcal{V}((u, v))$. It suffices to show that each 2-cut $F \subseteq E(G)$ in the cactus separating x and y also separates u and v; since we know in the augmented cactus u and v are 3-edge-connected, there exists a link covering F, and hence x and y are also 3-edge-connected.

Let F be a 2-cut in the cactus G that separates x and y. Now, by definition, every u-v path has x and y on it. So if we delete the edges in F from G, there is no x-y path left, and hence no u-v path left. Thus, F also separates u and v.

($\Leftarrow$) Assume that every pair of consecutive vertices (c_i, c_{i+1}) for $0 \leq i < m$ is 3-edge-connected in the augmented cactus. We show that u and v must also be 3-edge-connected. Suppose, for the sake of contradiction, that u and v are not 3-edge-connected. Then there exists a 2-cut $F \subseteq E(G)$ in the cactus that separates u and v, and F is not covered by any added link. Let V_u and V_v be the vertex sets of the two connected components of $G - F$ containing u and v, respectively. Since $c_0 = u \in V_u$ and $c_m = v \in V_v$, there must exist some index i $(0 \leq i < m)$ such that $c_i \in V_u$ and $c_{i+1} \in V_v$. This implies that the 2-cut F also separates c_i and c_{i+1}. Because F is not covered by any added link, it remains an uncovered 2-cut in the augmented graph separating c_i and c_{i+1}. However, this contradicts the assumption that c_i and c_{i+1} are 3-edge-connected. Thus, every 2-cut separating u and v must be covered, meaning u and v are indeed 3-edge-connected. □

Moreover, when we augment G with a single link ℓ, its endpoints immediately become 3-edge-connected, leading to the following immediate consequence.

Proposition 4.3. *If a link ℓ is added to a cactus G, then every pair of distinct vertices $(u, v) \in \mathcal{V}(\ell) \times \mathcal{V}(\ell)$ has three edge-disjoint paths between them.*

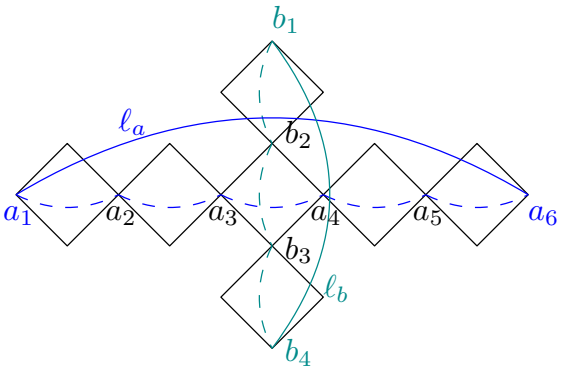


Figure 4.1: Links ℓ_a and ℓ_b cross, since their projections (a_3, a_4) and (b_2, b_3) cross within a cycle.

We now define the notion of when two links *cross*.

Definition 4.2 (Crossing Links)**.** *Let $G = (V, E)$ be a cactus graph.*

1. ***Within a cycle:*** *Let $\ell = (x, y)$ and $\ell' = (x', y')$ be two links whose endpoints are all in the same cycle C. We say that ℓ and ℓ'* cross in C *if one of the following two conditions holds:*

   (a) *The links share exactly one endpoint (e.g., $y = x'$ and $x \neq y'$).*

   (b) *The endpoints of the two links are distinct and they interleave on the cycle. That is, traversing a simple path from x to y along an arc of C encounters exactly one vertex from $\{x', y'\}$ as an internal vertex.*

2. ***General case:*** *Two links ℓ and ℓ' are said to* cross *if they share exactly one common endpoint, or if there exists a projection $\texttt{proj}_C(\ell)$ of ℓ and a projection $\texttt{proj}_C(\ell')$ of ℓ' onto the same cycle C, such that these two projections cross in C.*

In Figure 4.1, we are given links $\ell_a = \{a_1, a_6\}$ and $\ell_b = \{b_1, b_4\}$. Their projection vertex sets are $\mathcal{V}(\ell_a) = (a_1, a_2, \ldots, a_6)$ and $\mathcal{V}(\ell_b) = (b_1, b_2, \ldots, b_4)$ respectively. Note here that even though some path from a_1 to a_6 might pass through the articulation vertex b_2, *every* path between the vertices would not, and hence b_2 is not in the projection vertex set of ℓ_a. Further we can say that ℓ_a and ℓ_b cross since their projections cross within the cycle $\{b_2, a_4, b_3, a_3\}$. For another example of crossing links, see Figure 4.3c. The notion of crossing links enables us to determine whether adding a set of links $A \subseteq \mathcal{L}$ to the cactus G makes a given pair of vertices u and v 3-edge-connected. This is formalized in the following proposition.

Proposition 4.4. *If we add a set of links $A \subseteq \mathcal{L}$ to a cactus G, then a pair of distinct vertices (u, v) are 3-edge-connected in $G \cup A$ if and only if there exists a sequence of links in A such*

that (i) u lies in the projection set of the first link, (ii) v lies in the projection set of the last link, and (iii) every two consecutive links in the sequence cross. We refer to such a sequence as a sequence of crossing links from u to v.

Before giving a proof of Proposition 4.4, we first prove the following proposition.

Proposition 4.5. *Let G be a cycle and let $A \subseteq \mathcal{L}$ be a set of links added to G. A pair of distinct vertices (u, v) is 3-edge-connected in $G \cup A$ if and only if there exists a sequence of crossing links in A from u to v.*

Proof. We prove the proposition in two directions.

($\Leftarrow$) Suppose there exists a sequence of crossing links from u to v. We show that u and v are 3-edge-connected in $G \cup A$.

Consider any pair of edges e_1, e_2 of the cycle forming a 2-cut that separates u and v. Let V_u and V_v denote the vertex sets of the two resulting components, containing u and v, respectively. By assumption, there is a sequence of crossing links $\ell_1 - \ell_2 - \cdots - \ell_q$, where one endpoint of ℓ_1 is u and one endpoint of ℓ_q is v.

If none of the links in this sequence crossed the cut $\{e_1, e_2\}$, then there would exist an index i such that the links $\ell_1, \ell_2, \ldots, \ell_i$ have both endpoints in V_u, while ℓ_{i+1} has both endpoints in V_v. However, in this case ℓ_i and ℓ_{i+1} cannot cross, contradicting the assumption that consecutive links in the sequence are crossing.

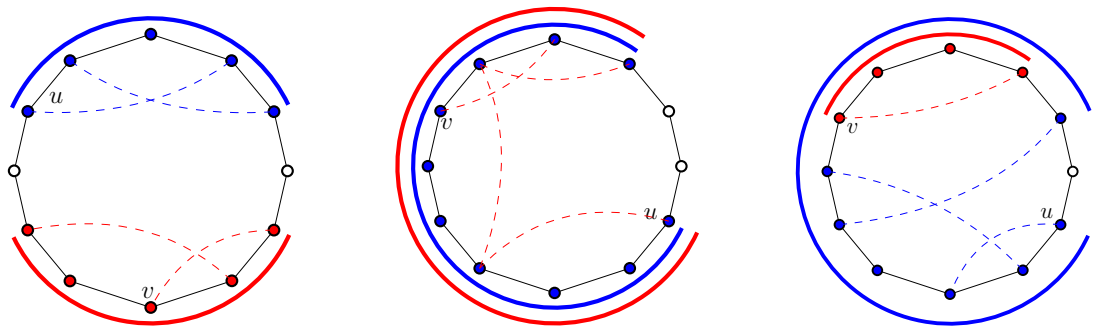
Therefore, every 2-cut separating u and v in the cycle is covered by at least one link in A. This implies that no 2-cut separates u and v in $G \cup A$, and hence u and v are 3-edge-connected.

($\Rightarrow$) Assume that u and v are 3-edge-connected in $G \cup A$. We show that there exists a sequence of crossing links in A from u to v.

Coloring Procedure: Label the vertices of the cycle by their clockwise distance from u. Initially, assign the color blue to u. While there exists an (as yet unselected) link with at least one endpoint colored blue, assign the color blue to all vertices whose labels lie between the labels of the two endpoints of that link (both inclusive). Next, assign the color red to v and repeat the same Coloring Procedure for the red color.

Let B and R denote the sets of vertices colored blue and red, respectively. Note that a vertex (including u or v) may receive both colors, and that the two endpoints of every link are always assigned the same set of colors (possibly none, one color, or both).

Observe that the set B forms a contiguous clockwise path starting at u and ending at some vertex u', and similarly R forms a contiguous path containing v. Let x be the vertex with the smallest label that is colored red. If x is ever assigned the color blue during the coloring



(a) No intersection. (b) $R = B$. (c) One contains the other.

Figure 4.2: Different ways in which sections of colored vertices can interact.

process, then all vertices that are subsequently colored red will also be colored blue. Hence, if $R \cap B \neq \emptyset$, then $R \subseteq B$.

We now analyze the possible interactions between R and B; see Figure 4.2 for an illustration.

- $R \cap B = \emptyset$ (Figure 4.2a). Consider the cut $(B, V \setminus B)$, which separates u and v. By construction, every link with one endpoint in B has its other endpoint also in B, and the same holds for R. Thus, no link in A covers this cut, implying that the cut has size two in $G \cup A$, contradicting the assumption that u and v are 3-edge-connected.

- $B = R$ (Figure 4.2b). We claim that there exists a sequence of crossing links from u to v. Let w be the colored vertex with the largest label. By the Coloring Procedure, there exists a sequence of links starting at u and ending at w, and similarly a sequence of links starting at v and ending at w. The last links in these two sequences both have w as an endpoint and therefore cross. Concatenating these two sequences yields a sequence of crossing links from u to v.

- $R \subsetneq B$ (Figure 4.2c). Consider the cut $(R, V \setminus R)$. Vertex v is colored both red and blue, while vertex u is colored only blue. (Indeed, if u were ever colored red, the coloring process would imply $B \subseteq R$, contradicting the strict containment.) Hence, this cut separates u and v. Since the endpoints of every link receive the same set of colors, no link covers the cut $(R, V \setminus R)$. Thus, this cut has size two in $G \cup A$, again contradicting that u and v are 3-edge-connected.

The above contradictions leave only the case $B = R$, which guarantees the existence of a sequence of crossing links from u to v. □

We are now ready to prove the earlier proposition.

Proof of Proposition 4.4. If u and v lie in the same cycle, the claim follows directly from Proposition 4.5. Hence, we assume that u and v lie in different cycles. We prove the proposition in two directions.

($\Rightarrow$) Assume that u and v are 3-edge-connected. We show that there exists a sequence of crossing links from u to v.

Let $v_1 = u, v_2, \ldots, v_q = v$ be the vertices that lie on every u–v path; note that $v_2, \ldots, v_{q-1}$ are articulation vertices. Each consecutive pair (v_i, v_{i+1}) lies on a common cycle and admits three edge-disjoint paths between them by Proposition 4.2.

Suppose, for contradiction, that no sequence of crossing links exists from u to v. Then there must exist an index i such that no sequence of crossing links exists from v_i to v_{i+1}; otherwise, concatenating such sequences for all consecutive pairs would yield a sequence from u to v.

Fix such a pair (v_i, v_{i+1}), and let R be the cycle containing them. Consider the projections of all added links onto the cycle R. Since v_i and v_{i+1} admit three edge-disjoint paths in $G \cup A$, they are 3-edge-connected in the graph induced by R together with these projected links. By Proposition 4.5, this implies the existence of a sequence of crossing links from v_i to v_{i+1}, contradicting our assumption. Therefore, a sequence of crossing links from u to v must exist.

($\Leftarrow$) Assume that there exists a sequence of crossing links $\ell_1 - \ell_2 - \cdots - \ell_t$ from u to v, where u lies in the projection vertex set of ℓ_1 and v lies in the projection vertex set of ℓ_t. We show that u and v are 3-edge-connected.

Let e_1, e_2 be any two edges of the cactus forming a 2-cut that separates u and v, and let V_u and V_v be the vertex sets of the two connected components of $G - \{e_1, e_2\}$ containing u and v, respectively. Note that e_1 and e_2 lie on a common cycle. We claim that some link ℓ_i in the sequence covers this 2-cut.

Suppose not. Then ℓ_1 does not cover the cut, and hence its endpoints lie on the same side. They cannot both lie in V_v, since in that case they would be connected within the component V_v of $G - \{e_1, e_2\}$, yielding an endpoint-to-endpoint path avoiding $u \in V_u$, contradicting that u lies in the projection vertex set of ℓ_1. Therefore, both endpoints of ℓ_1 lie in V_u. By the same argument, both endpoints of ℓ_t lie in V_v.

Since no link in the sequence covers the cut, there exists an index i such that $\ell_1, \ldots, \ell_i$ have both endpoints in V_u, while ℓ_{i+1} has both endpoints in V_v. But then the cut (V_u, V_v) separates the endpoints of ℓ_i and ℓ_{i+1}, so these two links cannot cross, contradicting the assumption that consecutive links in the sequence are crossing.

Thus, every 2-cut separating u and v is covered by at least one link, implying that u and v are 3-edge-connected. □

4.4 An $O(\log n)$-competitive Algorithm for Online CAP

In this section, we obtain an $O(\log n)$-competitive algorithm for CAP, thereby proving Theorem 4.1. From Proposition 4.1, we know that online CAP reduces to online CacAP. Thus, it suffices to prove the following theorem.

Theorem 4.4. *There exists an $O(\log n)$-competitive online algorithm for the online Cactus Augmentation Problem (CacAP).*

To prove Theorem 4.4, we first describe *structured instances* which satisfy some properties that can be assumed without loss of generality. In order to define structured instances, we first need to define a decomposition tree.

Decomposition Tree

To each cactus, we associate a rooted tree, called its *decomposition tree*.

To construct a decomposition tree T of a cactus G, first choose an arbitrary vertex $r \in V(G)$ to be the root of the tree T. The vertex set of the tree we construct will be $V(T) = \mathcal{A} \cup \{r\}$, where $\mathcal{A} \subseteq V(G)$ is the set of all articulation vertices in the cactus G. We will construct T incrementally. Initially, $V(T) = \emptyset$.

- Insert r into $V(T)$ and mark it visited.
- While there exists a cycle c in G that has only one visited vertex and the root of the cycle c has not been identified:
  - Denote the visited vertex to be the root of the cycle, r_c.
  - Add all articulation vertices in c other than r_c to $V(T)$ and mark them as visited. Furthermore, connect these articulation vertices in the cycle c to r_c by creating new edges in $E(T)$.

It is straightforward to verify that the above procedure visits all articulation vertices and, for each cycle c, identifies a corresponding root r_c.

In Figure 4.3a, we see a given cactus and a feasible set of links represented by blue lines. In Figure 4.3b, the decomposition tree is shown in red, with the root being the vertex labeled r.

Given a tree T and a link $\ell \in \binom{V(T)}{2}$, we say that ℓ *covers* an edge of the tree T if adding the link ℓ to T ensures that there are two edge-disjoint paths between the endpoints of the edge. Also, ℓ is said to be an *uplink* if one endpoint of ℓ is an ancestor of the other on the tree T.

We are now ready to define a Structured instance for online CacAP.

Definition 4.3 (Structured instance)**.** *An instance of online CacAP is structured if it satisfies the following properties with respect to its decomposition tree T:*

1. *Every online request lies within a cycle (i.e., each request specifies two vertices of a cycle).*
2. *Every link in the instance either is between two vertices of the decomposition tree or lies within a cycle.*
3. *All links between vertices of the decomposition tree are uplinks.*
4. *Every edge of the decomposition tree is covered by a link in the input.*

Now, the proof of Theorem 4.4 immediately follows from the following two lemmas. We prove Lemma 4.1 in Section 4.4.1, and Lemma 4.2 in Section 4.4.2.

Lemma 4.1. *If there exists an $O(\log n)$-competitive online algorithm for structured instances of CacAP, then there exists an $O(\log n)$-competitive algorithm for CacAP.*

Lemma 4.2. *There exists an $O(\log n)$-competitive online algorithm for structured instances of CacAP.*

4.4.1 Structuring the Input Instance

We first prove Lemma 4.1.

Proof. We claim that the four properties of a structured instance can be assumed with only a constant factor loss in the cost of the optimal solution. We sketch a brief proof for each of the four properties.

1. Every online request lies within a cycle: When a request (u, v) arrives, 3-edge-connecting (u, v) requires 3-edge-connecting every pair of vertices in its projection vertex set $\mathcal{V}((u, v)) = (u, c_1, \ldots, c_k, v)$ by Proposition 4.2. Equivalently, the adversary may instead present the sequence of requests $(u, c_1), (c_1, c_2), \ldots, (c_k, v)$, each confined to a single cycle, potentially making it harder for our algorithm to maintain a good competitive ratio. Formally, whenever a request (u, v) arrives, our algorithm processes it by sequentially handling the requests $(u, c_1), (c_1, c_2), \ldots, (c_k, v)$. Notice, however, that the offline optimum remains unchanged in both cases (by Proposition 4.2).
2. Every link in the instance either is between two vertices of the decomposition tree or lies within a cycle: Consider any link $\ell = (u, v)$ on the rooted cactus. If u and v lie in the

same cycle of the input cactus, then ℓ trivially satisfies the property. Otherwise, let the projection vertex set of ℓ be

$$\mathcal{V}(\ell) = (u, c_1, \ldots, c_k, v),$$

where $k \geq 1$. We know that any pair of consecutive vertices in this sequence lie on the same cycle and the c_is are articulation vertices. So we replace the link ℓ with at most three links as follows: the head $H(\ell) = \{u, c_1\}$, the middle $M(\ell) = \{c_1, c_k\}$, and the tail $T(\ell) = \{c_k, v\}$. If $c_1 = c_k$, $M(\ell)$ does not exist. Clearly, $H(\ell)$, $M(\ell)$, and $T(\ell)$ satisfy the property of being either within a cycle, or between two vertices of the decomposition tree (as all articulation vertices are on the decomposition tree).

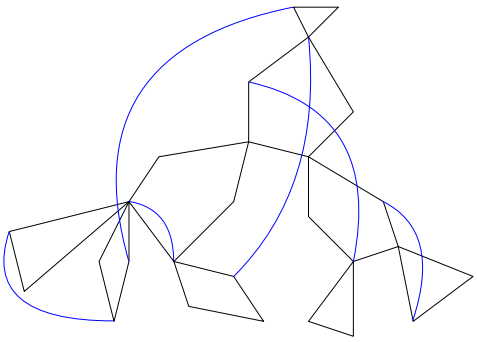
Since $H(\ell)$ crosses $M(\ell)$, and $M(\ell)$ crosses $T(\ell)$, by Proposition 4.4 they serve the same purpose as ℓ in 3-edge-connecting requests. Thus, we can safely do all such replacements, incurring at most a factor-3 loss in the size of the optimal solution.

3. All links between vertices of the decomposition tree are uplinks: Consider any link $\ell = (u, v)$ between vertices of the decomposition tree. If it is not an uplink, let w be the lowest common ancestor of u and v on the decomposition tree. Replace ℓ with two links: (u, w) and (w, v).

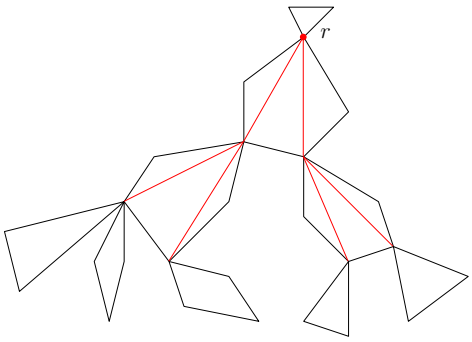
   Again, it is easy to see by Proposition 4.4, that $\{u, w\}$ and $\{w, v\}$ serve the same purpose as ℓ in 3-edge-connecting requests. Furthermore, by repeatedly doing this replacement, we ensure that every link between vertices of the decomposition tree is an uplink, and for this step we incur at most a factor-2 loss in the size of the optimal solution. For an example of splitting a link into uplinks, see Figure 4.3d. Here the (blue) link ℓ gets split into two (dashed blue) uplinks ℓ'_1, ℓ'_2 which now originate at r, which is the common ancestor of u and v.

4. Every edge of the decomposition tree is covered by a link in the input: If there exists an edge $e = (u, v)$ on the decomposition tree T that is not covered by any link, we break the cactus into two smaller cactus graphs by separating out an appropriate sub-cactus and then solve these instances independently.

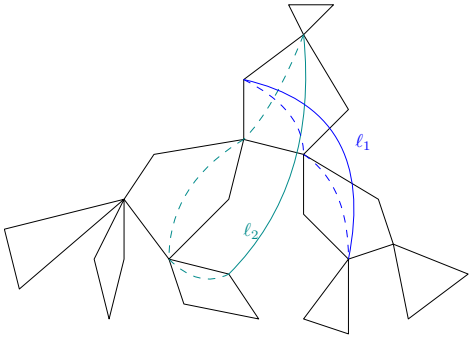
   More specifically, suppose the endpoints of e, u and v, lie on the same cycle C and u is the parent of v in the tree T. In this case, we split the cactus at v so that the subtree of T rooted at v becomes the tree corresponding to the separated sub-cactus. Note that u continues to be an articulation vertex, and hence will still be a vertex in the *parent* tree.



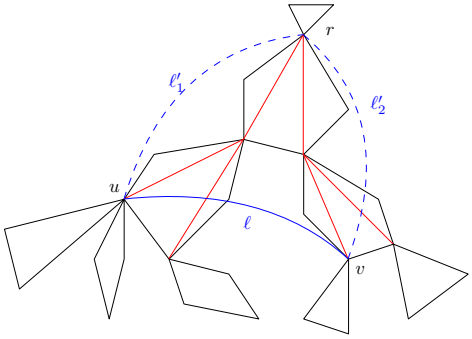
(a) Cactus and links forming a CacAP instance.



(b) Tree obtained from articulation vertices.



(c) Crossing Links.



(d) Splitting a link into uplinks.

Figure 4.3: Example instance of Cactus Augmentation Problem.

Meanwhile v need not be an articulation vertex anymore. We can repeatedly apply this splitting procedure until the assumption is satisfied for every edge of T.

Since each request is confined to a single cycle, no request has endpoints in two different obtained cacti. Furthermore, there cannot be any link with endpoints in the two obtained cactus graphs; since any such link is now necessarily between two articulation vertices (from (2) above), and is an uplink (from (3) above), it definitely covers the tree edge $\{u, v\}$, a contradiction. Now, each of the resulting cactus graphs contains fewer vertices, applying the $O(\log n)$-competitive algorithm in parallel to both the instances yields an overall competitiveness at most the maximum of the two, which remains $O(\log n)$. We can now repeatedly split the obtained cactus graphs further if any of their decomposition trees contain an uncovered edge. □

4.4.2 Solving Structured Instances

All that is left to prove is Lemma 4.2, i.e., obtaining an $O(\log n)$-competitive algorithm for structured instances. We make use of the following two theorems to give an algorithm that can solve such instances.

Theorem 4.5. *There exists a deterministic $O(\log n)$-competitive algorithm for online TAP.*

Theorem 4.6. *There exists a deterministic $O(\log n)$-competitive algorithm for online Cycle Augmentation Problem (CycAP).*

Theorem 4.5 follows from the work of Naor, Umboh, and Williamson [117]. We include an alternate proof in Section 5.2. Theorem 4.6 follows from a reduction to the online Steiner forest problem, and we give the proof in Section 4.5.

We will design an algorithm ALG for structured instances of online CacAP and prove that it is $O(\log n)$-competitive. Let ALG_T be a $(c_t \log n)$-competitive algorithm for online TAP and ALG_R be a $(c_r \log n)$-competitive algorithm for online CycAP promised by Theorems 4.5 and 4.6, respectively, where c_t and c_r are absolute positive constants. These algorithms will act as subroutines of our main algorithm ALG.

Let the input to our algorithm ALG be a cactus $G = (V, E)$ and the set of links $\mathcal{L}$. Given G and $\mathcal{L}$, we create an online CycAP instance for each cycle in G and a single online TAP instance. Each cycle instance is handled by an independent copy of ALG_R, while the TAP instance is handled by ALG_T. Upon the arrival of a request, we carefully generate corresponding subrequests for the relevant subroutines. The idea is that ALG_R handles local optimizations within individual cycles, while ALG_T manages global, long-range optimizations across the cactus structure.

The TAP instance. The TAP instance is defined on the decomposition tree of the cactus. And the set of links $\mathcal{M}$ for this instance consists of all the links which have both their endpoints on vertices of the decomposition tree.

The CycAP instances. Each cycle in G is treated as a separate CycAP instance. For the instance corresponding to a cycle C, we define its link set as

$$\mathcal{L}(C) = \bigcup_{\ell \in \mathcal{L}} \mathtt{proj}_C(\ell).$$

Notice that $\mathcal{L}(C)$ consists of the projections of all links in $\mathcal{L}$ onto the cycle C.

Algorithm. With the above setup, we can now describe our algorithm ALG. When an uncovered request (s_i, t_i) arrives in some cycle c:

1. TAP phase: For each vertex $v \in \{s_i, t_i\}$ that is on the decomposition tree, forward the request (r_c, v) to the online TAP algorithm ALG_T if it is not already covered. Add the links selected by ALG_T to our solution.

2. CycAP phase: If (s_i, t_i) remains uncovered, forward it to the online CycAP algorithm ALG_R^c corresponding to cycle c. For every projected link $\mathtt{proj}_c(\ell)$ selected by ALG_R^c, include the corresponding original link $\ell \in \mathcal{L}$ in our solution.

Notice that ALG always succeeds in serving any request whenever a feasible solution exists. This holds because every request lies entirely within a single cycle, and for each cycle instance, the online algorithm has access to the projections of all links in $\mathcal{L}$ onto that cycle.

4.4.3 Analysis via a Charging Argument

We now analyze the performance of ALG.

Let COPT denote the offline optimal solution for the given structured cactus instance. Fix a cycle c in the cactus G, and let COPT_c be the set of projections of COPT onto cycle c. We partition

$$\mathsf{COPT}_c = \mathsf{COPT}_c^W \sqcup \mathsf{COPT}_c^M,$$

where $\mathsf{COPT}_c^W \subseteq \mathcal{L}$ consists of all links in $\mathcal{L}$ that lie within the cycle c, and COPT_c^M consists of projections of the other links onto the cycle c. Intuitively, links in COPT_c^M consists of projections that are in the 'middle' of other links. Notice that one endpoint of each link in COPT_c^M is necessarily r_c, the root of the cycle c.

Let OPT_c denote the offline optimal solution for all the requests that our algorithm ALG forwarded to the cycle-specific algorithm ALG_R^c. For simplicity of exposition, we will refer to all elements of OPT_c and COPT_c as links rather than projections of links.

Let TOPT denote the optimal solution for all the requests that our algorithm ALG forwarded to the tree algorithm ALG_T.

We will prove the following two bounds on the sizes of COPT^c and TOPT.

Lemma 4.3. $17 \cdot |\mathsf{COPT}_c^W| \geq |\mathsf{OPT}_c|$.

Lemma 4.4. $2 \cdot |\mathsf{COPT}| \geq |\mathsf{TOPT}|$.

Before proving the above lemmas, we first show how they suffice to complete the proof of Lemma 4.2.

Proof of Lemma 4.2. Since each link in COPT which is within a cycle is part of only one cycle, we have

$$|\mathsf{COPT}| \geq \sum_c |\mathsf{COPT}_c^W|.$$

Also the cost of our algorithm is at most

$$\begin{aligned}
|\mathsf{ALG}| &\leq \left(\sum_c c_r \log n_c \cdot |\mathsf{OPT}_c|\right) + c_t \log n \cdot |\mathsf{TOPT}| \\
&\leq (c_r + c_t) \log n \left(\sum_c |\mathsf{OPT}_c| + |\mathsf{TOPT}|\right) \\
&\leq (c_r + c_t) \log n \left(17 \sum_c |\mathsf{COPT}_c^W| + 2 \cdot |\mathsf{COPT}|\right) \qquad \text{(From Lemmas 4.3 and 4.4)} \\
&\leq 19(c_r + c_t) \log n \cdot |\mathsf{COPT}| = O(\log n) \cdot |\mathsf{COPT}|.
\end{aligned}$$

Hence we see that the algorithm returns a solution that is at most $O(\log n)$ times the optimal offline solution, giving us the required competitive ratio. □

4.4.3.1 Bound for a Cycle Instance

We now prove Lemma 4.3, i.e., $17 \cdot |\mathsf{COPT}_c^W| \geq |\mathsf{OPT}_c|$.

Proof. Note that COPT_c is a feasible solution for the cycle instance c, but COPT_c^W alone need not be feasible. Therefore, consider the set $\mathsf{SOL} = \mathsf{COPT}_c^W \sqcup \mathsf{COPT}_c^{M'}$, where $\mathsf{COPT}_c^{M'} \subseteq \mathsf{COPT}_c^M$ is a minimal subset such that SOL covers all requests assigned to the cycle c. Thus, we have

$$|\mathsf{COPT}_c^{M'}| + |\mathsf{COPT}_c^W| \geq |\mathsf{OPT}_c|.$$

We will show via a charging argument that $|\mathsf{COPT}_c^{M'}| \leq 16\,|\mathsf{COPT}_c^W|$, which from above will immediately give us the desired bound:

$$17\,|\mathsf{COPT}_c^W| \;\geq\; |\mathsf{OPT}_c|.$$

Now to show $|\mathsf{COPT}_c^{M'}| \leq 16\,|\mathsf{COPT}_c^W|$, construct a bipartite graph whose left nodes correspond to all links in $\mathsf{COPT}_c^{M'}$, and whose right nodes correspond to all requests (s_i, t_i) sent to cycle c. To define edges in this bipartite graph, fix for each request (s_i, t_i) a shortest s_i–t_i path in c, meaning a sequence of crossing links from SOL where the first link is incident to s_i, the last link is incident to t_i (such a sequence of links exists by Proposition 4.4).

Observe that each link in $\mathsf{COPT}_c^{M'}$ belongs to the middle portion of some original link and is therefore incident on the root r_c of cycle c (since structured instances have only uplinks between two vertices that are not within a cycle). Consequently, any shortest s_i–t_i path can include at most two links from $\mathsf{COPT}_c^{M'}$, as otherwise it would revisit r_c multiple times, contradicting our

assumption that the path is shortest.

In the bipartite graph, we add an edge between a link $e_i \in \mathsf{COPT}_c^{M'}$ (left node) and a request (s_j, t_j) (right node) if e_i lies on the fixed shortest s_j–t_j path. By construction, the degree of any right node (request) is at most two, while every left node has degree at least one (by the minimality of $\mathsf{COPT}_c^{M'}$). It is easy to see that such a bipartite graph with these degree bounds has a matching D such that

$$2|D| \geq |\mathsf{COPT}_c^{M'}|.$$

Finally, note that a request (s, t) between two articulation vertices is never assigned to a cycle: such requests are instead handled in the TAP phase, which processes (s, r_c) and (t, r_c). Since the TAP instance is assumed feasible, it ensures (s, r_c) and (r_c, t) are 3-edge-connected, and hence also (s, t). Therefore, without loss of generality, we may assume that each s_i is a non-articulation vertex.

Now, consider the set of terminals of the requests matched in D above (i.e., the s_i's and t_i's) and form a graph where each request is represented as an edge between its two terminals. This graph cannot contain a cycle, since any edge that would close a cycle corresponds to a request that is already satisfied and therefore would never have been sent to the CycAP instance. Let there be a terminals labeled s_i (non-articulation vertices) and b terminals labeled t_i. Since this graph is a forest with $|D|$ edges, we have $a + b \geq |D| + 1 \geq |D|$.

Case 1, $a \geq |D|/2$**:** Consider a link e_i and a request (s_i, t_i) that are matched in D. Since s_i is a non-articulation vertex, the first link on the s_i–t_i path must lie in COPT_c^W, because both endpoints of any link in $\mathsf{COPT}_c^{M'}$ are articulation vertices. Thus, there are at least $|D|/4$ distinct links in COPT_c^W, which act as the first links in the s_i–t_i paths above. Thus, $|D| \leq 4|\mathsf{COPT}_c^W|$.

Case 2, $b \geq |D|/2$, and more than $|D|/4$ of the t_i's are non-articulation vertices: Consider a link e_i and a request (s_i, t_i) that are matched in D and t_i is a non-articulation vertex. Since t_i is not an articulation vertex, the last link on the s_i–t_i path must belong to COPT_c^W. Thus, there are at least $|D|/8$ distinct links in COPT_c^W, which act as the last links in the s_i–t_i paths above. Thus, $|D| \leq 8|\mathsf{COPT}_c^W|$.

Case 3, $b \geq |D|/2$, and more than $|D|/4$ of the t_i's are articulation vertices: Consider all requests (s_i, t_i) matched in D, where t_i is an articulation vertex. In this situation we show that all s_i in these requests must be distinct. To see why, suppose $s_i = s_j$ for some $j > i$. When the request (s_j, t_j) arrives, the algorithm would already have served the

requests (r_c, t_i), (s_i, t_i), and (r_c, t_j). Hence, (s_j, t_j) would already be connected at the time of arrival and would therefore never be sent to the cycle instance. Thus, there are at least $|D|/4$ distinct s_i, and similar to case 1 above, there are at least $|D|/8$ distinct links in COPT_c^W, giving us $|D| \leq 8|\mathsf{COPT}_C^W|$. We have shown from the above charging argument that $|D| \leq 8\,|\mathsf{COPT}_c^W|$, which implies $|\mathsf{COPT}_c^{M'}| \leq 16\,|\mathsf{COPT}_c^W|$, as desired. □

4.4.3.2 Bound for the Tree Instance

Finally, we prove Lemma 4.4. i.e., $2 \cdot |\mathsf{COPT}| \geq |\mathsf{TOPT}|$.

Proof. We will construct a feasible solution TOPT' for the TAP instance of size at most $2|\mathsf{COPT}|$. We partition $\mathsf{COPT} = \mathsf{COPT}' \cup \mathsf{COPT}''$, where COPT' consists of all links in COPT each of which covers at least one tree request forwarded to ALG_T. Add all links in COPT' to TOPT'.

Now, consider the set of all the tree requests, of cardinality q, generated for a cycle c: $\{\{s_1, r_c\}, \ldots, \{s_q, r_c\}\}$ that are not already covered by COPT'. A request $\{s_i, r_c\}$ was generated due to some *parent* request $\{s_i, t_i\}$ for the cactus instance. Thus, COPT must have a sequence of crossing links covering $\{s_i, t_i\}$, where s_i is in the projection vertex set of the first link of the sequence. If the first link in the sequence is $\{s_i, r_c\}$ or if it is not within the cycle c, then it must be an uplink covering $\{s_i, r_c\}$, a contradiction to our assumption that $\{s_i, r_c\}$ is not already covered by COPT'. Thus, the first link in the sequence is $\{s_i, w_i\} \in \mathsf{COPT}''$, which is within the cycle c and $w_i \neq r_c$. Thus, there are at least $q/2$ links in COPT'' within the cycle c that serve as the first links in the sequences of crossing links to cover the q requests $\{s_i, t_i\}$ corresponding to the q tree requests $\{s_i, r_c\}$. In TOPT' add q links each covering one $\{s_i, r_c\}$ tree request (which exist because each edge of the decomposition tree in a structured instance is covered by some link). Do this for each cycle c. Thus, clearly, $|\mathsf{TOPT}'| \leq |\mathsf{COPT}'| + 2|\mathsf{COPT}''| \leq 2|\mathsf{COPT}|$. Now since TOPT' is a feasible solution for the tree instance and TOPT is an optimal solution, $|\mathsf{TOPT}| \leq |\mathsf{TOPT}'|$, and we are done. □

4.5 Online Algorithms for Weighted CAP and Online CycCAP

In this section, we present an $O(\log^2 n)$-competitive algorithm for WCAP, proving Theorem 4.2. By Proposition 4.1, it suffices to design an $O(\log^2 n)$-competitive algorithm for the weighted cactus augmentation problem. We establish the following result.

Theorem 4.7. *There exists an $O(\log^2 n)$-competitive online algorithm for the online weighted Cactus Augmentation Problem (WCacAP).*

We then improve this competitive ratio for the special case in which the underlying cactus is a cycle and the links are unweighted, proving Theorem 4.6.

Theorem 4.6. *There exists a deterministic $O(\log n)$-competitive algorithm for online* Cycle Augmentation Problem (CycAP).

Our proof of Theorem 4.7 proceeds via a reduction to the Online Node-Weighted Steiner Forest (NWSF) problem and the application of a known online NWSF algorithm. For the special case of unweighted cycle augmentation, we show that the resulting NWSF instance admits an approximation via a reduction to the Online Edge-Weighted Steiner Forest (EWSF) problem, for which a better competitive algorithm is known, thereby yielding Theorem 4.6. We first recall the NWSF and EWSF problems.

Definition 4.4 (Node-Weighted Steiner Forest)**.** *Given an undirected graph $G = (V, E)$, a cost function $c : V \to \mathbb{R}_{\geq 0}$ on the vertices, and a set of terminal pairs $\mathcal{T} \subseteq V \times V$, find a minimum-cost subset of vertices $S \subseteq V$ such that for each $(s, t) \in \mathcal{T}$, the induced subgraph $G[S]$ contains a path between s and t.*

Definition 4.5 (Edge-Weighted Steiner Forest)**.** *Given an undirected graph $G = (V, E)$, a cost function $c : E \to \mathbb{R}_{\geq 0}$ on the edges, and a set of terminal pairs $\mathcal{T} \subseteq V \times V$. Find a minimum-cost subset of edges $F \subseteq E$ such that for each $(s, t) \in \mathcal{T}$, the subgraph (V, F) contains a path between s and t.*

In the online versions of the above problems, the set of terminal pairs is not given upfront; instead, request pairs arrive one by one in an online fashion. Upon the arrival of a request pair, an online algorithm must irrevocably select vertices or edges so as to connect the requested pair using the vertices or edges chosen so far.

4.5.1 Reduction to Online NWSF

The reduction from online WCacAP to online NWSF is inspired by a reduction of the offline CAP problem to the Steiner Tree problem [15, 32]. Given an instance of weighted CacAP with cactus $G = (V, E)$ and a set of available links $\mathcal{L}$, we create an instance of NWSF, denoted $H = (V_H \cup T_H, E_H)$, as follows:

- *Steiner nodes:* For every link $\ell = \{u, v\} \in \mathcal{L}$ of cost w_ℓ, create a Steiner node v_ℓ with cost $c(v_\ell) = w_\ell$ in V_H.
- *Terminal nodes:* For every vertex $v \in V$ create a zero-cost terminal node (also denoted v) in T_H.

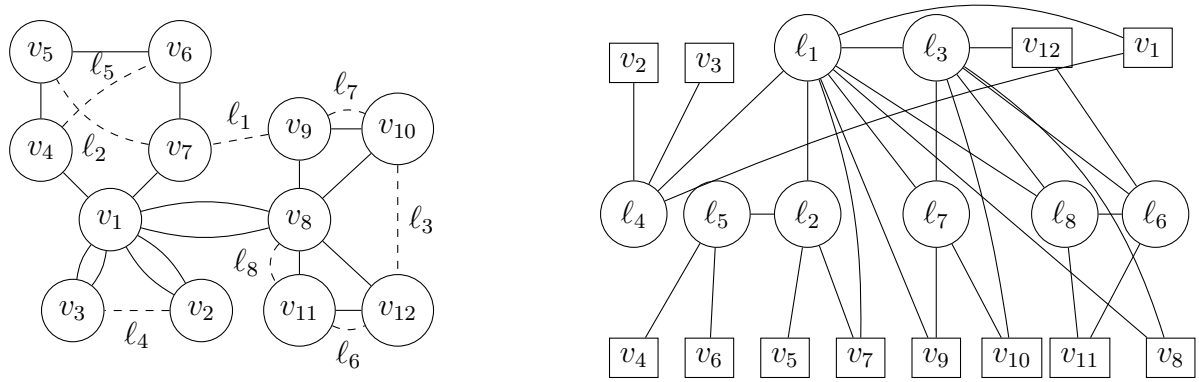


Figure 4.4: [32] (left) Instance of CacAP, where dashed edges denote links. (right) The Corresponding Steiner Forest instance, where square nodes denote terminals.

- *Incidence edges:* For every link ℓ, and for every vertex u in the projection vertex set of ℓ, add edges $\{v_\ell, u\}$ to E_H.
- *Crossing edges:* If two links ℓ_1 and ℓ_2 cross, add the edge $\{v_{\ell_1}, v_{\ell_2}\}$ to E_H.

When a WCacAP request (s, t) arrives, it is translated into an NWSF request to connect the corresponding terminal nodes s and t in the graph H. We handle these requests using an online algorithm for NWSF. For each Steiner node v_ℓ selected by the online NWSF algorithm, we purchase the corresponding link ℓ in the WCacAP instance. An example of this reduction is illustrated in Figure 4.4.

Lemma 4.5. *A set of links $\mathcal{L}' \subseteq \mathcal{L}$ ensures three edge-disjoint paths between some vertex pair (s, t) in G, if and only if s and t are connected in the induced subgraph $H[S_{\mathcal{L}'} \cup T_H]$, for $S_{\mathcal{L}'} := \{v_\ell \mid \ell \in \mathcal{L}'\}$.*

Proof. By Proposition 4.4, a set of links $\mathcal{L}' \subseteq \mathcal{L}$ ensures three edge-disjoint paths between s and t in $G \cup \mathcal{L}'$ if and only if there exists a sequence of links in $\mathcal{L}'$ such that s lies in the projection vertex set of the first link, t lies in the projection vertex set of the last link, and every two consecutive links in the sequence cross.

By construction of the reduced NWSF instance H, each link $\ell \in \mathcal{L}$ corresponds to a Steiner node v_ℓ, and for every pair of crossing links ℓ_1 and ℓ_2, the graph H contains the edge $\{v_{\ell_1}, v_{\ell_2}\}$. Moreover, each terminal node $u \in T_H$ is adjacent in H to all Steiner nodes corresponding to links whose projection vertex set contains u.

Hence, the existence of such a sequence of crossing links in $\mathcal{L}'$ from s to t is equivalent to the existence of a path between terminals s and t in the induced subgraph $H[S_{\mathcal{L}'} \cup T_H]$, where $S_{\mathcal{L}'} := \{v_\ell \mid \ell \in \mathcal{L}'\}$. □

Thus, Lemma 4.5 immediately implies that the above reduction is both competitiveness- and cost-preserving: each purchased link of cost w_ℓ in WCacAP corresponds one-to-one to a purchased Steiner node of the same cost in NWSF, and vice versa. Consequently, any competitive or approximation guarantee achieved for the online NWSF instance transfers unchanged to the online WCacAP instance. Since an $O(\log^2 n')$-competitive algorithm is known for NWSF [25], where $n' = |V(H)| = |V_H| + |T_H| = |V(G)| + |\mathcal{L}| \leq n + \binom{n}{2}$ and $n = |V(G)|$, we obtain the following theorem.

Theorem 4.7. *There exists an $O(\log^2 n)$-competitive online algorithm for the online weighted Cactus Augmentation Problem (WCacAP).*

In the next section we apply the above machinery only to the special case of an *unweighted single-cycle*.

4.5.2 Approximating CycAP using EWSF

While the above reduction allows us to reduce WCacAP exactly to NWSF, for the special case of *unweighted* cycle augmentation (CycAP) we further approximate the resulting NWSF instance by reducing it to an instance of EWSF.

Consider the NWSF graph H obtained from an unweighted CycAP instance consisting of a cycle G and a set of links $\mathcal{L}$. We construct an EWSF instance on the same graph H by assigning unit weight to every edge in $E(H)$. Note that in this setting, each Steiner node is adjacent to exactly two terminals, since the projection of any link consists precisely of its two endpoints.

Let a sequence of requests define a set of k distinct terminals. Let OPT_N denote an optimal solution to the NWSF instance, consisting of a set of Steiner nodes, with cost $|\mathsf{OPT}_N|$. Let OPT_E denote an optimal solution to the corresponding EWSF instance, consisting of a set of edges, with cost $|\mathsf{OPT}_E|$.

Claim 4.2. $|\mathsf{OPT}_E| \leq |\mathsf{OPT}_N| + k - 1$.

Proof. Let there be c connected components in the subgraph induced by the optimal NWSF solution. We can construct a spanning tree on each component to obtain a forest. This forest contains $|\mathsf{OPT}_N| + k - c \leq |\mathsf{OPT}_N| + k - 1$ edges, and serves all the requests. This is hence also a valid solution to the EWSF instance, giving the claimed bound on $|\mathsf{OPT}_E|$. □

Claim 4.3. $k \leq 2|\mathsf{OPT}_N|$.

Proof. In the case where every request introduces two new terminals, and requires exactly one Steiner node to be served, we have that $k = 2|\mathsf{OPT}_N|$. If any of the requests repeats a terminal, or if it requires more than one Steiner node to be served, we will have that $k < 2|\mathsf{OPT}_N|$. □

We now prove the main theorem of this subsection.

Proof of Theorem 4.6. We apply a known $O(\log n')$-competitive online algorithm for EWSF [90] to the constructed EWSF instance H, where $n' = |V(H)| = |V(G)| + |\mathcal{L}| \le n + \binom{n}{2}$. Let the competitive ratio of this algorithm be $c_e \log n'$ for some absolute constant c_e. The algorithm outputs an edge set ALG_E with

$$|\mathsf{ALG}_E| \le c_e \log n' \cdot |\mathsf{OPT}_E|.$$

Our algorithm for NWSF (and hence for CycAP) selects both endpoints of each edge chosen in ALG_E. Therefore, the number of Steiner nodes selected, denoted by ALG_N, satisfies

$$|\mathsf{ALG}_N| \le 2|\mathsf{ALG}_E|.$$

Using Claim 4.2, we obtain

$$\frac{|\mathsf{ALG}_N|}{|\mathsf{OPT}_N|} \le \frac{2|\mathsf{ALG}_E|}{|\mathsf{OPT}_N|} \le \frac{2c_e \log n' \cdot |\mathsf{OPT}_E|}{|\mathsf{OPT}_N|} \le \frac{2c_e \log n' \cdot (|\mathsf{OPT}_N| + k)}{|\mathsf{OPT}_N|},$$

where the second and third inequalities follow from [90] and Claim 4.2 respectively.

Applying Claim 4.3, which gives $k \le 2|\mathsf{OPT}_N|$, we conclude

$$\frac{|\mathsf{ALG}_N|}{|\mathsf{OPT}_N|} \le 2c_e \log n' \cdot \frac{3|\mathsf{OPT}_N|}{|\mathsf{OPT}_N|} = 6c_e \log n' = O(\log n).$$

Finally, since the reduction from Online CycAP to Online NWSF is competitiveness-preserving (Lemma 4.5), this yields an $O(\log n)$-competitive online algorithm for the unweighted Online CycAP. □

Chapter 5

Algorithms for Tree Augmentation

5.1 Introduction

In the previous chapter, we addressed the Connectivity Augmentation Problem (CAP) by leveraging heavy machinery involving reductions to cactus graphs and Steiner Forest. While that approach provides a unified framework for general graphs, the resulting algorithms rely on complex "black-box" subroutines. In this chapter, we shift our focus to the specific case of the Tree Augmentation Problem (TAP). We aim to exploit the structural simplicity of trees to design combinatorial algorithms that are simple and self-contained.

We present a simple $O(\log n)$-competitive algorithm for online TAP, providing a self-contained analysis. In contrast, the earlier algorithm by Naor, Umboh, and Williamson [117], which also applies to the weighted version (online WTAP), is based on a more involved primal-dual framework. For online TAP (and WTAP), without loss of generality each request can be assumed to correspond to an edge in the underlying tree. For each such request, the TAP algorithm selects two links: one that connects to the highest ancestor and another that descends to the deepest reachable vertex. The "deepest" vertex is determined via a greedy traversal, always descending into the subtree with the most nodes. The key intuition is that each non-optimal link selected by the algorithm eliminates a substantial portion of the tree, thereby limiting the adversary's ability to mislead the algorithm in future steps. This structural property allows us to bound the total number of such mistakes, ultimately yielding the desired competitive ratio.

We then extend our focus to the general weighted Tree Augmentation Problem (WTAP), presenting a randomized combinatorial algorithm that achieves an $O(\log n)$-competitive ratio. Unlike previous primal-dual approaches, our method relies on structural reductions and geometric insights. We utilize a heavy path decomposition to reduce the tree instance into a collection of weighted Path Augmentation Problem (WPAP) instances. Crucially, this de-

composition allows us to classify links on a path as either *rooted* (incident to the path's upper endpoint) or *unrooted* (arbitrary segments). To solve these instances efficiently, we introduce preprocessing techniques, including geometric pruning for the rooted links and an interval flattening procedure for the unrooted links, that bound the frequency of available options. The core algorithm employs a randomized oracle to select unrooted links and maintains a buy-or-rent charging scheme to determine when to switch to rooted links, ensuring a balanced cost distribution relative to the optimal solution.

To complement our upper bounds, we establish an $\Omega(\log n)$ lower bound on the competitive ratio of online TAP, even under random-order arrivals, showing that the inherent hardness persists despite the weaker adversary. The construction is based on a complete binary tree in which each edge is replaced by a path whose length decreases geometrically with its level, resulting in a slightly larger tree. Links are placed between the root and each leaf, and the online requests consist of pairs of consecutive vertices along a uniformly random root-to-leaf path. As a result, the offline optimum can satisfy all requests using a single link. A key insight is that requests from only the higher levels are likely to appear early in the sequence, revealing little about the specific path being followed. This uncertainty forces the algorithm to commit to incorrect links, incurring an expected cost of $\Omega(\log n)$, matching our upper bounds up to constant factors.

Organization. The remainder of this chapter is organized as follows. In Section 5.2, we present the deterministic algorithm for unweighted TAP and its analysis. In Section 5.3, we present the combinatorial algorithm for the weighted setting. Finally, in Section 5.4, we present the lower bound for the random-order model.

5.2 Simple $O(\log n)$-competitive Algorithm for TAP

In this section, we describe a simple $O(\log n)$-competitive algorithm for the online Tree Augmentation Problem. The underlying graph is a rooted tree T, and without loss of generality, we may assume that all links are uplinks (i.e., one endpoint of a link is an ancestor of the other in T), at the cost of increasing the competitive ratio by at most a factor of 2. Furthermore, without loss of generality, we may assume that the online requests are *edge-requests* (i.e., the two requested vertices are adjacent in T). Both of these assumptions are standard and were also used in [117].

The inspiration for our algorithm comes from a simple $O(1)$-competitive algorithm for online interval set cover (or equivalently, the online path augmentation problem). In the online interval set cover problem, we are given a collection of intervals, and points that need to be covered

arrive online. The algorithm considers a newly arrived uncovered request together with all intervals that cover it. Among these feasible intervals, it selects the interval with the leftmost left endpoint and the interval with the rightmost right endpoint. It can then be shown that, for every interval I in the optimal solution, the algorithm selects at most two intervals that together cover all requests covered by I. This proves a competitive ratio of 2 for the algorithm.

Similarly, for online TAP, we intuitively wish to select a *topmost link* and a *bottommost link* among the links that cover a newly arrived uncovered edge request e_i. Given a rooted tree, we say that a link t_i is a topmost link for e_i if, among all links covering e_i, it covers the maximum number of edges on the path from e_i to the root. We now describe the bottommost link b_i for the request e_i. Starting from e_i, we traverse the tree along links covering e_i in the direction of increasing distance from the root. Whenever we encounter a branching node at which multiple branches still contain valid links (i.e., links covering e_i), we continue along the branch whose subtree contains the largest number of vertices. The traversal stops once only a single valid link remains, or when a leaf is reached. We call the resulting link the bottommost link for the request e_i. In both definitions above, ties may be broken arbitrarily.

The definition of the bottommost link that we use is based on the number of vertices contained in a subtree. The high-level intuition is that every incorrect choice made by the algorithm (i.e., selecting a non-optimal link) eliminates a large portion of the tree that an adversary could otherwise exploit to further force the algorithm into making additional incorrect choices. We therefore bound the number of such incorrect choices in order to obtain a good upper bound on the competitive ratio.

5.2.1 Algorithm

As mentioned earlier, the algorithm is very simple. For each uncovered request e, we select a topmost link and a bottommost link that cover e. A formal description of the algorithm is provided in Algorithm 1.

5.2.2 Analysis

Fix an optimal solution OPT. Partition the set of input requests $\{e_1, \ldots, e_s\}$ into $|\mathsf{OPT}|$ parts, one for each link in OPT, such that the part corresponding to a link $\ell \in \mathsf{OPT}$ contains only requests covered by ℓ. If a request is covered by multiple links in OPT, assign it arbitrarily to one of their corresponding parts.

Observe that our algorithm selects at most two links for each request. Therefore, to prove Theorem 4.5, it suffices to show that, for every part in the above partition, the algorithm selects at most $O(\log n)$ links. We establish the following lemma.

Algorithm 1 Online algorithm for unweighted online tree augmentation problem.

Input: A rooted tree $T = (V, E)$, a set of uplinks $L \subseteq \binom{V}{2}$. At each time step i, an online request $e_i \in E$ arrives.

Output: At each time step i, a subset of links $L'_i \subseteq L$ that cover $\{e_1, \ldots, e_i\}$ such that $L'_{i-1} \subseteq L'_i$, i.e., we add the set of links $L'_i \setminus L'_{i-1}$ to the existing solution L'_{i-1} so that the request e_i is covered.

$L'_0 \leftarrow \emptyset$
for each arriving request e_i at time step i **do**
  $L'_i \leftarrow L'_{i-1}$.
  if e_i is uncovered by L'_i **then**
    $L'_i \leftarrow L'_{i-1} \cup \{\text{topmost link covering } e_i\} \cup \{\text{bottommost link covering } e_i\}$.
  end if
end for
return L'_i.

Lemma 5.1. *For any set of requests assigned to a link in* OPT*, Algorithm 1 selects at most* $O(\log n)$ *links while serving those requests, where* $n = |V|$.

Proof. Fix a link $\ell_{\mathsf{OPT}} \in \mathsf{OPT}$, and let $R \subseteq \{e_1, \ldots, e_s\}$ be an arbitrary set of requests covered by ℓ_{OPT}. For any link ℓ, let $P(\ell)$ denote the unique path in the tree T between the endpoints of ℓ. Since every request in R is covered by ℓ_{OPT}, all requests in R lie on the path $P(\ell_{\mathsf{OPT}})$.

Let ALG_R denote the set of links selected by our algorithm while serving the requests in R. We will show that $|\mathsf{ALG}_R| = O(\log n)$.

For each link $b \in \mathsf{ALG}_R$ that is selected as a bottommost link for some uncovered request in R, we associate a unique vertex $w(b)$ on the path $P(\ell_{\mathsf{OPT}})$. Specifically, $w(b)$ is the vertex that lies on both $P(\ell_{\mathsf{OPT}})$ and $P(b)$ and is farthest from the root. We refer to $w(b)$ as the *branching vertex* of b.

We first show that distinct bottommost links have distinct branching vertices. Let b_1 and b_2 be two distinct bottommost links selected for requests r_1 and r_2, respectively, and assume without loss of generality that r_1 arrives before r_2. Since b_2 is selected for r_2, the request r_2 must be uncovered upon arrival. Both *edges* r_1 and r_2 lie on the path $P(\ell_{\mathsf{OPT}})$. We claim that r_2 cannot lie above r_1 on this path (that is, r_2 cannot be closer to the root than r_1). Indeed, when r_1 arrived, the algorithm selected a topmost link covering r_1. Since ℓ_{OPT} also covers r_1, it was one of the candidate links, and therefore the selected topmost link must cover every edge of $P(\ell_{\mathsf{OPT}})$ above r_1, including r_2. Hence r_2 would already have been covered before its arrival, a contradiction. Similarly, r_2 cannot lie on the portion of $P(\ell_{\mathsf{OPT}})$ between r_1 and the branching vertex $w(b_1)$. By definition, this segment is covered by the bottommost link b_1, and thus r_2 would again be covered before its arrival. Therefore, r_2 must lie strictly below both r_1

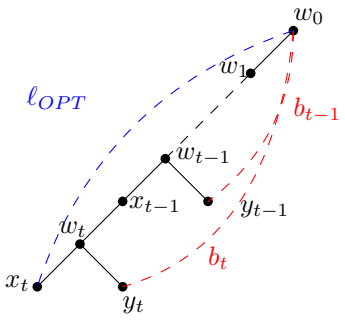


Figure 5.1: w_i is the branching vertex for the bottommost link b_i. x_i and y_i are its two children.

and $w(b_1)$ on $P(\ell_{\mathsf{OPT}})$. It follows that the branching vertex $w(b_2)$ lies strictly below $w(b_1)$, and hence $w(b_1) \neq w(b_2)$.

Let $w_1, \ldots, w_t$ be the branching vertices induced by the bottommost links selected by the algorithm for requests in R, ordered by increasing distance from the root along the path $P(\ell_{\mathsf{OPT}})$. We claim that there are exactly t requests in R that are uncovered upon arrival. To see this, observe that each uncovered request $r \in R$ causes the algorithm to select a distinct bottommost link (if two requests shared the same bottommost link, the one arriving later would already be covered by it, a contradiction). Since we established that distinct bottommost links have distinct branching vertices, the number of uncovered requests in R must equal the number of branching vertices, which is t. Since the algorithm selects at most two links for each uncovered request, it follows that

$$|\mathsf{ALG}_R| \leq 2t.$$

Bound on t. Let $N(v)$ denote the number of vertices in the subtree rooted at a vertex v. We first show that, assuming

$$N(w_i) \geq 1 + 2N(w_{i+1})$$

for every $i \in \{1, \ldots, t-1\}$, we obtain the desired bound on t. Indeed, the above inequality immediately implies that

$$N(w_1) \geq 1 + 2 + 2^2 + \cdots + 2^{t-1} N(w_t) \geq 2^t - 1.$$

Since $N(w_1) \leq n$, it follows that

$$n \geq N(w_1) \geq 2^t - 1,$$

and hence $t = O(\log n)$. Combining this with the bound $|\mathsf{ALG}_R| \leq 2t$, we conclude that

$$|\mathsf{ALG}_R| = O(\log n).$$

It remains to prove the claim that

$$N(w_i) \geq 1 + 2N(w_{i+1})$$

for every $i \in \{1, \ldots, t-1\}$. Fix any such i. We will argue that w_i has two distinguished children. One child lies on the path $P(\ell_{\mathsf{OPT}})$; denote it by x.

Let b be the bottommost link whose branching vertex is w_i, i.e., $w(b) = w_i$. By the definition of a branching vertex, either w_i is an endpoint of b, or the path $P(b)$ branches away from $P(\ell_{\mathsf{OPT}})$ at w_i. The first case is impossible, since then ℓ_{OPT} would be a strictly better candidate for the corresponding request, contradicting the choice of b as a bottommost link. Therefore, $P(b)$ must branch away from $P(\ell_{\mathsf{OPT}})$ at w_i. Let y denote the child of w_i that lies on $P(b)$.

By the definition of a bottommost link, we have $N(y) \geq N(x)$. Since the subtrees rooted at x and y are disjoint,

$$N(w_i) \geq 1 + N(x) + N(y) \geq 1 + 2N(x).$$

Finally, because w_{i+1} lies below w_i on the path $P(\ell_{\mathsf{OPT}})$, the subtree rooted at w_{i+1} is contained in the subtree rooted at x. Hence $N(x) \geq N(w_{i+1})$, and therefore

$$N(w_i) \geq 1 + 2N(w_{i+1}),$$

as required. □

5.3 Combinatorial Algorithm for Online WTAP

In this section, we present a combinatorial algorithm for the weighted Tree Augmentation Problem (WTAP). In the weighted setting, the input links have an associated weight, given by the weight function $w : \mathcal{L} \to \mathbb{R}^+$. For any set of links $S \subseteq \mathcal{L}$, we define $w(S) = \sum_{\ell \in S} w(\ell)$ as the total weight of the set.

5.3.1 Reduction from Online WTAP to Nice Online WPAP

We begin by summarizing a reduction established by Naor, Umboh, and Williamson [117]. The core difficulty in online WTAP arises from the arbitrary structure of the tree. To handle this,

the tree T is decomposed into a set of paths with low interaction width. Specifically, using the heavy path decomposition technique of Sleator and Tarjan [121], any tree T on n vertices admits a *rooted path decomposition* $\mathcal{P}$ of width $O(\log n)$. This means that any simple path in T intersects at most $O(\log n)$ paths in $\mathcal{P}$, and that the least common ancestor of the vertices on any path in $\mathcal{P}$ is an endpoint of the path, which we call its root.

This decomposition allows us to view the problem as a collection of online WPAP instances, one for each path $Q \in \mathcal{P}$. When a request (edge) e arrives in the tree, it maps to a sub-request on a specific path Q in the decomposition. To service these requests, any link ℓ on the tree provides coverage on the paths it intersects. We formalize this via the notion of a *projection*.

Definition 5.1 (Projection of a Link)**.** *Suppose $\mathcal{P}$ is a rooted path decomposition of tree T and ℓ a link. For $Q \in \mathcal{P}$, let $\pi_Q(\ell)$ be the link whose endpoints are endpoints of the path $P(\ell) \cap Q$; we call $\pi_Q(\ell)$ the projection of ℓ onto Q. We say that ℓ is Q-rooted if one of the endpoints of $\pi_Q(\ell)$ is the root of Q, and Q-non-rooted otherwise. See Figure 5.2 for an illustration.*

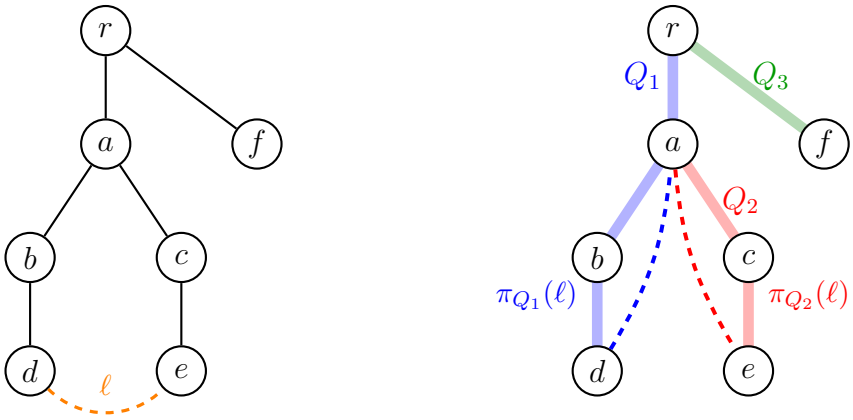


Figure 5.2: Path decomposition of tree rooted at r, into three paths Q_1, Q_2, and Q_3. The link ℓ on the tree gets projected as a Q_2-rooted link $\pi_{Q_2}(\ell)$, and a Q_1-non-rooted link $\pi_{Q_1}(\ell)$.

Crucially, the decomposition ensures that for any link ℓ, its non-empty projections are rooted for all but at most one path $Q \in \mathcal{P}$. This structural insight motivates the definition of a *Nice Algorithm* for WPAP.

Definition 5.2 (Nice Solution)**.** *Consider an instance of WPAP on a path with a designated root. A solution* ALG *is called* nice *if, for any feasible solution* SOL *partitioned into rooted links R^* and unrooted links N^*, the total weight satisfies:*

$$w(\mathsf{ALG}) \leq O(1) \cdot w(R^*) + O(\log n) \cdot w(N^*).$$

An algorithm that always outputs nice solutions is called a nice algorithm.

If we possess a deterministic nice algorithm for WPAP, we can run independent instances of it on each path in the decomposition. Since any link in the optimal tree solution acts as a non-rooted link in at most one, and as a rooted link in the remaining path sub-problems, summing the weights yields an overall competitive ratio of $O(\log n)$ for WTAP. While [117] provides a primal-dual algorithm to achieve this, we present a purely combinatorial randomized approach below.

5.3.2 Preprocessing Links

For a given instance of TAP, we can arbitrarily root the tree, and obtain a rooted path decomposition on it. We can now project the given set of input links onto the paths in the decomposition, and classify the resulting links into the two sets of rooted links R and unrooted links N. We then identify and remove "redundant" links. A link $\ell \in R$ ($\ell \in N$ respectively) is redundant if there exists another link $k \in R$ ($k \in N$ respectively) such that $w(\ell) \geq w(k)$ and $P(\ell)$ is a subset of $P(k)$. This ensures that link weight is monotonic with link length.

5.3.2.1 Geometric Pruning for Rooted Links

For the set of rooted links R, we iteratively select the link $r \in R$ with the largest weight, add it to our active set R', and discard all other links in R with weights in the interval $[w(r)/2, w(r)]$. We repeat this until R is empty. This ensures that the weights of links in R' are upper-bounded by a geometric progression. Note that this procedure can increase the optimal cost of an instance by at most a factor of 2.

5.3.2.2 Frequency Reduction for Unrooted Links

To ensure that one of the subroutines we will design (called the Unrooted Oracle) performs efficiently, we require the set of available options to have low depth (frequency). Since the path decomposition reduces the problem to a line, the unrooted links N are geometrically equivalent to a set of weighted intervals. We therefore define a general procedure for "flattening" a set of weighted intervals to bound their depth by $O(\log n)$. This flattening technique is a key combinatorial tool in its own right, and we will leverage it again in a different context in Chapter 6 for our dynamic Geometric Set Cover algorithm.

Let $\mathcal{I}$ be a set of weighted intervals on the real line. We transform $\mathcal{I}$ into a new set $\mathcal{I}'$ where the depth of any point is bounded by $O(\log n)$, while preserving the optimal covering cost within a constant factor.

Restricted Setting: Intervals Sharing a Common Point. First, consider a restricted subset $\mathcal{I}_p \subseteq \mathcal{I}$ of intervals that all contain a common point p. We decompose each interval $I \in \mathcal{I}_p$ into a left segment I_L and a right segment I_R relative to p, retaining the original weight $w(I)$ for both segments. We process the right segments (denoted $\mathcal{I}_R$) as follows (the left side is symmetric):

1. **Pruning Redundancy:** We again remove any redundant intervals that might have formed because of the decomposition into left and right segments. Let $\text{Rem}(\mathcal{I}_R)$ denote the set of remaining non-redundant intervals.

2. **Geometric Decomposition:** We iteratively select intervals from $\text{Rem}(\mathcal{I}_R)$. Let I_1 be the interval in $\text{Rem}(\mathcal{I}_R)$ with the largest weight. Initialize the output set $\mathcal{I}'_R \leftarrow \{I_1\}$. We perform the following in rounds, maintaining a current interval I_{curr} (initially I_1):

   - Find the interval $I_{\text{next}} \in \text{Rem}(\mathcal{I}_R)$ with the largest weight strictly less than $w(I_{\text{curr}})/2$.
   - If such an interval exists:
     - Add the segment $I_{\text{curr}} \setminus I_{\text{next}}$ to $\mathcal{I}'_R$ with weight $w(I_{\text{curr}})$.
     - Update $I_{\text{curr}} \leftarrow I_{\text{next}}$.
   - If no such interval exists:
     - Add the remaining segment I_{curr} to $\mathcal{I}'_R$ with weight $w(I_{\text{curr}})$.
     - Terminate.

The result is a set of disjoint intervals $\mathcal{I}'_R$ that partition the union of the original segments in $\mathcal{I}_R$. See Figure 5.3 for an illustration.

General Case: The Interval Tree. To handle the general case where intervals do not share a single point, we construct a standard interval tree (refer to Section 3.5.1 for details) on the set of given intervals. Each interval $I \in \mathcal{I}$ is assigned to the unique highest node v in the interval tree such that I covers the split point of v. Let $\mathcal{I}(v)$ be the set of intervals assigned to node v.

Since all intervals in $\mathcal{I}(v)$ share the split point of v, we apply the *Restricted Setting* procedure described above to generate a flattened set $\mathcal{I}'(v)$. The final set of flattened intervals is the union over all tree nodes:

$$\mathcal{I}' = \bigcup_v \mathcal{I}'(v).$$

Lemma 5.2 (Interval Flattening Lemma)**.** *Given a set of weighted intervals $\mathcal{I}$, the construction above produces a set $\mathcal{I}'$ satisfying the following properties:*

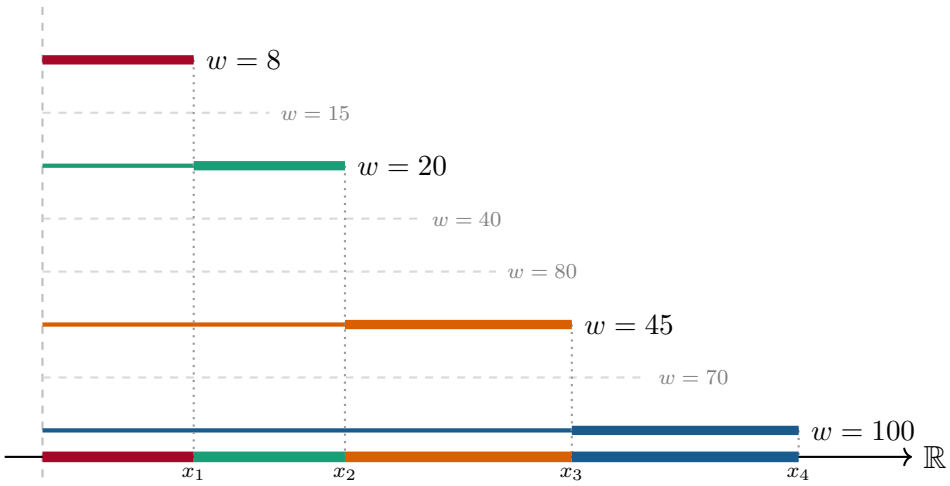


Figure 5.3: A subset of intervals selected in $\mathcal{I}'_R$ is shown in solid colors. Overlapping regions are "chopped" to create a partition of disjoint segments on the real line.

1. Structure preservation: *Every interval in $\mathcal{I}'$ is a sub-segment of some interval in $\mathcal{I}$ with the same weight.*

2. Sparse cover: *For any original interval $I \in \mathcal{I}$, there exists a subset of intervals in $\mathcal{I}'$ that completely covers I with total weight at most $8 \cdot w(I)$.*

3. Small frequency: *The depth (number of intervals covering any point) in $\mathcal{I}'$ is $O(\log n)$.*

Proof. *Property 1* follows directly from the construction, as output intervals are chopped segments of input intervals retaining the original weight.

Property 2: In the restricted setting, the weights of the selected intervals decrease geometrically by a factor of at least 2. Summing the geometric series for the segments covering an original interval I yields a cost at most $4 \cdot w(I)$ for the right side and $4 \cdot w(I)$ for the left side, totaling $8 \cdot w(I)$.

Property 3: In the restricted setting, the generated intervals $\mathcal{I}'(v)$ are disjoint (depth 1). Since the interval tree has height $O(\log n)$, and any point on the line is contained in the restricted sets of at most $O(\log n)$ nodes (its ancestors in the tree), the total depth is bounded by $O(\log n)$. □

Lemma 5.3. *The time taken to construct the set $\mathcal{I}'$ is $O(n \log n)$.*

Proof. For a given collection of intervals $\mathcal{I}$ in the restricted setting, it is easy to verify that the flatten subroutine can be performed in $O(|\mathcal{I}| \log |\mathcal{I}|)$ time. Therefore, the total time spent at all the nodes in the interval tree will be $\sum_v O(|\mathcal{I}(v)| \log |\mathcal{I}(v)|) = O(n \log n)$, since each interval is stored at exactly one node. □

Application to Unrooted Links. We apply Lemma 5.2 to the input set of unrooted links N. This yields a processed set of links N' with frequency $f = O(\log n)$. We replace the input set N with N' for the remainder of the algorithm. Although the cost of the optimal solution on N' may increase, it is bounded by $8 \cdot w(N^*)$ (by Property 2), which is absorbed into the asymptotic competitive ratio.

5.3.3 The Algorithm

We maintain a variable $\rho(r)$ for every rooted link $r \in R'$, initialized to 0. This variable represents a charge accumulated by the rooted link when it is bypassed in favor of an unrooted link.

When a new edge request p arrives that is not yet covered:

Case 1: Covered by one type only.

- If p is covered only by links in R', pick the link $\ell_r \in R'$ with the minimum weight covering p.
- If p is covered only by links in N', pick a link $\ell_u \in N'$ covering p using the Unrooted Oracle described below.

Case 2: Covered by both.

- Let ℓ_r be the minimum weight link in R' covering p.
- Let ℓ_u be the link returned by the Unrooted Oracle for N' covering p.
- Comparison step:
  - If $w(\ell_u) + \rho(\ell_r) \geq w(\ell_r)$: Pick ℓ_r and add it to the solution.
  - Else: Pick ℓ_u and add it to the solution. Update the charge of the rooted link:

$$\rho(\ell_r) \leftarrow \rho(\ell_r) + w(\ell_u).$$

Unrooted Oracle. For a request p covered by a subset of unrooted links $V_p \subseteq N'$:

1. Calculate the normalization factor $\alpha_p = \left(\sum_{j \in V_p} \frac{1}{w(j)}\right)^{-1}$.
2. Independently sample a link $i \in V_p$ with probability $\Pr(\text{pick } i) = \frac{\alpha_p}{w(i)}$.

Algorithm 2 Combinatorial Algorithm for Online Weighted TAP

Input: A tree T, set of links $\mathcal{L}$ partitioned into rooted R and unrooted N, weights $w : \mathcal{L} \to \mathbb{R}^+$. Request p arrives online.

Output: A set of links S_{ALG} covering all requests.

Preprocessing:

Remove redundant links from R and N. {As defined in Section 5.3.2}
$R' \leftarrow \mathsf{GeometricPruning}(R)$. {See Section 5.3.2.1}
$N' \leftarrow \mathsf{Flattening}(N)$. {See Lemma 5.2}

Algorithm:

Initialize charges: $\rho(r) \leftarrow 0$ for all $r \in R'$.
$S_{\mathsf{ALG}} \leftarrow \emptyset$.
while uncovered request p arrives **do**
  Let $R'_p \subseteq R'$ and $N'_p \subseteq N'$ be the links covering p.
  if $R'_p \neq \emptyset$ **then**
    $\ell_r \leftarrow \operatorname{argmin}_{r \in R'_p} w(r)$
  end if
  if $N'_p \neq \emptyset$ **then**
    $\ell_u \leftarrow \mathsf{UnrootedOracle}(p, N'_p)$
  end if
  if $R'_p \neq \emptyset$ **and** ($N'_p = \emptyset$ **or** $w(\ell_u) + \rho(\ell_r) \geq w(\ell_r)$) **then**
    $S_{\mathsf{ALG}} \leftarrow S_{\mathsf{ALG}} \cup \{\ell_r\}$
  else if $N'_p \neq \emptyset$ **then**
    $S_{\mathsf{ALG}} \leftarrow S_{\mathsf{ALG}} \cup \{\ell_u\}$
    if $R'_p \neq \emptyset$ **then**
      $\rho(\ell_r) \leftarrow \rho(\ell_r) + w(\ell_u)$
    end if
  end if
end while
return S_{ALG}

5.3.4 Analysis

We now show that ALG is a nice algorithm, which implies the required $O(\log n)$-competitive algorithm for WTAP.

Theorem 5.1. *For any feasible solution* SOL *to the WPAP instance, which can be partitioned into rooted links R^* and unrooted links N^*, the algorithm* ALG *returns a solution satisfying:*

$$\mathbb{E}[w(\mathsf{ALG})] \leq O(1) \cdot w(R^*) + O(\log n) \cdot w(N^*).$$

Proof. We define two sets of links. Let S_{oracle} be the set of unrooted links returned by the Unrooted Oracle for all requests encountered (regardless of whether the algorithm eventually picks them or swaps to a rooted link). Let $R_{\text{picked}} \subseteq R'$ be the set of rooted links added to the solution by the algorithm. The total weight of the solution ALG is bounded by $w(S_{\text{oracle}}) + w(R_{\text{picked}})$. We analyze these components relative to a partition of the feasible solution $\mathsf{SOL} = R^* \cup N^*$.

Part 1: Analysis of Unrooted Oracle Cost. We analyze the expected weight of S_{oracle} for the subset of requests covered by N^*. Let $\ell^* \in N^*$ be a link in the feasible solution, and let $p_1, \ldots, p_k$ be the requests covered by ℓ^*.

We prove via induction on k that the expected weight of links generated by the Oracle to cover these requests is at most $f \cdot w(\ell^*)$, where $f \in O(\log n)$.

Inductive Hypothesis: Assume that for any sequence of $k-1$ requests covered by ℓ^*, the expected weight of the set of links H_{k-1} generated by the Oracle satisfies $\mathbb{E}[w(H_{k-1})] \leq f \cdot w(\ell^*)$.

Base Case: For $k = 0$, the weight is 0.

Inductive Step: Consider the request sequence $\{p_1, \ldots, p_k\}$. Let p_1 (denoted as p for ease of exposition) be the current request. Let H_{rem} be the set of links generated by the Oracle for the remaining requests $\{p_2, \ldots, p_k\}$. When processing p, the Oracle calculates $\alpha_p = \left(\sum_{j \in V_p} \frac{1}{w(j)}\right)^{-1}$. The Oracle returns ℓ^* with probability $\frac{\alpha_p}{w(\ell^*)}$. If ℓ^* is returned, it covers p and all remaining requests $\{p_2, \ldots, p_k\}$ at no extra cost. If the Oracle returns some $i \in V_p \setminus \{\ell^*\}$, we pay $w(i)$. In the worst case, i does not cover the remaining requests, so we incur the additional expected weight $\mathbb{E}[w(H_{\text{rem}})]$.

$$\begin{aligned}\mathbb{E}[w(H_{\ell^*})] &\leq \Pr(\text{pick } \ell^*) \cdot w(\ell^*) + \sum_{i \in V_p \setminus \{\ell^*\}} \Pr(\text{pick } i) \cdot [w(i) + \mathbb{E}[w(H_{\text{rem}})]] \\ &= \frac{\alpha_p}{w(\ell^*)} \cdot w(\ell^*) + \sum_{i \in V_p \setminus \{\ell^*\}} \frac{\alpha_p}{w(i)} \cdot [w(i) + \mathbb{E}[w(H_{\text{rem}})]]\end{aligned}$$

$$= \alpha_p + \sum_{i \in V_p \setminus \{\ell^*\}} \alpha_p + \left(\sum_{i \in V_p \setminus \{\ell^*\}} \frac{\alpha_p}{w(i)} \right) \mathbb{E}[w(H_{\text{rem}})]$$
$$= \alpha_p |V_p| + \left(1 - \frac{\alpha_p}{w(\ell^*)}\right) \mathbb{E}[w(H_{\text{rem}})]$$

Using the inductive hypothesis $\mathbb{E}[w(H_{\text{rem}})] \leq f \cdot w(\ell^*)$ and $|V_p| \leq f$:

$$\mathbb{E}[w(H_{\ell^*})] \leq \alpha_p f + \left(1 - \frac{\alpha_p}{w(\ell^*)}\right) f \cdot w(\ell^*)$$
$$= f \cdot w(\ell^*).$$

By summing over all unrooted links $\ell^* \in N^*$, we get,

$$\mathbb{E}[w(S_{\text{oracle}})] \leq \sum_{\ell^* \in N^*} f \cdot w(\ell^*) = f \cdot w(N^*) = O(\log n) \cdot w(N^*). \tag{5.1}$$

Note that the Oracle is only active for requests not covered by the current rooted set, so requests covered by R^* contribute 0 or are handled in Part 2.

Part 2: Analysis of Algorithm and Rooted Costs. We now bound the total weight of ALG. The algorithm includes an unrooted link $\ell_u \in S_{\text{oracle}}$ in the solution only if it is cheaper than picking a corresponding rooted link (considering charges). If the algorithm picks a rooted link ℓ_r, it is because $w(\ell_r) \leq w(\ell_u) + \rho(\ell_r)$, or because unrooted links do not cover the particular request.

Consider some rooted link $\ell^* \in R^*$. Since rooted links have weights which decrease geometrically (by our preprocessing), the sum of weights of rooted links required to cover ℓ^* is at most $4w(\ell^*)$. Any additional rooted links picked by the algorithm (for covering requests in N^*) are paid for by the charges ρ accumulated from S_{oracle} (since it also includes links which are selected by the oracle, but overridden by our algorithm). Specifically, we can bound:

$$w(R_{\text{picked}}) \leq O(1) \cdot w(R^*) + w(S_{\text{oracle}}).$$

Combining Part 1 and Part 2:

$$\mathbb{E}[w(\mathsf{ALG})] \leq \mathbb{E}[w(S_{\text{oracle}})] + \mathbb{E}[w(R_{\text{picked}})]$$
$$\leq \mathbb{E}[w(S_{\text{oracle}})] + [O(1) \cdot w(R^*) + \mathbb{E}[w(S_{\text{oracle}})]]$$
$$= 2 \cdot \mathbb{E}[w(S_{\text{oracle}})] + O(1) \cdot w(R^*)$$

$$\leq 2f \cdot w(N^*) + O(1) \cdot w(R^*) \qquad \text{(by Equation (5.1))}$$
$$\leq O(\log n) \cdot w(N^*) + O(1) \cdot w(R^*). \qquad \square$$

5.4 An $\Omega(\log n)$ Lower Bound for TAP under Random-Order Arrivals

Consider a complete binary tree B of height $h \approx \log n$. We subdivide the edges of B to obtain a new tree T as follows: edges at depth d in B (where the edges incident to the root are at depth 0) are replaced by paths of length $L_d = 2^{h-1-d}$ in T. Thus, edges incident to the root are replaced by paths of length 2^{h-1}, edges at the next level by paths of length 2^{h-2}, and so on. The branching vertices in T correspond one-to-one with the vertices of B. We say an edge in T belongs to *level* i (for $1 \leq i \leq h$) if it lies on a subdivided path replacing an edge at depth $i-1$ in B. Note that the number of vertices in T is $O(n \log n)$, and T has the same number of leaves as B.

The tree augmentation instance consists of the tree T and a set of candidate links. For each leaf ℓ of T, we include a link (r, ℓ), where r is the root. The adversary selects a leaf ℓ^* uniformly at random. The set of online requests consists of all edges on the unique path from r to ℓ^* in T. The requests are revealed to the online algorithm in a uniformly random order (i.e., every new request has equal probability of being the one that is presented next). Without loss of generality, we assume that when the online algorithm is presented with an uncovered request, it picks precisely one link covering that request.

We define a request e at level i to be a *fresh* request if e appears in the input sequence before any other request from levels $\{i, i+1, \ldots, h\}$. If a request at level i is fresh, the algorithm has not yet seen any requests from the subtrees below level i. Since B is a binary tree, the path to ℓ^* proceeds through one of the two children of the current branching node. However, the algorithm must immediately select a link (r, ℓ) to cover the fresh request. With probability at least $1/2$ (depending on the specific link chosen), the leaf ℓ lies in the subtree of the *other* child. Let this event be called 'Wrong Branch'. If this occurs, the selected link covers the current level i but fails to cover any edges in levels $i+1$ through h. Consequently, the algorithm will be forced to pick a new link when requests from those lower levels eventually arrive.

Let Y_i be an indicator random variable for the event that the algorithm makes a "mistake" at level i. A mistake occurs if:

- A request from level i is fresh (it arrives before any request from levels $> i$), and
- the algorithm selects a link that does not cover the true path in levels $> i$.

Note that the total cost is bounded below by $\sum_{i=1}^{h} Y_i$.

First, we lower bound the probability that a request at level i is fresh. The number of edges in level i is $L_{i-1} = 2^{h-i}$. The total number of edges in all levels $j \geq i$ is:

$$\sum_{j=i}^{h} 2^{h-j} = 2^{h-i} + 2^{h-i-1} + \cdots + 1 = 2^{h-i+1} - 1.$$

The probability that the first request revealed from the set of edges in levels $\{i, \ldots, h\}$ belongs to level i is proportional to the number of edges:

$$\Pr[\text{Level } i \text{ is fresh}] = \frac{2^{h-i}}{2^{h-i+1} - 1} > \frac{2^{h-i}}{2^{h-i+1}} = \frac{1}{2}.$$

Conditioned on level i being fresh, the algorithm has zero information about the random walk's direction beyond level i. Thus, with probability at least $1/2$, the chosen link fails to cover the remainder of the path. These events are independent across levels because a mistake at level i effectively "resets" the covering problem for the subtree rooted at the end of the level i path.

By linearity of expectation, the expected cost is:

$$\begin{aligned}
\mathbb{E}[\text{Cost}] &\geq \sum_{i=1}^{h} \Pr[Y_i = 1] \\
&= \sum_{i=1}^{h} \Pr[\text{Level } i \text{ is fresh}] \cdot \Pr[\text{Wrong branch} \mid \text{Level } i \text{ is fresh}] \\
&\geq \sum_{i=1}^{h} \left(\frac{1}{2}\right) \cdot \left(\frac{1}{2}\right) \\
&= \frac{h}{4} = \Omega(\log n).
\end{aligned}$$

This establishes the $\Omega(\log n)$ lower bound.

Chapter 6

Dynamic Weighted Set Cover for Hyperrectangles

6.1 Introduction

In Section 3.4, we defined the fundamental Geometric Set Cover and Geometric Hitting Set problems. In this chapter, we extend our study to their dynamic and weighted variants, specifically for axis-aligned hyperrectangles in $\mathbb{R}^d$.

We focus on the *weighted* versions of these problems. In weighted Geometric Set Cover, each object $S \in \mathcal{S}$ has an associated weight $w(S) \geq 0$, and the goal is to minimize the total weight of a sub-collection $\mathcal{S}'$ that covers the point set P. Dually, in weighted Geometric Hitting Set, we seek a minimum-weight subset of points $P' \subseteq P$ that intersects every object in $\mathcal{S}$. Let $n = |P|$ be the number of points, and $m = |\mathcal{S}|$ be the number of sets. An important parameter in this context is the *frequency* f of the set system, defined as the maximum number of sets in $\mathcal{S}$ that contain any single point in P.

The general Set Cover problem is NP-hard, and admits an $O(\log n)$ approximation [48], but geometric instances often allow for better ratios by exploiting the underlying spatial structure [29, 130, 112, 6]. Similarly, for offline Geometric Hitting Set, specific objects allow for significantly improved approximation ratios, such as an $O(\log \log \mathsf{OPT})$-approximation for rectangles [9] and a PTAS for squares [113]. However, these results often do not translate directly to the dynamic setting.

6.1.1 Dynamic Geometric Set Cover and Geometric Hitting Set

While the offline variants are well-understood, the dynamic variants of geometric Set Cover and Hitting Set present significant challenges. In the dynamic Geometric Set Cover problem, the sets are given offline and at each time step a point is inserted or deleted. Conversely, in the dynamic Geometric Hitting Set problem, the points are given offline and at each time step a set is inserted or deleted. In both cases, the goal is to design an algorithm that updates the approximate solution as quickly as possible. The performance of such an algorithm is determined by its approximation factor and its update time, where achieving polylogarithmic update times is particularly desirable.

Regarding the state-of-the-art for geometric instances, Agarwal, Chang, Suri, Xiao, and Xue [4] studied the fully dynamic setting and presented $(1+\varepsilon)$- and $O(1)$-approximations with sublinear update times for intervals and unit squares, respectively. Chan and He [38] extended these results to Set Cover with arbitrary squares. Recently, Chan, He, Suri, and Xue [39] provided a $(1+\varepsilon)$-approximation for intervals in $O(\log^3 n/\varepsilon^3)$ amortized update time. They also gave $O(1)$-approximation for dynamic Set Cover for unit squares, arbitrary squares, and weighted intervals in amortized update time of $2^{O(\sqrt{\log n})}$, $n^{1/2+\varepsilon}$, and $2^{O(\sqrt{\log n \log\log n})}$, respectively.

In our preliminary work [95], we addressed the unweighted version of these problems for d-dimensional hyperrectangles. By utilizing an *extended quad-tree* approach and a *frequency reduction* technique, we obtained $(\log n)^{O(d)}$ approximation ratios with $(\log n)^{O(d)}$ worst-case update times. This was the first result to achieve polylogarithmic update times for hyperrectangles (and even squares), partially answering an open question posed by Chan, He, Suri, and Xue [39]. However, that approach was primarily suited for the unweighted setting and lacked the refinement necessary to eliminate the dependence on W or n in the weighted case.

A major hurdle in the dynamic setting is handling weighted instances. Standard techniques for dynamic Set Cover often depend linearly on the frequency f and, in the weighted case, incur factors dependent on W (the ratio of the maximum to minimum weight) in the approximation ratio. For geometric instances, where points are defined by coordinates rather than explicit set lists, polylogarithmic bounds that are independent of both n and W were not previously known.

Abboud, Addanki, Grandoni, Panigrahi, and Saha [1] showed that under the Strong Exponential Time Hypothesis, any general (dynamic) Set Cover algorithm with an amortized update time of $O(f^{1-\varepsilon})$ must have an approximation factor of $\Omega(n^{\alpha})$ for some constant $\alpha > 0$,

where f can be as large as $\Theta(m)$. This hardness result makes the study of geometric variants of dynamic Set Cover particularly important, as the geometry may allow us to bypass these lower bounds.

6.1.2 Our Results and Techniques

In this chapter, we focus on dynamic algorithms for high-dimensional geometric objects. We present algorithms for dynamic weighted Set Cover and Hitting Set for hyperrectangles in d dimensions that are simpler and more efficient than previous approaches.

Dynamic Set Cover for Hyperrectangles. In Section 6.2, we present a dynamic weighted Set Cover algorithm for hyperrectangles in d dimensions. This work improves upon the preliminary results in [95], which handled the unweighted case or incurred a dependence on the maximum weight W in the approximation factor. Our improved algorithm removes this dependence entirely, achieving a strict polylogarithmic approximation factor $O(\log^{d-1} m \cdot \log n)$ with an amortized update time of $O(\log^{d} m \cdot \log n)$ (see Table 6.1 and Theorem 6.1).

To the best of our knowledge, this is the first dynamic algorithm for hyperrectangles that achieves polylogarithmic bounds for both update time and approximation ratio in the weighted setting without depending on the weight values.

Our technique relies on constructing a mapping from the original hyperrectangles to a new family of objects. This mapping is designed to satisfy three key properties:

1. *Sparse cover:* Every original hyperrectangle can be covered by a collection of new objects with a small total weight increase.
2. *Reverse mapping:* Every new object maps efficiently back to an original hyperrectangle.
3. *Small frequency:* The depth of any point with respect to the new objects is polylogarithmic.

To achieve these properties, we introduce a refined *flattening* procedure for intervals on the real line. Unlike previous weight-sensitive approaches, this procedure processes intervals based on relative weights to generate a sparse cover directly. This is combined with nested segment trees to efficiently scale to higher dimensions. This simplification not only removes the W factor but also shaves logarithmic factors compared to existing unweighted techniques.

Running a general dynamic Set Cover algorithm [80] on this new sparse set system yields the result with amortized update times. Furthermore, our reduction framework allows us to seamlessly plug in recent advances as highly efficient black-boxes. By substituting the general dynamic algorithm with the very recent breakthrough by Bhattacharya, Cen, and Panigrahi [18],

we immediately grant our geometric algorithms strictly *worst-case* update times of $O(\log^d m \cdot \log^3 n)$ and bounded worst-case recourse.

Dynamic Hitting Set for Hyperrectangles. In Section 6.3, we further adapt the above dynamic Set Cover algorithm to the dynamic weighted Hitting Set problem for d-dimensional hyperrectangles. In this setting, the weighted points are given offline and at each time step a hyperrectangle is inserted or deleted. We utilize a geometric duality to transform the Hitting Set instance in $\mathbb{R}^d$ into a Set Cover instance in $\mathbb{R}^{2d}$, maintaining polylogarithmic update time and approximation factor.

Problem	Objects	Approximation ratio	Update time
Set Cover	2-D unit squares	$O(1)$ [4]	$(\log n)^{O(1)}$
	d-D hyperrectangles	$O(\log^{d-1} m \cdot \log n)$ [Thm 6.1]	$O(\log^d m \cdot \log n)$
Hitting Set	2D unit squares	$O(1)$ [4]	$(\log n)^{O(1)}$
	d-D hyperrectangles	$O(\log^{2d-1} n \cdot \log m)$ [Thm 6.2]	$O(\log^{2d} n \cdot \log m)$

Table 6.1: Dynamic algorithms for Geometric Set Cover and Hitting Set. Update times in [4] are for the unweighted case, while ours hold for the weighted setting. All update times are amortized.

6.1.3 Related Work

Dynamic Set Cover. The general Set Cover problem is well-studied in the dynamic setting. Gupta, Krishnaswamy, Kumar, and Panigrahi [80] initiated the study and provided an $O(\log n)$-approximation algorithm with $O(f \log n)$ amortized update time, even in the weighted setting. Following this, there has been a series of works [1, 11, 19, 20, 21, 22, 82]. For example, Bhattacharya, Henzinger, Nanongkai, and Wu [22] gave a deterministic $(1+\varepsilon)f$-approximation with polylogarithmic amortized update time, and Assadi and Solomon [11] gave a randomized f-approximation algorithm with $O(f^2)$ amortized update time.

Recently, there have been several notable follow-up works. Bukov, Solomon, and Zhang [31] improved the earlier result [22] to obtain a deterministic algorithm with $O\left(\frac{f}{\varepsilon} \log f + \frac{f}{\varepsilon^3} + \frac{f \log W}{\varepsilon^2}\right)$-amortized update time, while maintaining a $(1+\varepsilon)f$ approximation ratio. Meanwhile, Solomon and Uzrad [122] gave a $(1+\varepsilon)\ln n$-approximation with an $O(\texttt{poly}(1/\varepsilon) f \log n)$ amortized update time. Very recently, Bhattacharya, Cen, and Panigrahi [18] achieved a breakthrough for fully dynamic Set Cover by providing algorithms with simultaneous non-trivial *worst-case* bounds for both recourse and update time. They achieve near-optimal $O(\log n)$ and $O(f)$ approximation ratios with $O(\log n)$ worst-case recourse and $O(f \log^3 n)$ worst-case update time.

Online Set Cover and Hitting Set. A closely related model is the *online* setting, where elements arrive one by one and must be covered irrevocably. For general Set Cover, Alon, Awerbuch, and Azar [7] established an almost tight $O(\log n \log m)$ competitive ratio. In the geometric context, better ratios are sometimes possible, though results are sparse. Even and Smorodinsky [65] provided $O(\log n)$-competitive algorithms for online Hitting Set for intervals and unit disks. Recently Bhore, Gupta, and Kumar [23] presented an $O(\log n \log \log n)$-competitive algorithm for weighted online hitting set on set systems with bounded shallow-cell complexity.

There is also recent work that has explored online piercing and dominating sets [56, 57]. While this thesis focuses only on the dynamic model (where revocations are allowed), in our paper [95], we also give $O(\log n)$-competitive algorithms for online Set Cover on squares.

Offline Geometric Optimization. Offline Geometric Set Cover remains a central problem. Set Cover for rectangles is known to be APX-Hard [36], and the best known approximation ratio is $O(\log \mathsf{OPT})$ [49]. Mustafa, Raman, and Ray [111] gave quasipolynomial-time approximation schemes (QPTAS) for weighted Geometric Set Cover for pseudodisks and 3-D Halfspaces. Aronov, Ezra and Sharir [9] gave an $O(\log \log \mathsf{OPT})$-approximation algorithm for offline Hitting Set for rectangles. Varadarajan [130] introduced the method of quasi-uniform sampling for weighted Geometric Set Cover, later used by Chan, Grant, Könemann, and Sharpe [37] to provide improved approximation algorithms for several variants of weighted capacitated cover, weighted Set Cover, Hitting Set, and Dominating Set.

Organization. The rest of this chapter is organized as follows. In Section 6.2, we present our main algorithm for dynamic weighted Set Cover on d-dimensional hyperrectangles. This includes the extension of the 1-D flattening procedure to higher dimensions using segment trees in Section 6.2.3. In Section 6.3, we describe our results for the dynamic weighted Hitting Set problem, showing how a geometric duality transformation allows us to leverage our Set Cover results.

6.2 Weighted Dynamic Set Cover on Hyperrectangles

In this section we will design an algorithm to dynamically maintain an approximate weighted set cover on d-dimensional hyperrectangles. Recall that in this setting in each round either a point is inserted or deleted, and the goal is to maintain a set cover of the points with weighted hyperrectangles. The main result we prove in this section is the following.

Theorem 6.1. *There is an algorithm for dynamic weighted Set Cover for d-dimensional hyperrectangles with an approximation factor of $O(\log^{d-1} m \cdot \log n)$, amortized update time of*

$O(\log^d m \cdot \log n)$ *(or worst-case update time of* $O(\log^d m \cdot \log^3 n)$*), and a preprocessing time of* $O(m \log^d m)$.

We note here that in the context of axis-aligned hyperrectangles in $\mathbb{R}^d$ for a fixed dimension d, the number of sets m and the number of points n are polynomially related. Specifically, for a fixed set of n points, there are only $O(n^{2d})$ combinatorially distinct hyperrectangles (defined by the coordinates of the points). In the weighted setting, while multiple rectangles may cover the same subset of points, we only need to consider the one with the minimum weight for any given subset.

This implies that $\log m = O(\log n)$ and $\log n = O(\log m)$. Under this standard geometric assumption, the results in Theorem 6.1 simplify to an approximation ratio of $O(\log^d n)$ with an amortized update time of $O(\log^{d+1} n)$. Similarly, the Hitting Set bounds in Theorem 6.2 simplify to an $O(\log^{2d} n)$ approximation and $O(\log^{2d+1} n)$ update time. We maintain the variables m and n in our formal statements to explicitly show how the complexity depends on the set system's frequency versus the number of elements.

6.2.1 An Overview of the Algorithm

Our set system is $(P, \mathcal{R})$ where P is a dynamic set of n points and $\mathcal{R}$ is a fixed collection of m axis-aligned hyperrectangles in a d-dimensional space. For the dynamic Set Cover problem in the *general* setting, Gupta, Krishnaswamy, Kumar, and Panigrahi [80] obtained an algorithm with approximation factor $O(\log n)$ and amortized update time $O(f \log n)$, where f is the *frequency* of the set system (i.e., the maximum number of sets containing an element). In the general setting, the sets are fixed and an element is inserted or deleted in each round. Our goal is to use this algorithm for the general setting as a blackbox to solve our problem: one challenge which can be immediately observed is that the frequency of our set system $(P, \mathcal{R})$ can be as large as $\Omega(n)$, leading to a poor update time. The main technical contribution in this section is the construction of a new set system $(P, \mathcal{R}')$ with the following *three properties*:

- *Sparse cover.* For each hyperrectangle $R \in \mathcal{R}$ with weight $w(R)$, there is a collection of hyperrectangles in $\mathcal{R}'$ with total weight $O(w(R) \log^{d-1} m)$ which completely cover R.
- *Efficient reverse mapping.* For each hyperrectangle $R' \in \mathcal{R}'$, there is a hyperrectangle $R \in \mathcal{R}$ such that $R' \subseteq R$ and $w(R') = w(R)$.
- *Small frequency.* The *depth* of any point p w.r.t. $\mathcal{R}'$ is only $O(\log^d m)$ (where depth of p is the number of hyperrectangles in $\mathcal{R}'$ containing p). This in turn implies that the frequency of the set system $(P, \mathcal{R}')$ is $O(\log^d m)$.

Assuming the existence of $(P, \mathcal{R}')$ with the above three properties, our algorithm is the following:

Run the dynamic Set Cover *algorithm of Gupta, Krishnaswamy, Kumar, and Panigrahi [80] on* $(P, \mathcal{R}')$.

By the third property, the amortized update time of our algorithm will be $O(f \log n) = O(\log^d m \cdot \log n)$. Alternatively, by running the recent dynamic Set Cover algorithm of Bhattacharya, Cen, and Panigrahi [18] on $(P, \mathcal{R}')$, we achieve a strictly worst-case update time of $O(f \log^3 n) = O(\log^d m \cdot \log^3 n)$ alongside bounded worst-case recourse.

Let OPT and OPT' be the optimal set cover for $(P, \mathcal{R})$ and $(P, \mathcal{R}')$, respectively. Using the mapping in the second property, an $O(\log n)$-approximate set cover for $(P, \mathcal{R}')$ can be used to maintain a feasible set cover, say F, for $(P, \mathcal{R})$ of weight $O(w(\mathsf{OPT}') \log n)$. Finally, via the first property, we note that $w(\mathsf{OPT}') = O(w(\mathsf{OPT}) \log^{d-1} m)$. Therefore, F has weight $O(w(\mathsf{OPT}') \log n) = O(w(\mathsf{OPT}) \log^{d-1} m \cdot \log n)$, establishing an approximation factor of $O(\log^{d-1} m \cdot \log n)$ for $(P, \mathcal{R})$.

On the real line, the construction of the new set system is obtained by a combination of interval tree and a *Flatten* procedure (which outputs "chopped" portion of some carefully chosen intervals). In higher dimensions, we will make use of segment trees. A basic introduction to the data structures can be found in Section 3.5, while one can refer to standard textbooks (such as [59]) for a more detailed description.

6.2.2 Construction of the New Set System on the Real Line

We will start by constructing the new set system on the real line. We will denote the original set system $(P, \mathcal{R})$ by $(P, \mathcal{I})$ on the real line (where the notation $\mathcal{I}$ denotes intervals). To do this, we employ the interval flattening procedure that we developed in Chapter 5 for preprocessing unrooted links in the Tree Augmentation Problem. As established in Lemmas 5.2 and 5.3, this procedure allows us to construct a new set system $(P, \mathcal{I}')$ in $O(m \log m)$ time that satisfies

1. (Sparse cover) For each interval $I \in \mathcal{I}$ with weight $w(I)$, there is a collection of intervals in $\mathcal{I}'$ with total weight at most $8 \cdot w(I)$ which completely cover I.

2. (Reverse mapping) For each interval $I' \in \mathcal{I}'$, there is an interval $I \in \mathcal{I}$ such that $I' \subseteq I$ and $w(I') = w(I)$.

3. (Small Frequency) The depth of any point w.r.t. $\mathcal{I}'$ is $O(\log m)$.

6.2.3 Construction of New Set System in Higher Dimensions

In this section we will finally construct the new set system $(P, \mathcal{R}')$ in a d-dimensional space which satisfies the three properties mentioned in Section 6.2.1.

The case of 2-dimensional rectangles. The input is a set of 2-dimensional rectangles $\mathcal{R}$. In the first round, we project the rectangles of $\mathcal{R}$ onto dimension 1, and let $\mathcal{I}^{(1)}$ be the *corresponding* set of intervals. Construct a segment tree $\mathcal{T}^{(1)}$ on $\mathcal{I}^{(1)}$. Let $\mathcal{I}^{(1)}(u) \subseteq \mathcal{I}^{(1)}$ be the intervals *assigned* to a node $u \in \mathcal{T}^{(1)}$, and let $\mathcal{R}(u) \subseteq \mathcal{R}$ be the set of rectangles corresponding to the intervals in $\mathcal{I}^{(1)}(u)$. In the second round, at each node $u \in \mathcal{T}^{(1)}$, we project the rectangles of $\mathcal{R}(u)$ onto dimension 2, and let $\mathcal{I}^{(2)}(u)$ be the resulting set of intervals. Each interval in $\mathcal{I}^{(2)}(u)$ is assigned weight equal to the weight of its corresponding rectangle in $\mathcal{R}(u)$. At each node $u \in \mathcal{T}^{(1)}$, based on the intervals $\mathcal{I}^{(2)}(u)$, build an instance of the structure described in Section 6.2.2 and let $\mathcal{I}'^{(2)}(u)$ be the intervals reported.

For each interval $I \in \mathcal{I}'^{(2)}(u)$ with weight $w(I)$, construct a new rectangle $\text{range}(u) \times I$ with weight $w(I)$, where $\text{range}(u)$ is the range of node u on the first dimension. Let $\mathcal{R}'(u)$ be the collection of these new rectangles constructed at u. The union of $\mathcal{R}'(u)$ over all the nodes $u \in \mathcal{T}^{(1)}$ leads to $\mathcal{R}'$.

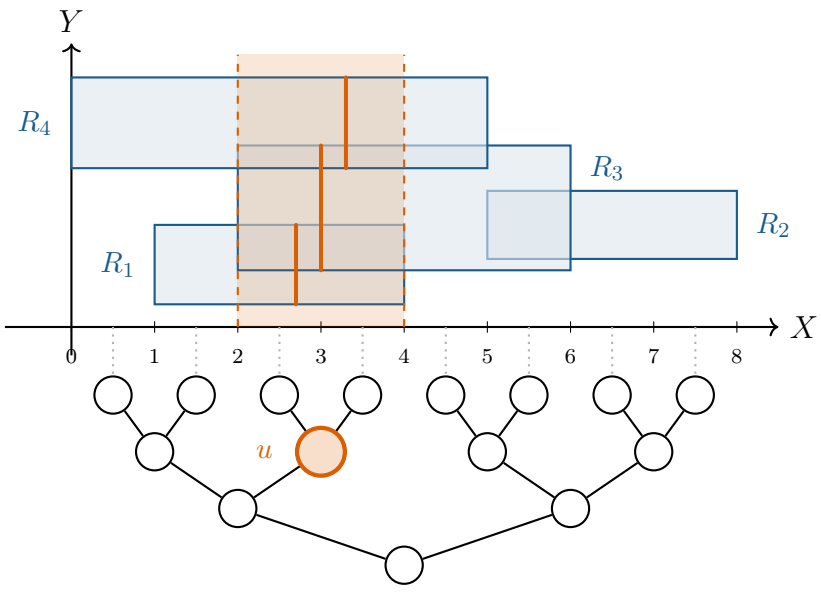


Figure 6.1: Frequency reduction for 2D rectangles.

Look at Figure 6.1 for an example instance. A segment tree $\mathcal{T}^{(1)}$ is constructed over the projections of the rectangles on the first dimension (X-axis). At node u representing X-span $[2, 4]$, we identify the rectangles (being R_1, R_3, and R_4) that completely cover this X-span. Their

vertical Y-projections (orange segments) constitute a 1D sub-problem $\mathcal{I}^{(2)}(u)$. The segment tree ensures each rectangle appears in $O(\log m)$ such sub-problems, and each sub-problem can be handled using the 1D interval flattening procedure from Section 6.2.2.

Lemma 6.1. *The set system $(P, \mathcal{R}')$ satisfies the following properties:*

1. *(Sparse cover) For each rectangle $R \in \mathcal{R}$ with weight $w(R)$, there is a collection of rectangles in $\mathcal{R}'$ with total weight at most $O(w(R) \log m)$ which completely cover R.*
2. *(Reverse mapping) For each rectangle $R' \in \mathcal{R}'$, there is a rectangle $R \in \mathcal{R}$ such that $R' \subseteq R$ and $w(R') = w(R)$.*
3. *(Small Frequency) The depth of any point w.r.t. $\mathcal{R}'$ is $O(\log^2 m)$.*

Proof. Consider any rectangle $R \in \mathcal{R}$. Let $u_1, \ldots, u_t$ be the nodes in $\mathcal{T}^{(1)}$ where R's projection (say $I^{(1)}$) is assigned. By the property of segment trees, we have $\bigcup_{i \in [t]} \text{range}(u_i) = I^{(1)}$ and $t = O(\log m)$. Combining this with Lemma 5.2(1), we get a collection of rectangles in $\bigcup_{i \in [t]} \mathcal{R}'(u_i)$ of total weight $O(w(R) \log m)$ which cover R. This proves Property (1). From the construction and Lemma 5.2(2), Property (2) follows.

For any point p in the plane, consider the path Π from root to the appropriate leaf in $\mathcal{T}^{(1)}$. Among the rectangles in $\mathcal{R}'$, only the rectangles in $\mathcal{R}'(u)$ for all $u \in \Pi$ can contain p by property of segment tree. Since the depth of p in $\mathcal{R}'(u)$ is $O(\log m)$ (by Lemma 5.2(3)), the total depth of p w.r.t. $\mathcal{R}'$ will be $O(|\Pi| \log m) = O(\log^2 m)$. This proves Property (3). □

Extension to d-dimensional hyperrectangles. We now extend the construction to a set of d-dimensional hyperrectangles $\mathcal{R}$. For induction, assume the existence of the construction for $(d-1)$-dimensional hyperrectangles (with base case $d = 2$). Using the same notation as before, construct a segment tree $\mathcal{T}^{(1)}$ on $\mathcal{I}^{(1)}$. At each node $u \in \mathcal{T}^{(1)}$, project the hyperrectangles $\mathcal{R}(u)$ onto the last $(d-1)$ dimensions (label them $\mathcal{R}^{(2\ldots d)}(u)$) and build an instance of the $(d-1)$-dimensional structure on $\mathcal{R}^{(2\ldots d)}(u)$ which reports $\mathcal{R}'^{(2\ldots d)}(u)$. Each hyperrectangle in $\mathcal{R}^{(2\ldots d)}(u)$ is assigned weight equal to the weight of its corresponding hyperrectangle in $\mathcal{R}(u)$.

For each $(d-1)$-dimensional hyperrectangle $R \in \mathcal{R}'^{(2\ldots d)}(u)$ with weight $w(R)$, construct a new d-dimensional hyperrectangle $\text{range}(u) \times R$ with weight $w(R)$, where $\text{range}(u)$ is the range of node u on the first dimension. Let $\mathcal{R}'(u)$ be the collection of these new hyperrectangles constructed at u. The union of $\mathcal{R}'(u)$ over all the nodes $u \in \mathcal{T}^{(1)}$ leads to $\mathcal{R}'$.

Final analysis. We are now ready to establish the three properties of the new set system $(P, \mathcal{R}')$ described in Section 6.2.1. For induction, assume that all the three properties are true for $(d-1)$-dimensions (with base case Lemma 6.1).

Consider any hyperrectangle $R \in \mathcal{R}$. Let $u_1, \ldots, u_t$ be the nodes in $\mathcal{T}^{(1)}$ where R's projection (say $I^{(1)}$) is assigned. By the property of segment trees, we have $\bigcup_{i\in[t]} \text{range}(u_i) = I^{(1)}$ and $t = O(\log m)$. Combining this with the induction hypothesis, leads to a collection of hyperrectangles in $\bigcup_{i\in[t]} \mathcal{R}'(u_i)$ with total weight $O(\log m) \times O(w(R) \log^{d-2} m) = O(w(R) \log^{d-1} m)$ which cover R. This proves the sparse cover property. The reverse mapping property can be seen via the construction and the induction hypothesis.

For any point p in the space, recall the definition of Π. The depth of p w.r.t. $\mathcal{R}'$ will be $O(|\Pi| \log^{d-1} m)$ (via induction hypothesis) which is equal to $O(\log^d m)$. This proves the small frequency property.

Lemma 6.2. *The time taken to construct the set $\mathcal{R}'$ is $O(m \log^d m)$.*

Proof. We will perform an induction on the dimension size d. The base case is the real line where constructing $\mathcal{I}'$ takes $O(m \log m)$ time (Lemma 5.3). Assume that the statement is true for $(d-1)$-dimensions. Note that $\sum_{u\in\mathcal{T}^{(1)}} |\mathcal{R}(u)| = O(m \log m)$, since each hyperrectangle in $\mathcal{R}$ is assigned to $O(\log m)$ nodes in the segment tree. Therefore, the total time spent at all the nodes in $\mathcal{T}^{(1)}$ is $\sum_{u\in\mathcal{T}^{(1)}} O(|\mathcal{R}(u)| \log^{d-1} |\mathcal{R}(u)|) = O(m \log^d m)$. □

This finishes the proof of Theorem 6.1.

6.3 Weighted Dynamic Hitting Set on Hyperrectangles

In this section we present a dynamic algorithm for weighted Hitting Set on d-dimensional hyperrectangles. Recall that in this setting in each round either a hyperrectangle is inserted or deleted, and the goal is to maintain a hitting set of the hyperrectangles with weighted points.

We will reduce the problem to an instance of dynamic Set Cover in $2d$-dimensional space and use the algorithm designed in the previous section (Theorem 6.1). The input is a set P of n points and a set $\mathcal{S}$ of m hyperrectangles. Assume that all the points of P and the hyperrectangles in $\mathcal{S}$ lie in the box $[0, N]^d$. For all $p \in P$, we transform $p = (p_1, \ldots, p_d)$ to a $2d$-dimensional hypercube $\mathcal{S}(p)$ of side length N and "lower-left" corner $p' = (-p_1, \ldots, -p_d, p_1, \ldots, p_d)$. Let $\mathcal{S}(P)$ be the transformed hypercubes. Next, we transform each hyperrectangle, say $S \in \mathcal{S}$, with "lower-left" corner $a = (a_1, \ldots, a_d)$ and "top-right" corner $b = (b_1, \ldots, b_d)$ into a $2d$-dimensional point $P(S) = (-a_1, \ldots, -a_d, b_1, \ldots, b_d)$. Let $P(\mathcal{S})$ be the transformed points. See Figure 6.2.

Lemma 6.3. *In $\mathbb{R}^d$ a point p lies inside a hyperrectangle S, if and only if in $\mathbb{R}^{2d}$ the hypercube $\mathcal{S}(p)$ contains the point $P(S)$.*

Proof. We define a point $q = (q_1, \ldots, q_{2d})$ to *dominate* another point $q' = (q'_1, \ldots, q'_{2d})$ if and only if $q_i \geq q'_i$, for all $1 \leq i \leq 2d$. Assume that p lies inside S. Then we have $p_i \geq a_i$ and

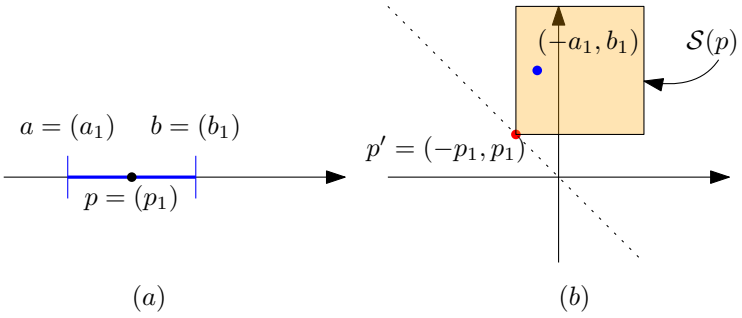


Figure 6.2: (a) A point p in 1-D lying inside an interval $S = [a_1, b_1]$, and (b) the transformation of p into a square $\mathcal{S}(p)$, and transformation of S into a point $(-a_1, b_1)$ in 2D.

$p_i \leq b_i$, for all $1 \leq i \leq d$. This implies that $P(S) = (-a_1, \ldots, -a_d, b_1, \ldots, b_d)$ dominates the point $p' = (-p_1, \ldots, -p_d, p_1, \ldots, p_d)$, which is the lower-left corner of $\mathcal{S}(p)$.

The coordinates of the top-right corner of $\mathcal{S}(p)$ is $(-p_1+N, \ldots, -p_d+N, p_1+N, \ldots, p_d+N)$. For all $1 \leq i \leq d$, since $N \geq p_i - a_i$, it implies that $-p_i + N \geq -a_i$. For all $1 \leq i \leq d$, since $p_i + N \geq N \geq b_i$, it implies that $p_i + N \geq b_i$. This finally implies that the top-right corner of $\mathcal{S}(p)$ dominates $P(S)$. Therefore, $\mathcal{S}(p)$ contains $P(S)$.

Assume that p lies outside S. Then there exists a dimension i such that $p_i < a_i$ or $p_i > b_i$. If $p_i < a_i$, then $-p_i > -a_i$, which implies that $P(S)$ cannot dominate the lower-left corner of $\mathcal{S}(p)$. If $p_i > b_i$, again $P(S)$ cannot dominate the lower-left corner of $\mathcal{S}(p)$. This implies that $P(S)$ lies outside $\mathcal{S}(p)$. □

The final algorithm is the following. We will use the reduction in Lemma 6.3 to transform the points in P into hypercubes in $\mathbb{R}^{2d}$ and transform the hyperrectangles in $\mathcal{S}$ into points in $\mathbb{R}^{2d}$. Therefore, the Hitting Set problem in $\mathbb{R}^d$ on hyperrectangles has been reduced to the Set Cover problem in $\mathbb{R}^{2d}$ with hypercubes. Then we will run the Set Cover algorithm in Theorem 6.1 on the transformed set system. The performance of the algorithm is summarized below.

Theorem 6.2. *There is an algorithm for dynamic weighted Hitting Set on d-dimensional hyperrectangles with an approximation factor of $O(\log^{2d-1} n \cdot \log m)$, amortized update time of $O(\log^{2d} n \cdot \log m)$ (or worst-case update time of $O(\log^{2d} n \cdot \log^3 m)$), and a preprocessing time of $O(n \log^{2d} n)$.*

Chapter 7

Stabbing Rectangles and Rectilinear Polygons

7.1 Introduction

In the previous chapters, we explored covering problems in online and dynamic settings. In this chapter, we shift our focus to the static setting of *geometric stabbing problems*. These are structured instances of weighted Set Cover where the universe consists of geometric objects (such as rectangles or polygons) and the resources are axis-parallel line segments. A segment is said to *cover* an object if it *stabs* it, i.e., the segment fully traverses the shape, intersecting its opposite boundaries.

Geometric stabbing models fundamental resource allocation problems. For instance, in a simple bandwidth allocation scenario, a request might be represented by a single axis-aligned rectangle, corresponding to a specific time interval and a contiguous frequency band. To minimize the total resource usage, we seek the minimum total length of allocation "channels" (segments) required to satisfy all requests. However, modern applications often require more flexibility. In heterogeneous computing architectures (such as ARM's big.LITTLE [77] or Intel's hybrid architecture [119]), a task might be serviceable by different types of cores: a high-performance core might process the task quickly (short time, high resource type), while an efficiency core might take longer (long time, low resource type). Modeling such a request requires a geometric shape formed by the union of multiple rectangles (known as a *rectilinear polygon*) representing the trade-off between resource type and duration.

7.1.1 Prior Work

Stabbing problems are special cases of the weighted Set Cover problem. While general Set Cover is APX-hard and cannot be approximated better than $(1 - o(1)) \ln n$ (unless P = NP), the specific geometry of axis-aligned shapes often allows for better results.

The baseline case is the horizontal Rectangle Stabbing problem, where the input consists of axis-aligned rectangles, and we have to stab every shape with a set of horizontal segments. Chan, van Dijk, Fleszar, Spoerhase, and Wolff [40] initiated the study of this problem, providing a constant-factor approximation. Recently, Eisenbrand, Gallato, Svensson, and Venzin [63] presented a Quasi-Polynomial-Time Approximation Scheme (QPTAS), suggesting that the problem is not APX-hard. However, the existence of a Polynomial-Time Approximation Scheme (PTAS) remained an open question.

Moving beyond rectangles, we consider *k-shapes*, for which the landscape is more nuanced. In our joint work, we first establish that for specific restricted classes of k-shapes (specifically those satisfying an "hourglass" condition) a Quasi-Polynomial-Time Approximation Scheme (QPTAS) exists [96]. While these results suggest that k-Stabbing might not be APX-hard for certain polygons, it leaves some questions unanswered: what shapes allow for a PTAS, and which structural violations trigger APX-hardness?

7.1.2 Our Results and Techniques

In this chapter, we resolve the open question regarding rectangles and establish a boundary of tractability for general rectilinear polygons.

1. **PTAS for Rectangles:** We present a PTAS for Rectangle Stabbing, improving upon the previous QPTAS results. Our algorithm relies on a hierarchical grid decomposition combined with dynamic programming. We also extend the technique to obtain better bounds for adjacent problems like Square-Stabbing (shapes to be stabbed are squares), HV-Stabbing (both horizontal and vertical segments allowed), and other variants.

2. **PTAS for Bounded Width k-shapes:** We generalize our technique to show that if the input consists of k-shapes where the widths of constituent rectangles differ by at most a constant factor, the problem still admits a PTAS.

3. **Hardness for General Polygons:** We show that this geometric tractability is fragile. We prove that for general k-shapes ($k \geq 3$) without width constraints, the problem becomes APX-hard. Furthermore, for arbitrary rectilinear polygons, we show the problem is as hard as general Set Cover, reintroducing the $\Omega(\log n)$ lower bound.

Our main algorithmic contribution is a dynamic program (DP) that recursively subdivides the plane into smaller rectangular regions using a *hierarchical* grid decomposition. Unlike the earlier QPTAS [63] which used equally spaced grid lines, our hierarchical approach allows us to process strips of varying widths. In each level of our decomposition, we guess $O_\varepsilon(1)$ line segments from the optimal solution. A crucial technical ingredient is the alignment of guessed segments: we slightly extend segments so they align with our grid. This alignment ensures that local decisions can be "forgotten" after advancing a constant number of levels in the recursion. This bounded memory property is what reduces the complexity from quasipolynomial to polynomial.

Organization. Section 7.2 defines the notation, the specific geometric shapes, and the discretization techniques used throughout the chapter. Section 7.3 details the PTAS for the fundamental case of rectangles, and Section 7.4 extends the results to some special cases like that of δ-large rectangles, and Generalized Minimum Manhattan Network. Section 7.5 extends the result to k-shapes with bounded width variance. Finally, Section 7.6 presents the hardness results for general rectilinear polygons.

7.2 Preliminaries

We consider the two-dimensional Euclidean plane. As defined in Section 3.4.3, our baseline geometric object is an axis-aligned rectangle $R = [x_1, x_2] \times [y_1, y_2]$. We assume that all input coordinates are non-negative.

For our algorithmic design and analysis, we establish the following specific notation. We denote the width of a rectangle R by $w(R) = |x_2 - x_1|$, its height by $h(R) = |y_2 - y_1|$, its bottom edge by $b(R)$, and its top edge by $t(R)$. Building upon standard rectangles, we define a hierarchy of more complex rectilinear polygons.

Definition 7.1 (k-shape)**.** *Let $k \in \mathbb{N}$. A k-*shape *K is a rectilinear polygon formed by the union of a sequence of at most k axis-aligned rectangles $(R_1, R_2, \ldots, R_m)$ where $m \leq k$, satisfying the vertical connectivity property:*

$$t(R_i) \subseteq b(R_{i+1}) \quad \text{or} \quad t(R_i) \supseteq b(R_{i+1})$$

for all $1 \leq i < m$. Note that a 1-shape is simply an axis-aligned rectangle.

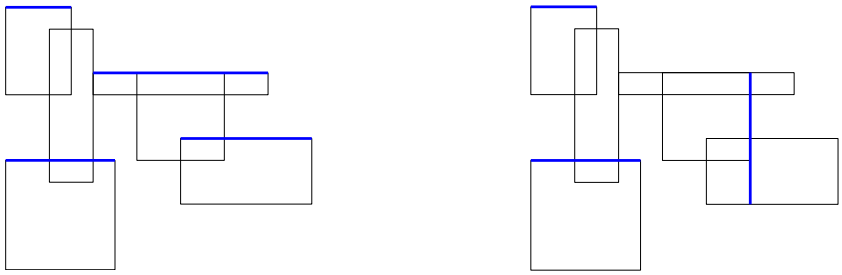
For a k-shape K, we denote the minimum and maximum widths of its constituent rectangles by $w_{\min}(K) = \min\{w(R_i) : R_i \in K\}$ and $w_{\max}(K) = \max\{w(R_i) : R_i \in K\}$.

Recall from Section 3.4.3 that a horizontal line segment stabs a rectangle if it cuts completely across its width. We extend this definition to k-shapes: a segment stabs a k-shape K if it stabs

at least one of its constituent rectangles $R_i \subseteq K$.

Problem Definitions. Building on the fundamental Rectangle Stabbing problem (Section 3.4.3), we study the following optimization variants:

- **Horizontal-Vertical Stabbing (HV-Stabbing):** Given a set $\mathcal{B}$ of rectangles, find a minimum length set of axis-parallel segments (horizontal or vertical) to stab all $R \in \mathcal{B}$. (A vertical segment stabs a rectangle if it completely traverses its height).
- **k-Stabbing:** Given a set $\mathcal{K}$ of k-shapes, find a minimum length set of horizontal segments to stab all $K \in \mathcal{K}$.



(a) Horizontal Rectangle Stabbing. (b) Horizontal-Vertical Stabbing.

Figure 7.1: A solution to an instance of Rectangle Stabbing and HV-Stabbing.

For any set of segments $\mathcal{S}$, we denote its cost (total length) by $c(\mathcal{S}) = \sum_{\ell \in \mathcal{S}} |\ell|$, where $|\ell|$ is the length of an individual segment ℓ. For any problem instance, we denote the optimal solution set by OPT and its cost by $c(\mathsf{OPT})$. We use SOL to denote the solution returned by our algorithms.

7.2.1 Structural Properties

A specific subclass of k-shapes plays a crucial role in understanding the boundary of approximability.

Definition 7.2 (Hourglass Condition)**.** *A k-shape K formed by the sequence of rectangles $(R_1, \ldots, R_m)$ satisfies the* hourglass condition *if the sequence of widths $(w(R_1), \ldots, w(R_m))$ contains a local minimum (that is also the global minimum), and the widths are non-decreasing as one moves away from this minimum. Formally, there exists an index $p \in \{1, \ldots, m\}$ such that:*

$$w(R_1) \geq w(R_2) \geq \cdots \geq w(R_p) \leq w(R_{p+1}) \leq \cdots \leq w(R_m).$$

Intuitively, this implies that the shape does not "bulge" in the middle. Note that any 2-shape (such as an L-shape) trivially satisfies this condition. Figure 7.2 illustrates shapes satisfying this property, while Figure 7.3 (left) shows a shape violating it.

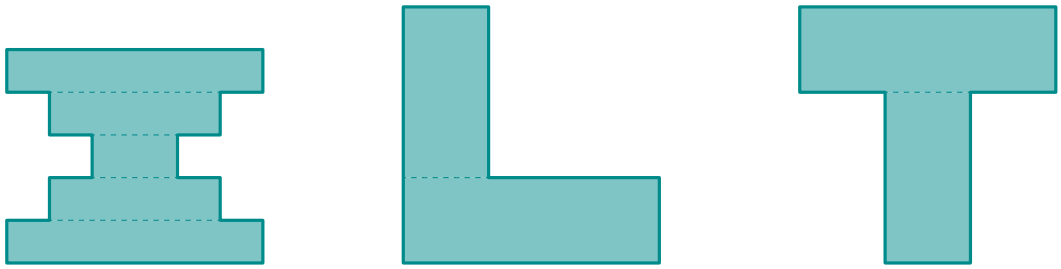


Figure 7.2: Examples of k-shapes satisfying the hourglass condition.



Figure 7.3: A 3-shape not satisfying the hourglass condition (left), and a stack of rectangles that does not form a k-shape (right).

We also consider shapes with *Bounded Width Variance*. A set of k-shapes has width variance $\delta \in (0, 1]$ if for every k-shape K and any two constituent rectangles $R_i, R_j \subseteq K$, it holds that $\delta \cdot w(R_j) \leq w(R_i) \leq w(R_j)/\delta$.

7.2.2 Discretization

Our algorithms rely on a discrete coordinate system to limit the search space for dynamic programming. We show that any instance of Rectangle Stabbing or k-Stabbing (under bounded width constraints) can be transformed into a well-behaved discrete instance with negligible loss in the approximation factor.

Let $\varepsilon \in (0, 1/3)$ be an error parameter. We assume $1/\varepsilon \in \mathbb{N}$. We define a coordinate $x \in \mathbb{R}$ as *discrete* if it is an integral multiple of ε^d, where $d \in \mathbb{N}$ is a sufficiently large integer such that $\varepsilon^3/n < \varepsilon^d \leq \varepsilon^2/n$. Similarly, a y-coordinate is discrete if it is integral. A geometric object is discrete if its defining points have discrete coordinates.

Let $\alpha \geq 1$ be a scaling factor. For Rectangle Stabbing, we set $\alpha = 1$. For k-Stabbing with width variance δ, we set $\alpha = k/\delta$.

Lemma 7.1 (General Discretization)**.** *Given an instance $\mathcal{I}$ of* Rectangle Stabbing *or* k-Stabbing *with width variance δ, we can compute in polynomial time a transformed instance $\mathcal{I}'$ satisfying the following properties:*

(i) All k-shapes (or rectangles) in $\mathcal{I}'$ have coordinates that are discrete.

(ii) All k-shapes (or rectangles) in $\mathcal{I}'$ lie within a bounding box $[0, 2\alpha n] \times [0, (k+1)n]$.

(iii) For every shape $K \in \mathcal{I}'$, the width satisfies $\alpha\varepsilon/n < w_{\min}(K) < w_{\max}(K) < \alpha$.

(iv) The instance $\mathcal{I}'$ admits a solution consisting only of discrete segments of length at most α/ε, with cost at most $(1 + O(\varepsilon)) \cdot \mathsf{OPT}(\mathcal{I})$.

Furthermore, a solution to $\mathcal{I}'$ can be transformed back to a valid solution for $\mathcal{I}$ with the same cost bound.

Proof. We first compute a constant-factor approximation $\mathsf{SOL}_{\text{apx}}$ for the input instance $\mathcal{I}$. For rectangles ($\alpha = 1$), a simple greedy algorithm or the algorithm of Chan, van Dijk, Fleszar, Spoerhase, and Wolff [40] provides this. For k-shapes with bounded width variance, we approximate each k-shape by its bounding box and solve the resulting rectangle instance; this yields an $O(\alpha)$-approximation (see Lemma 7.2).

Let $C = c(\mathsf{SOL}_{\text{apx}})$. We know $c(\mathsf{OPT}) \in [C/\alpha, C]$. We scale the entire instance uniformly by a factor of C/α. Consequently, the scaled optimal cost satisfies $c(\mathsf{OPT}') \in [1, \alpha]$.

Property (iii) - Width Bounds: We identify all shapes with width $w_{\min}(K) \leq \alpha\varepsilon/n$. We can stab all such shapes greedily using segments of total length at most $\alpha\varepsilon$ (one segment per shape). Since $c(\mathsf{OPT}') \geq 1$, the added cost is at most $\alpha\varepsilon \leq \alpha\varepsilon \cdot c(\mathsf{OPT}')$, which is a factor of $(1 + \alpha\varepsilon)$. We remove these shapes from the instance to ensure $\alpha\varepsilon/n < w_{\min}(K)$.

Now, since the total length of segments in OPT is at most α, there can be no segments of length longer than α in the solution. This means that rectangular segments of k-shapes which are wider than α can be discarded (since OPT cannot stab the k-shape along this segment) to ensure $w_{\max}(K) < \alpha$.

Property (i) - Discrete Coordinates: We first discretize the x-coordinates. Let $\Delta = \varepsilon^d$ be the grid granularity. We round every x-coordinate in the instance to the nearest multiple of Δ by extending each constituent rectangle $R = [x_1, x_2] \times [y_1, y_2]$ to $R' = [\lfloor x_1 \rfloor_\Delta, \lceil x_2 \rceil_\Delta] \times [y_1, y_2]$. This extension increases the width of any rectangular section by at most $2\Delta = 2\varepsilon^d < 2\varepsilon/n$, and hence the cost of the solution goes up by a factor of at most $1 + 2\varepsilon$.

Next, we discretize the y-coordinates. Since there are at most n shapes, and each k-shape is composed of at most k rectangles, there are at most $(k+1)n$ distinct y-coordinates. We can

map these y-coordinates to the set of integers $\{0, 1, \ldots, (k+1)n\}$ while preserving their relative order. This "stretching" of vertical space does not increase the optimal cost because horizontal segments have zero height.

Property (ii) - Bounding Box: Since $c(\mathsf{OPT}') \leq \alpha$, no segment in the optimal solution exceeds length α. Consequently, shapes separated by a horizontal gap greater than α belong to independent sub-instances. For any such instance, the total width is at most $n \cdot w_{\max} + (n-1)\alpha < 2\alpha n$. By translating the x-coordinates, we ensure the instance fits within the horizontal range $[0, 2\alpha n]$. Combined with the y-coordinate mapping from Property (i), the instance lies within the bounding box $[0, 2\alpha n] \times [0, (k+1)n]$.

Property (iv) - Segment Structure: Consider any segment ℓ in the optimal solution. If $|\ell| > \alpha/\varepsilon$, we split it into smaller pieces of length $\alpha/\varepsilon - 2\alpha$. Now extend each smaller segment to ensure it stabs any shape it intersects (before the extension). Since the maximum width of a k-shape is α, the cost increase is bounded by 2α. So, we increase the length of ℓ by at most a factor of

$$\begin{aligned}
\left(|\ell| + \left\lceil \frac{|\ell|}{\alpha/\varepsilon - 2\alpha} \right\rceil \cdot 2\alpha\right) \cdot \frac{1}{|\ell|} &\leq 1 + \frac{2\alpha}{|\ell|} \cdot \left\lceil \frac{\varepsilon|\ell|}{\alpha - 2\alpha\varepsilon} \right\rceil \\
&\leq 1 + \frac{2\alpha}{|\ell|} \cdot \left(1 + \frac{\varepsilon|\ell|}{\alpha - 2\alpha\varepsilon}\right) \\
&\leq 1 + 2\varepsilon + 2\varepsilon \cdot \frac{\alpha}{\alpha - 2\alpha\varepsilon} && \text{(since } |\ell| > \alpha/\varepsilon\text{)} \\
&\leq 1 + 2\varepsilon + 2\varepsilon \cdot \frac{1}{1 - 2/3} && \text{(since } \varepsilon < 1/3\text{)} \\
&\leq 1 + 8\varepsilon.
\end{aligned}$$

□

We now look at the constant-factor approximation for k-shapes we alluded to in the earlier proof.

Lemma 7.2 (Constant-Factor Approximation)**.** *If the widths of rectangles within any k-shape differ by at most a factor of $1/\delta$, the problem admits an $O(k/\delta)$-approximation.*

Proof. Given an instance $\mathcal{K}$ of k-Stabbing, let OPT denote its optimal solution. We create an instance of Rectangle Stabbing as follows: for each $K \in \mathcal{K}$, we create a rectangle R_K that is the smallest axis-parallel rectangle containing K (the bounding box). Let OPT' be the optimal solution for the constructed rectangle instance.

We claim that $\mathsf{OPT}' \leq O(k \cdot \mathsf{OPT}/\delta)$. To see this, consider any segment in OPT. If we extend this segment by a factor of roughly k/δ in each direction, it becomes wide enough to stab the bounding box of any k-shape it originally stabbed. This is because, even if the k-shape does not satisfy the hourglass property, the bounded width variance implies the total width of the shape is at most $O(k/\delta)$ times the width of its narrowest part (see Figure 7.4).

Conversely, any solution to the rectangle instance is a valid solution to the k-shape instance (stabbing the bounding box implies stabbing the shape). By applying a known constant-factor approximation for Rectangle Stabbing [40] to the bounding boxes, we obtain a solution that approximates OPT' and thus yields an $O(k/\delta)$-approximation for the original problem. □

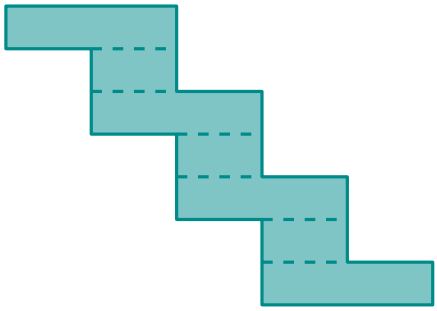


Figure 7.4: For $\delta = 1/2$, a k-shape that spans a width greater than $k/2 = O(k)$ times the narrowest part.

Throughout the remainder of this chapter, we assume the input instance satisfies the properties of Lemma 7.1.

7.3 PTAS for Horizontal Rectangle Stabbing

We present a dynamic program that computes a $(1+\varepsilon)$-approximation to HV-Stabbing, for any given constant $\varepsilon > 0$, for the case when for each input rectangle $R_i \in \mathcal{B}$ its width is at most its height, i.e., $w(R_i) \leq h(R_i)$. This directly implies a PTAS for the setting of squares for the same problem, and we will argue that it also yields a PTAS for Rectangle Stabbing. Also, we will use it later as a subroutine to obtain a $(2+\varepsilon)$-approximation for HV-Stabbing and a PTAS for the setting of δ-large rectangles of HV-Stabbing.

7.3.1 Dynamic Program

Our algorithm is based on a dynamic program. It has a cell $\mathrm{DP}(S, \mathcal{L})$ for each combination of

- a rectangular region $S \subseteq [0, 2n] \times [0, 2n]$ with discrete coordinates (that is not necessarily equal to an input rectangle in $\mathcal{B}$).

- a set $\mathcal{L}$ of at most $3(1/\varepsilon)^3$ axis-aligned line segments, such that for each $\ell \in \mathcal{L}$ we have that $\ell \subseteq S$, and ℓ is discrete.

This DP cell corresponds to the subproblem of stabbing all rectangles in $\mathcal{B}$ that are contained in S and that are not already stabbed by the line segments in $\mathcal{L}$. Therefore, the DP stores a near-optimal solution $\mathsf{SOL}(S, \mathcal{L})$ in the cell $\mathrm{DP}(S, \mathcal{L})$ such that $\mathsf{SOL}(S, \mathcal{L}) \cup \mathcal{L}$ stabs all rectangles in $\mathcal{B}$ that are contained in S.

Given a DP cell $\mathrm{DP}(S, \mathcal{L})$, our dynamic program computes a solution for it as follows. If $\mathcal{L}$ already stabs each rectangle from $\mathcal{B}$ that is contained in S, then we simply define the solution $\mathsf{SOL}(S, \mathcal{L}) := \emptyset$ for the cell $\mathrm{DP}(S, \mathcal{L})$ and do not compute anything further. Another simple case is when there is a line segment $\ell \in \mathcal{L}$ such that $S \setminus \ell$ is divided into two rectangular regions S_1, S_2 (these regions are henceforth referred to as connected components; note that these components need not be closed rectangles). In this case, we define $\mathsf{SOL}(S, \mathcal{L}) := \mathsf{SOL}(S_1, \mathcal{L} \cap S_1) \cup \mathsf{SOL}(S_2, \mathcal{L} \cap S_2) \cup \{\ell\}$, where for any set of line segments $\mathcal{L}'$ and any rectangle S' we define $\mathcal{L}' \cap S' := \{\ell' \cap S' | (\ell' \in \mathcal{L}') \wedge (\ell' \cap S' \neq \emptyset)\}$. In case there is more than one such line segment $\ell \in \mathcal{L}$ then we pick one according to some arbitrary but fixed global tie-breaking rule. We will later refer to this as a *trivial operation*.

Otherwise, we do each of the following operations which produce a set of candidate solutions:

1. *Add operation:* Consider each set $\mathcal{L}'$ of line segments with discrete coordinates such that $|\mathcal{L} \cup \mathcal{L}'| \leq 3\varepsilon^{-3}$ and each $\ell \in \mathcal{L}'$ is axis-aligned and contained in S. For each such set $\mathcal{L}'$ we define the solution $\mathcal{L}' \cup \mathsf{SOL}(S, \mathcal{L} \cup \mathcal{L}')$ as a candidate solution.

2. *Line operation:* Consider each axis-aligned line ℓ passing through some discrete point such that $S \setminus \ell$ has two connected components S_1 and S_2. Let $\mathcal{B}_\ell$ denote the rectangles from $\mathcal{B}$ that are contained in S and that are stabbed by ℓ. For the line ℓ we do the following:

   (a) Compute an $O(1)$-approximate solution $\mathcal{L}(\mathcal{B}_\ell)$ for the rectangles in $\mathcal{B}_\ell$ using a polynomial-time algorithm [40].

   (b) Produce the candidate solution $\mathcal{L}(\mathcal{B}_\ell) \cup \mathsf{SOL}(S_1, \mathcal{L} \cap S_1) \cup \mathsf{SOL}(S_2, \mathcal{L} \cap S_2)$.

Note that in the line operation we consider entire lines, not just line segments. We define $\mathsf{SOL}(S, \mathcal{L})$ to be the solution of minimum cost among all the candidate solutions produced above and store it in $\mathrm{DP}(S, \mathcal{L})$.

We do the operation above for each DP cell $\mathrm{DP}(S, \mathcal{L})$. Finally, we output the solution $\mathsf{SOL}([0, 2n] \times [0, 2n], \emptyset)$, i.e., the solution corresponding to the cell $\mathrm{DP}([0, 2n] \times [0, 2n], \emptyset)$.

We remark that instead of using the existing $O(1)$-approximation algorithm [40] for stabbing the rectangles in $\mathcal{B}_\ell$, one could design an algorithm with a better approximation guarantee, using the fact that all rectangles in $\mathcal{B}_\ell$ are stabbed by the line ℓ. However, for our purposes an $O(1)$-approximate solution is good enough.

7.3.2 DP-decision Trees

We want to show that the DP above computes a $(1+\varepsilon)$-approximate solution. For this, we define a tree T in which each node corresponds to a cell $\mathrm{DP}(S, \mathcal{L})$ of the DP and a corresponding solution $\overline{\mathsf{SOL}}(S, \mathcal{L})$ to this cell. The root node of T corresponds to the cell $\mathrm{DP}([0, 2n] \times [0, 2n], \emptyset)$. Intuitively, the outgoing edges (when considered in root to leaf direction) from a node represent application of some DP operation (as described in the section above) to the parent node, and the child nodes are the subproblems obtained due to application of said operation. The corresponding solution stored at a node is the solution obtained by choosing certain operations, corresponding to the edges in the subtree at that node. This structure now allows us to represent a valid sequence of operations by the algorithm as a tree. Since the DP always picks the solution of minimum total cost this implies that the computed solution has a cost that is at most the cost of the root, $c(\overline{\mathsf{SOL}}([0, 2n] \times [0, 2n], \emptyset))$ of any such tree. Formally, we require T to satisfy the following properties:

1. We require that a node v is a leaf if and only if for the corresponding DP cell $\mathrm{DP}(S, \mathcal{L})$ the DP directly defined that $\mathsf{SOL}(S, \mathcal{L}) = \emptyset$. This is because all rectangles in $\mathcal{B}$ that are contained in S are already stabbed by the segments in $\mathcal{L}$.

2. If a node v for a DP cell $\mathrm{DP}(S, \mathcal{L})$ has exactly one child, then we require that we reduce the problem for $\mathrm{DP}(S, \mathcal{L})$ to the child by applying the add operation, that is, there is a set $\mathcal{L}'$ of axis-aligned line segments with discrete coordinates such that $|\mathcal{L} \cup \mathcal{L}'| \leq 3(1/\varepsilon)^3$, the child node of v corresponds to the cell $\mathrm{DP}(S, \mathcal{L} \cup \mathcal{L}')$, and $\overline{\mathsf{SOL}}(S, \mathcal{L}) = \overline{\mathsf{SOL}}(S, \mathcal{L} \cup \mathcal{L}') \cup \mathcal{L}'$.

3. Similarly, if a node v has two children then we require that we can reduce the problem of $\mathrm{DP}(S, \mathcal{L})$ to these two children by applying the trivial operation or the line operation. Formally, assume that the child nodes correspond to the subproblems $\mathrm{DP}(S_1, \mathcal{L}_1)$ and $\mathrm{DP}(S_2, \mathcal{L}_2)$.

   - If there is a segment $\ell \in \mathcal{L}$ such that $S_1 \cup S_2 \cup \ell = S$, then the applied operation was a trivial operation, and it must also be true that $\mathcal{L}_1 \cup \mathcal{L}_2 \cup \{\ell\} = \mathcal{L}$ and $\overline{\mathsf{SOL}}(S, \mathcal{L}) = \overline{\mathsf{SOL}}(S_1, S_1 \cap \mathcal{L}) \cup \overline{\mathsf{SOL}}(S_2, S_2 \cap \mathcal{L}) \cup \{\ell\}$.

- If no such segment exists, then the applied operation was a line operation along the line ℓ, such that $S_1 \cup S_2 \cup \ell = S$, $\mathcal{L}_1 \cup \mathcal{L}_2 = \mathcal{L}$, and $\overline{\mathsf{SOL}}(S, \mathcal{L}) = \overline{\mathsf{SOL}}(S_1, S_1 \cap \mathcal{L}) \cup \overline{\mathsf{SOL}}(S_2, S_2 \cap \mathcal{L}) \cup \mathcal{L}(\mathcal{B}_\ell)$; where $\mathcal{L}(\mathcal{B}_\ell)$ is an $O(1)$-approximate solution to the set of rectangles intersected by ℓ.

We call a tree T with these properties a *DP-decision tree*.

Lemma 7.3. *If there is a DP-decision tree T' for which $c(\overline{\mathsf{SOL}}([0,2n]\times[0,2n],\emptyset)) \leq (1+\varepsilon)\mathsf{OPT}$, then the DP is a $(1+\varepsilon)$-approximation algorithm with a running time of $(n/\varepsilon)^{O(1/\varepsilon^3)}$.*

Proof. We know that the DP always picks a solution that corresponds to the DP-decision tree with the minimum cost. Since T' is a valid DP-decision tree, the solution picked by the DP is of cost at most $c(\overline{\mathsf{SOL}}([0,2n] \times [0,2n], \emptyset))$.

Now let us consider the running time of the algorithm. Since a DP problem is defined on a discrete rectangular cell, there are at most

$$\frac{2n}{\varepsilon^d} \times \frac{2n}{\varepsilon^d} = \frac{4n^2}{\varepsilon^{2d}} \leq \frac{4n^4}{\varepsilon^6} \qquad \text{since } \varepsilon^3/n < \varepsilon^d \leq \varepsilon^2/n$$

possibilities for a corner vertex of a rectangle, and hence $\binom{4n^4/\varepsilon^6}{2} = O(n^8/\varepsilon^{12})$ possible rectangles.

Similarly, the subproblem definition also includes a set of segments $\mathcal{L}$ of size at most $3\varepsilon^{-3}$. Since the segments are discrete, we can count the number of vertical segments by picking two points (corresponding to the starting and ending points) from the available discrete points on a vertical line, and then fixing its x-coordinate. We repeat a similar process for horizontal lines giving us,

$$\binom{2n/\varepsilon^d}{2} \times \frac{2n}{\varepsilon^d} + \binom{2n/\varepsilon^d}{2} \times \frac{2n}{\varepsilon^d} \leq \binom{2n^2/\varepsilon^3}{2} \times \frac{4n^2}{\varepsilon^3}$$

that is, $O(n^6/\varepsilon^9)$ possible segments[1]. So there are at most $(n/\varepsilon)^{O(1/\varepsilon^3)}$ subsets of segments $\mathcal{L}$ of size at most $3\varepsilon^{-3}$, and at most $(n/\varepsilon)^{O(1/\varepsilon^3)}$ valid DP cells.

For each DP cell, we have to consider all possible candidate solutions and select the minimum. There are at most $2n^2/\varepsilon^3 + 2n^2/\varepsilon^3 = O(n^2/\varepsilon^3)$ possible line operations and $(n/\varepsilon)^{O(1/\varepsilon^3)}$ possible add operations. Note that if the DP performs a trivial operation, then there is no choice to make here, but the trivial operation is selected automatically.

Hence, the total number of possible operations for a given DP cell is $(n/\varepsilon)^{O(1/\varepsilon^3)}$. For each line operation we call the $O(1)$-approximation algorithm which also runs in polynomial time [40].

[1]Note that we allow all possible discrete segments. Hence, the simple bound of $O(n^3)$ on the number of segments used in [63] is not valid.

Since we have $(n/\varepsilon)^{O(1/\varepsilon^3)}$ DP cells, our overall running time is bounded by $(n/\varepsilon)^{O(1/\varepsilon^3)}$. □

We now define a DP-decision tree for which $c(\overline{\mathsf{SOL}}(S,\mathcal{L})) \leq (1+\varepsilon)\mathsf{OPT}$. Recall that $1/\varepsilon \in \mathbb{N}$. We start by defining a hierarchical grid of vertical lines. Let $a \in [0,n]$ be a discrete random offset to be defined later. The grid lines have levels. For each level $j \in \mathbb{N}_0$, there is a grid line $\{a+k\cdot\varepsilon^{j-2}\}\times\mathbb{R}$ for each $k \in \mathbb{N}$. Note that for each $j \in \mathbb{N}_0$ each grid line of level j is also a grid line of level $j+1$, and that for all $j \leq d+2$, the grid lines of level j have discrete x-coordinates. We also note that any two consecutive lines of some level j are exactly ε^{j-2} units apart.

We say that a line segment $\ell \in \mathsf{OPT}$ is *of level* j if the length of ℓ is in $(\varepsilon^j, \varepsilon^{j-1}]$ (Note that we can have vertical segments which are longer than $1/\varepsilon$, we consider these also to be of level 0). We say that a *horizontal* line segment of some level j is *well-aligned* if both its left and its right x-coordinates lie on a grid line of level $j+3$, i.e., if both of its x-coordinates are of the form $a + k\cdot\varepsilon^{j+1}$, for some $k \in \mathbb{Z}$. We say that a *vertical* line segment of some level j is *well-aligned* if both its top and bottom y-coordinates are integral multiples of ε^{j+1}. This would be similar to the segment's endpoints lying on an (imaginary) horizontal grid line of level $j+3$. Recall that by Lemma 7.1, each horizontal segment $\ell \in \mathsf{OPT}$ satisfies that $\varepsilon/n < |\ell| \leq \varepsilon^{-1}$. By our choice of d we have $\varepsilon^{d-1} \leq \varepsilon/n < \varepsilon^{d-2}$ which implies $\varepsilon^{d-1} < |\ell| \leq \varepsilon^{-1}$. Since a segment is of level j if its length is in the range $(\varepsilon^j, \varepsilon^{j-1}]$, we can conclude that all segments in OPT belong to levels in the range $\{0,\ldots,d-1\}$. From this we can infer that any well-aligned horizontal segment is aligned to a vertical grid line of level at most $d+2$, which as we noted earlier has discrete x-coordinates.

Lemma 7.4. *There exists a solution* SOL' *to the problem that consists only of well-aligned segments, and the cost of this solution is* $(1+O(\varepsilon))\cdot\mathsf{OPT}$.

Proof. We show the existence of such a solution by considering an optimal solution, and extending its segments on both sides, such that they get well-aligned. A segment $\ell \in \mathsf{OPT}$ in some level j will be of length in $(\varepsilon^j, \varepsilon^{j-1}]$. To align it to a grid line of level $j+3$ we would need to extend it by at most ε^{j+1} on each side. The new segment ℓ' thus obtained is of length

$$|\ell'| \leq |\ell| + 2\varepsilon^{j+1} \leq |\ell|\left(1+\frac{2\varepsilon^{j+1}}{|\ell|}\right) < |\ell|\left(1+\frac{2\varepsilon^{j+1}}{\varepsilon^j}\right) = |\ell|\cdot(1+2\varepsilon).$$

Therefore, the sum of weights over all the well-aligned segments is

$$\sum_{\ell\in\mathsf{OPT}} |\ell'| \leq \sum_{\ell\in\mathsf{OPT}} |\ell|\cdot(1+2\varepsilon) = (1+2\varepsilon)\cdot\mathsf{OPT}.$$ □

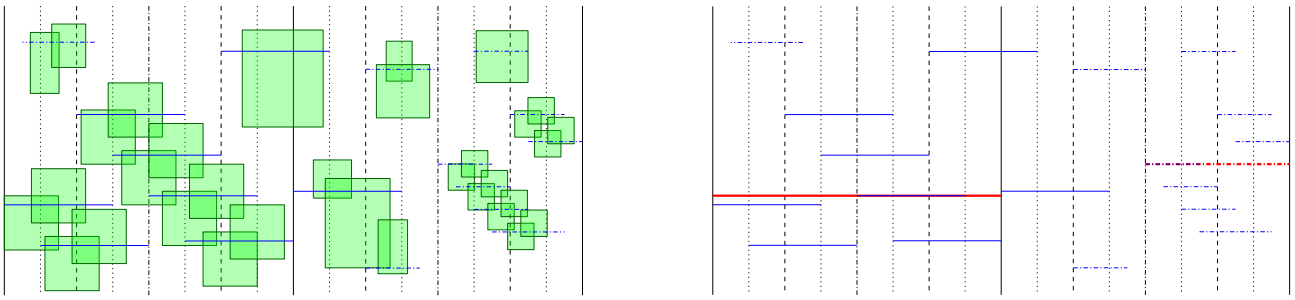
We note that since the input rectangles have been discretized, any vertical (respectively horizontal) segment in the well-aligned solution SOL' can be shifted vertically (respectively horizontally) to the nearest grid line without changing the set of rectangles it intersects. This fact, in conjunction with their well-alignment (from Lemma 7.4) ensures that the endpoints of segments in SOL' are discrete points. Since SOL' is only a $(1 + O(\varepsilon)$ factor more in cost than OPT, it is a permissible solution when constructing a PTAS. We shall hence assume that there exists an optimal solution with only discrete segments that are also well-aligned.

We define the tree T by defining recursively one of the possible operations (trivial operation, add operation, line operation) for each node v of the tree. After applying an operation, we always add children to the processed node v that corresponds to the subproblems that we reduce to, i.e., for a node v corresponding to the subproblem $\mathrm{DP}(S, \mathcal{L})$, if we are applying the trivial (respectively line) operation along a segment (respectively line) ℓ, then we add children corresponding to the DP subproblems $\mathrm{DP}(S_1, S_1 \cap \mathcal{L})$ and $\mathrm{DP}(S_2, S_2 \cap \mathcal{L})$, where S_1 and S_2 are the connected components of $S \backslash \ell$. Similarly if we apply the add operation on v with the set of segments $\mathcal{L}'$ then we add the child node corresponding to the subproblem $\mathrm{DP}(S, \mathcal{L} \cup \mathcal{L}')$.

First level ($j = 0$). Consider the line operations corresponding to vertical grid lines of level 0, in some arbitrary order. We start by applying the first operation to the root $\mathrm{DP}([0, 2n] \times [0, 2n], \emptyset)$. We then continue by applying the following operations, in sequence, to the appropriate child node obtained from the earlier operation. Consider one of the resulting subproblems $\mathrm{DP}(S, \emptyset)$. Suppose that there are more than ε^{-3} line segments (horizontal or vertical) from OPT of level 0 inside S. We want to partition S into smaller rectangles, such that within each of these rectangles S' at most ε^{-3} of these level 0 line segments start or end. This will make it possible for us to guess them. To this end, we consider the horizontal and vertical line segments from OPT of level 0 inside S, take (the multiset of) their endpoints and order these endpoints non-decreasingly by their y-coordinates. Let $p_1, p_2, \ldots, p_k$ be these points in this order. For each $k' \in \mathbb{N}$ with $k'/\varepsilon^3 \leq k$, we consider the point p_{k'/ε^3}. Let ℓ' be the horizontal line that contains p_{k'/ε^3}. We apply the line operation to ℓ'.

Proposition 7.1. *Let $\mathrm{DP}(S', \emptyset)$ be one of the subproblems after applying the operations above. There are at most ε^{-3} line segments $\mathcal{L}'$ (horizontal or vertical) from OPT of level 0 that have an endpoint inside S'.*

In each resulting subproblem $\mathrm{DP}(S', \emptyset)$, for each vertical line segment $\ell \in \mathsf{OPT}$ that crosses S', i.e., such that $S' \setminus \ell$ has two connected components, we apply the line operation for the line that contains ℓ. (As earlier, we do the operations in arbitrary order, by applying a line operation to the appropriate child node obtained after previous operations.) In each subproblem



(a) Input Instance, and segments in OPT. (b) Line operations.

Figure 7.5: Horizontal line operations.

$\mathrm{DP}(S'', \emptyset)$ obtained after this step, we apply the add operation to the line segments from OPT of level 0 that intersects S'' (or to be more precise, their intersection with S''), i.e., to the set $\mathcal{L}' := \{\ell \cap S'' \mid (\ell \in \mathsf{OPT}) \wedge (\ell \cap S'' \neq \emptyset) \wedge (\ell \text{ is of level } 0)\}$. Proposition 7.1 implies that $|\mathcal{L}'| \leq \varepsilon^{-3}$. In each obtained subproblem we apply the trivial operation until it is no longer applicable. We say that all these operations correspond to level 0.

Subsequent levels. Next, we do a sequence of operations that correspond to levels $j = 1, 2, 3, \ldots$ Assume by induction that for some j each leaf in the current tree T corresponds to a subproblem $\mathrm{DP}(S, \mathcal{L})$ such that for any line segment $\ell \in \mathsf{OPT}$ of level $j' \in [j-3, j)$ with $\ell \cap S \neq \emptyset$, we have $\ell \cap S \in \mathcal{L}$. (It is easy to see from the following construction that all such segments from OPT will be added to the subproblem. We shall formally show in Lemma 7.5, that segments from the smaller levels will in fact also get removed.) Consider one of these leaves and suppose that it corresponds to a subproblem $\mathrm{DP}(S, \mathcal{L})$. We apply the line operation for each vertical line that corresponds to a (vertical) grid line of level j intersecting S.

Consider a resulting subproblem $\mathrm{DP}(S', \mathcal{L})$. Suppose that there are more than ε^{-3} line segments (horizontal or vertical) from OPT of level j that have an endpoint inside S'. Like above, we consider these endpoints and we order them non-decreasingly by their y-coordinates. Let $p_1, p_2, \ldots, p_k$ be these points in this order. For each $k' \in \mathbb{N}$ with $k'/\varepsilon^3 \leq k$, we consider the point p_{k'/ε^3} and apply the line operation for the horizontal line ℓ' that contains p_{k'/ε^3}. If for a resulting subproblem $\mathrm{DP}(S'', \mathcal{L})$ there is a vertical line segment $\ell \in \mathcal{L}$ of some level $j' < j - 2$ with an endpoint p inside S'', then we apply the line operation for the horizontal line that contains p.

Proposition 7.2. *Let* $\mathrm{DP}(S'', \mathcal{L})$ *be one of the subproblems after applying the operations above. There are at most* ε^{-3} *line segments* $\mathcal{L}'$ *(horizontal or vertical) from* OPT *of level* j *that have an endpoint inside* S''.

Consider a resulting subproblem $\mathrm{DP}(S'', \mathcal{L})$. For each line segment $\ell \in \mathsf{OPT}$ such that ℓ crosses S'', i.e., $S'' \setminus \ell$ has two connected components, we apply the line operation to the line that contains ℓ. In each subproblem $\mathrm{DP}(S''', \mathcal{L})$ obtained after this step, we intuitively apply the add operation to the line segments of level j that intersect S'''. Formally, we apply them to the line segments in the set $\mathcal{L}' := \{\ell \cap S''' \mid (\ell \in \mathsf{OPT}) \wedge (\ell \cap S''' \neq \emptyset) \wedge (\ell \text{ is of level } j)\}$. We will prove later that $|\mathcal{L}'|$ is sufficiently small such that $|\mathcal{L} \cup \mathcal{L}'| \leq 3\varepsilon^{-3}$ and, hence, we are allowed to apply the add operation to $|\mathcal{L}'|$. Also, we will show that after this operation, the resulting set of line segments $\mathcal{L} \cup \mathcal{L}'$ contains $\ell \cap S'''$ for each line segment $\ell \in \mathsf{OPT}$ of level at least $j-3$ that intersects S'''.

We now apply the trivial operation until it is no longer applicable. We say that the above series of operations correspond to level j.

As an example, look at Figure 7.5, with $\varepsilon = 1/2$. The horizontal and vertical solid segments in it are of level 0, the horizontal and vertical dashed-dotted segments are of level 1, the vertical dashed segments are of level 2, and the vertical dotted segments are of level 3. Also, the black segments are vertical grid lines, the blue segments are (well-aligned) segments in OPT and the red segments are lines along which horizontal line operations are applied. Here, say for level 0, we first apply line operations along the vertical grid lines of that level to get two subproblems. Then in the left subproblem, we start counting endpoints of segments in OPT, and at every eighth point, we do a horizontal line operation along its y-coordinate. Then we continue doing operations for the further levels. It can also be seen here that a well-aligned segment of level 0, always starts and ends at grid lines of level 3. So when we do operations corresponding to level 3, it will get removed by a trivial operation (we prove this formally in Lemma 7.5).

7.3.3 Analysis of the DP-Decision Tree

We want to prove that the resulting tree T is indeed a DP-decision tree corresponding to a solution of cost at most $(1+\varepsilon)\mathsf{OPT}$. To this end, first we need to show that whenever we apply the add operation to a subproblem $\mathrm{DP}(S, \mathcal{L})$ for a set $\mathcal{L}'$ then $|\mathcal{L}| + |\mathcal{L}'| \leq 3\varepsilon^{-3}$. The *key insight* for this is that if we added a line segment $\ell \in \mathsf{OPT}$ of some level j, then it will not be included in the respective set $\mathcal{L}$ of later subproblems of level $j+3$ or higher since ℓ is well-aligned. More precisely, if ℓ is horizontal then its x-coordinates are aligned with the grid lines of level $j+3$. Hence, if ℓ or a part of ℓ is contained in a set $\mathcal{L}$ of some subproblem $\mathrm{DP}(S, \mathcal{L})$ of level $j+3$, then we applied the trivial operation to ℓ (since it is well-aligned) and thus ℓ "disappeared" from $\mathcal{L}$ (note that here, by disappear we mean that the segment does not need to be considered in $\mathcal{L}$ anymore, and gets added to the solution of the DP subproblem). If ℓ is vertical and it appears in a $\mathrm{DP}(S, \mathcal{L})$ of level $j+3$ then we applied the line operation to the horizontal lines

that contain the two endpoints of ℓ. Afterward, we applied the trivial operation to ℓ until ℓ "disappeared" from $\mathcal{L}$.

In particular, for each subproblem $\mathrm{DP}(S, \mathcal{L})$ constructed by operations of level j, the set $\mathcal{L}$ can contain line segments of levels $j-2, j-1$, and j; but no line segments of a level j' with $j' < j-2$. We make this and some other technicalities formal in the proof of the next lemma.

Lemma 7.5. *The constructed tree T is a DP-decision tree.*

Proof. We first notice that any reduction in the tree corresponding to a trivial or line operation on $\mathrm{DP}(S, \mathcal{L})$ can only lead to subproblems of the form $\mathrm{DP}(S', \mathcal{L}')$ with $\mathcal{L}' \subseteq \mathcal{L}$, and not lead to any violations of the DP-decision tree properties.

We now consider the add operations to ensure that any newly created node, by applying the add operation on the set $\mathcal{L}' := \{\ell \cap S' \mid (\ell \in \mathsf{OPT}) \wedge (\ell \cap S' \neq \emptyset) \wedge (\ell \text{ is of level } j)\}$ to the subproblem $\mathrm{DP}(S, \mathcal{L})$, maintains the property that $|\mathcal{L} \cup \mathcal{L}'| \leq 3\varepsilon^{-3}$. We do this by induction,

Hypothesis: We assume that all add operations in the sequence of operations corresponding to level $j-1$, satisfied the property that $|\mathcal{L}| + |\mathcal{L}'| \leq 3\varepsilon^{-3}$.

Base case: For the sequence of operations of level $j < 3$, we know by Proposition 7.1 and Proposition 7.2 that the 'added' set $\mathcal{L}'$ always satisfies $|\mathcal{L}'| \leq \varepsilon^{-3}$. Since these are the only added sets in the first 3 levels, we can be sure that $|\mathcal{L}| + |\mathcal{L}'| \leq 3\varepsilon^{-3}$.

Induction: Consider an add operation for segments in OPT of level j. Any such operation was preceded by a series of line operations along grid lines of level j and all viable trivial operations. The grid lines of level j are separated by ε^{j-2} units, and hence any horizontal segments from OPT in $\mathcal{L}$ of level $j' < j-2$ (which have length $\geq \varepsilon^{j-2}$ and being well-aligned, completely cut across any cell they intersect) can have the trivial operation applied to them, and be removed from $\mathcal{L}$. Similarly, for any vertical segment in OPT of level $j' < j-2$, we explicitly apply the line operation to the horizontal lines that contain its endpoints, and hence again this also gets removed from $\mathcal{L}$ when we apply all possible trivial operations. Thus, we are left only with segments added in operations of levels $j-1$ and $j-2$ in $\mathcal{L}$. But we know from Proposition 7.2 that at each level we add at most ε^{-3} segments. Thus, the application of an add operation at this stage ensures that $|\mathcal{L}| + |\mathcal{L}'| \leq 3\varepsilon^{-3}$. □

We want to show that the cost of the solution corresponding to T is at most $(1+O(\varepsilon))\mathsf{OPT}$. In fact, depending on the offset a, this might or might not be true. However, we show that there is a choice for a such that this is true (in fact, we will show that for a random choice

for a, the cost will be at most $(1 + O(\varepsilon))\mathsf{OPT}$ in expectation). Intuitively, when we apply the line operation to a vertical grid line ℓ of some level j then the incurred cost is at most $O(1)$ times the cost of the line segments from OPT of level j or larger that stab at least one rectangle intersected by ℓ. A line segment $\ell' \in \mathsf{OPT}$ of level j stabs such a rectangle only if ℓ' is intersected by ℓ (if ℓ' is horizontal) or the x-coordinate of ℓ' is close to ℓ (if ℓ' is vertical). Here we use that $h(R_i) \geq w(R_i)$ for each rectangle $R_i \in \mathcal{B}$.

Thus, we want to bound the total cost over all levels j of the line segments from OPT that are in level j and that are intersected or close to grid lines of level j (or smaller levels, but this need not be separately considered since all grid lines of smaller levels are also grid lines of level j). We will show that if we choose a randomly then the total cost of such grid lines is at most $\varepsilon \cdot \mathsf{OPT}$ in expectation. Hence, by using a constant approximation algorithm [40], in expectation the total cost due to all line operations for vertical line segments is at most $O(\varepsilon) \cdot \mathsf{OPT}$.

When we apply the line operation for a horizontal line, then the cost of stabbing the corresponding rectangles is at most the width of the rectangle S of the current subproblem $\mathrm{DP}(S, \mathcal{L})$. We will charge this cost to the line segments of OPT inside S of the current level or higher levels (that is, levels of higher numerical value). We will argue that we can charge each such line operation to line segments from OPT whose total width is at least $1/\varepsilon$ times the width of S. This costs another $O(\varepsilon) \cdot \mathsf{OPT}$ in total due to all applications of line operations for horizontal line segments.

The add operation yields a cost of exactly OPT and the trivial operation does not cost anything. This yields a total cost of $(1 + O(\varepsilon))\mathsf{OPT}$.

Formally, we prove this in the following lemma.

Lemma 7.6. *There is a discrete value for the offset a such that $a \in \{0, \varepsilon^d, 2\varepsilon^d, \ldots, \varepsilon^{-2}\}$ and the solution $\overline{\mathsf{SOL}}([0, 2n] \times [0, 2n], \emptyset)$ in T has a cost of at most $(1 + O(\varepsilon))\mathsf{OPT}$.*

Proof. In the tree as defined above, the add operations are only applied on segments from OPT (or their parts), and hence the cost across all such add operations is at most $c(\mathsf{OPT})$. Similarly, the trivial operations are applied on segments that were 'added' before, and hence their cost is also already accounted for. So we are left with analyzing the cost of stabbing the rectangles which are intersected by the lines along which we apply the line operations. We claim that for a discretized random offset $a \in \{0, \varepsilon^d, 2\varepsilon^d, \ldots, \varepsilon^{-2}\}$, the expected cost is $O(\varepsilon \cdot \mathsf{OPT})$, which would give us the required result.

Let us first consider any line operation of level j that is applied to a horizontal line ℓ. This operation would create two cells of width at most ε^{j-2} (since vertical grid lines of level j are separated by ε^{j-2}), one of which either contains at least ε^{-3} endpoints of segments (horizontal

or vertical) from OPT of level j; or contains at least one vertical segment from OPT of level $j' < j-2$, i.e., the cost of the segments from OPT with at least one endpoint in this cell is at least ε^{j-3} (in the former case, each of the segments is of length at least ε^j, and in the latter the single segment is of the required length). Since a segment of width ε^{j-2} (width of the cell) is sufficient to stab all rectangles stabbed by ℓ, we see that this horizontal line takes only ε times the cost of the segments in OPT with at least one endpoint in the cell. We charge the cost of this horizontal segment to these corresponding endpoints. Since each such segment in OPT of level j can be charged at most twice, by summing over all horizontal line operations over all levels we get that the cost of such line operations is at most $2\varepsilon \cdot \mathsf{OPT}$.

Now, let us consider the line operations applied to vertical grid lines. We wish to bound the cost of stabbing all the rectangles intersected or *close to* grid lines (will be formally defined shortly), over all levels j. This as mentioned above can also be stated as bounding the cost, over all levels j, of line segments of level j in OPT (call this set OPT_j) intersected or close to grid lines of level j. For a horizontal segment $\ell \in \mathsf{OPT}_j$, let I_ℓ be the indicator variable representing the event that a grid line of level j intersects ℓ ($I_\ell = 0$ for vertical segments). Now, since any two consecutive grid lines of level j are separated by ε^{j-2}, there are $\varepsilon^{j-2}/\varepsilon^d$ possible shifts for these grid lines due to our offset, and each of these shifts has the same probability. Similarly, since $|\ell| \leq \varepsilon^{j-1}$ (by virtue of being well-aligned), at most $(\varepsilon^{j-1}/\varepsilon^d) + 1$ of the valid offsets will cause a grid line to intersect ℓ. So, if we take a random discrete offset $a \in \{0, \varepsilon^d, 2\varepsilon^d, \ldots, \varepsilon^{-2}\}$, we have that

$$\mathbb{E}[I_\ell] \leq \frac{(\varepsilon^{j-1}/\varepsilon^d) + 1}{\varepsilon^{j-2}/\varepsilon^d} = \varepsilon + \varepsilon^{2+(d-j)} \leq 2\varepsilon \qquad \text{(since } j \in [0, d-1].\text{)}$$

For a vertical segment $\ell \in \mathsf{OPT}_j$, let J_ℓ be the indicator variable representing the event that a grid line of level j intersects a rectangle stabbed by ℓ ($J_\ell = 0$ for horizontal segments). Since for $j > 0$, $|\ell| \leq \varepsilon^{j-1}$, we know that for each rectangle R_i stabbed by ℓ, the dimensions satisfy $w(R_i) \leq h(R_i) \leq \varepsilon^{j-1}$. This means that to stab such a rectangle, ℓ has to lie *close to* (that is, within $\pm\varepsilon^{j-1}$ of) the vertical grid line, further implying that at most $(2\varepsilon^{j-1}/\varepsilon^d) + 1$ of the discrete offsets are valid for event J_ℓ. So for a discrete random offset $a \in \{0, \varepsilon^d, 2\varepsilon^d, \ldots, \varepsilon^{-2}\}$ and level $j > 0$ we have that:

$$\mathbb{E}[J_\ell] \leq \frac{(2\varepsilon^{j-1})/\varepsilon^d + 1}{\varepsilon^{j-2}/\varepsilon^d} = 2\varepsilon + \varepsilon^{2+(d-j)} \leq 3\varepsilon.$$

For level $j = 0$, we note that even though the vertical segments can be very long, the maximum width of a rectangle is at most 1. So ℓ has to lie within ± 1 of the grid line. So if we take a

random offset $a \in \{0, \varepsilon^d, 2\varepsilon^d, \ldots, \varepsilon^{-2}\}$ we have that

$$\mathbb{E}[J_\ell] \leq \frac{2/\varepsilon^d + 1}{\varepsilon^{-2}/\varepsilon^d} = 2\varepsilon^2 + \varepsilon^{d+2} \leq 3\varepsilon.$$

With the expectations computed above, we can upper bound the expected cost of segments in OPT intersected by, or *close to* vertical grid lines as:

$$\begin{aligned}
\mathbb{E}\left[\sum_j \sum_{\ell \in \mathsf{OPT}_j} (I_\ell + J_\ell) \cdot |\ell|\right] &= \sum_j \sum_{\ell \in \mathsf{OPT}_j} \mathbb{E}\left[(I_\ell + J_\ell) \cdot |\ell|\right] \\
&= \sum_j \sum_{\ell \in \mathsf{OPT}_j} |\ell| \cdot (\mathbb{E}[I_\ell] + \mathbb{E}[J_\ell]) \\
&\leq \sum_j \sum_{\ell \in \mathsf{OPT}_j} |\ell| \cdot (2\varepsilon + 3\varepsilon) \\
&= 5\varepsilon \cdot \mathsf{OPT}.
\end{aligned}$$

Recall that the bound above gives us a bound on the total cost of vertical line operations.

Now, by using an α-approximation algorithm for stabbing [40], where α is a constant, the solution returned by our algorithm takes an additional cost of $5\alpha \cdot \varepsilon \cdot \mathsf{OPT}$. □

For a fixed value of our random offset a, the running time of our algorithm is polynomial. There are only $\varepsilon^{-2}/\varepsilon^d = \varepsilon^{-d-2} < n/\varepsilon^5$ possible choices for a. Therefore, we can derandomize our algorithm by simply trying all of these possible choices and returning the best solution obtained in this way.

Now we prove our main theorem.

Theorem 7.1. *There is a $(1+\varepsilon)$-approximation algorithm for HV-Stabbing with a running time of $(n/\varepsilon)^{O(1/\varepsilon^3)}$, assuming that $h(R_i) \geq w(R_i)$ for each rectangle $R_i \in \mathcal{B}$.*

Proof. We gave a DP algorithm in Section 7.3.1 which was shown to have the required running time in Lemma 7.3. Further in Lemma 7.6 we showed the correctness and that the solution computed by the DP is actually a $1 + O(\varepsilon)$ approximation of the solution.

□

Theorem 7.1 has some direct implications. First, it yields a PTAS for the horizontal-vertical square stabbing problem.

Corollary 7.1. *There is a PTAS for the horizontal-vertical square stabbing problem.*

Also, it yields a $(2+\varepsilon)$-approximation algorithm for the HV-Stabbing problem for arbitrary rectangles: we can simply split the input into rectangles R_i for which $h(R_i) \geq w(R_i)$ holds, and those for which $h(R_i) < w(R_i)$ holds, and output the union of these two solutions.

Corollary 7.2. *There is a $(2+\varepsilon)$-approximation algorithm for the HV-Stabbing problem with a running time of $(n/\varepsilon)^{O(1/\varepsilon^3)}$.*

Finally, it yields a PTAS for the horizontal Rectangle Stabbing problem: we can take the input of that problem and stretch all input rectangles vertically such that it is always very costly to stab any rectangle vertically (so in particular our $(1+\varepsilon)$-approximate solution would never do this). Then we apply the algorithm due to Theorem 7.1.

Corollary 7.3. *There is a $(1+\varepsilon)$-approximation algorithm for the horizontal Rectangle Stabbing problem with a running time of $(n/\varepsilon)^{O(1/\varepsilon^3)}$.*

7.4 Extensions to Similar Problems

7.4.1 δ-large Rectangles

We now consider the case of δ-large rectangles for some given constant δ, i.e., where for each input rectangle R_i we assume that $w(R_i) \leq 1$, and $h(R_i) \leq 1$, and additionally $w(R_i) \geq \delta$ or $h(R_i) \geq \delta$. In other words, none of the rectangles are small (less than δ) in both dimensions at the same time. For this case, we again give a PTAS in which we use our algorithm due to Theorem 7.1 as a subroutine.

First, by losing only a factor of $1+\varepsilon$, we divide the instance into independent subproblems which are disjoint rectangular cells. For each cell C_i, we denote by $\mathsf{OPT}(C_i)$ the segments from OPT that are contained in C_i and our routine ensures that $c(\mathsf{OPT}(C_i)) \leq O(1/\varepsilon^3)$. Then for each cell C_i, the number of segments in $\mathsf{OPT}(C_i)$ with a length longer than δ is bounded by $O(1/(\delta\varepsilon^3))$. We guess them in polynomial time. Now, the remaining segments in OPT are all of length smaller than δ, and hence they can stab a rectangle only along its shorter dimension. Hence, we can divide the remaining rectangles into two disjoint sets, one with $h(R_i) \geq w(R_i)$ and the other with $w(R_i) > h(R_i)$, and use Theorem 7.1 to get a $1+\varepsilon$ approximation of the remaining problem. In the following, we describe our algorithm in detail.

7.4.1.1 Guessing Long Segments in the Solution

In the following, we will call a coordinate *discrete* if it is an integral multiple of ε/n. We first start by discretizing the input in a similar way as in Lemma 7.1,

Lemma 7.7. *By losing a factor $1 + O(\varepsilon)$ in the approximation ratio, we can assume for each $R_i \in \mathcal{B}$ that the following properties hold:*

(i) $x_1^{(i)}, x_2^{(i)}, y_1^{(i)}, y_2^{(i)}$ *are discrete and within* $[0, n]$,

(ii) each line segment in OPT *has length of at most* $1/\varepsilon$.

Proof. We first prove Property (i). Since each rectangle $R_i \in \mathcal{B}$ has a width $w(R_i) \leq 1$, the instance either has some x-coordinate that is not covered by any rectangle – in which case we can split the instance into smaller subproblems around this x-coordinate – or the total width of the instance is at most n. Hence, all the x-coordinates of the rectangles can be assumed to be between 0 and n. Further, we extend the rectangles on both sides to make their x-coordinates align with the next nearest multiple of ε/n (discretization). Clearly this involves extension by at most $2\varepsilon/n$ to the width of every rectangle, and at most a 2ε addition to the cost of the solution. We can use the same argument to show that the heights are also discretized and between 0 and n, since the heights of all rectangles are also less than 1. The height discretization similarly adds another 2ε to the cost of the solution.

Now we look at Property (ii). Consider any horizontal (respectively vertical) segment $\ell \in \mathsf{OPT}$, that is longer than $1/\varepsilon$. From the left (respectively bottom) endpoint, we divide the segment into consecutive smaller segments of length $1/\varepsilon - 2$ each, with one potential last piece being smaller than $1/\varepsilon - 2$. Now, for each smaller segment, we extend it on both sides in such a way that it completely stabs the rectangles that it intersects. Since the maximum width (respectively height) of a rectangle is 1, we extend each such segment by at most 2 units.

With the same calculation as in the proof of Lemma 7.1, we can bound the increase of our cost by

$$\left(|\ell| + \left\lceil \frac{|\ell|}{1/\varepsilon - 2} \right\rceil \cdot 2\right) \cdot \frac{1}{|\ell|} \leq 1 + 8\varepsilon.$$

□

Then we continue by constructing vertical grid lines at each x-coordinate $a + k \cdot \varepsilon^{-2}$ with $k \in \mathbb{N}$ and a random offset $a \in \mathbb{N}_0$. By removing the rectangles that intersect with these grid lines, we divide the input instance into strips of width $1/\varepsilon^2$ (and height n) each. Next, we show that the removed rectangles can be stabbed at very low total cost.

Lemma 7.8. *Consider the input rectangles that are intersected by vertical grid lines. They can all be stabbed by a set of segments of total expected cost $O(\varepsilon) \cdot \mathsf{OPT}$.*

Proof. Similar to Lemma 7.6, we wish to bound the cost of stabbing all the rectangles intersected or *close to* grid lines, which can also be stated as bounding the cost, of line segments of OPT intersected or close to the grid lines. For a horizontal segment $\ell \in \mathsf{OPT}$, let I_ℓ be the indicator variable representing the event that a grid line intersects ℓ ($I_\ell = 0$ for vertical segments). Since $|\ell| \leq 1/\varepsilon$ (by Lemma 7.7), if we take a random offset a, we have that,

$$\mathbb{E}[I_\ell] \leq \frac{\varepsilon^{-1}}{\varepsilon^{-2}} = \varepsilon.$$

For a vertical segment $\ell \in \mathsf{OPT}$, let J_ℓ be the indicator variable representing the event that a grid line intersects a rectangle stabbed by ℓ ($J_\ell = 0$ for horizontal segments). Since the width of all rectangles is less than 1, to stab a rectangle, ℓ has to lie *close to* (that is, within ± 1 of) the vertical grid line. So for a random offset a we have that,

$$\mathbb{E}[J_\ell] \leq \frac{2}{\varepsilon^{-2}} = 2\varepsilon^2 \leq 2\varepsilon.$$

With the expectations computed above, we can upper bound the expected cost of segments in OPT intersected by vertical line operations as,

$$\begin{aligned}
\mathbb{E}\left[\sum_{\ell \in \mathsf{OPT}} (I_\ell + J_\ell) \cdot |\ell|\right] &= \sum_{\ell \in \mathsf{OPT}} \mathbb{E}\left[(I_\ell + J_\ell) \cdot |\ell|\right] \\
&= \sum_{\ell \in \mathsf{OPT}} |\ell| \cdot (\mathbb{E}[I_\ell] + \mathbb{E}[J_\ell]) \\
&\leq \sum_{\ell \in \mathsf{OPT}} |\ell| \cdot (\varepsilon + 2\varepsilon) \\
&= 3\varepsilon \cdot \mathsf{OPT}
\end{aligned}$$

Now, by using the α-approximate algorithm for HV-Stabbing [40], where α is a constant, we can find a set of segments that stabs the set of rectangles intersected by the vertical grid lines with an additional cost of $3\alpha \cdot \varepsilon \cdot \mathsf{OPT}$.

□

Now we divide these vertical strips into smaller rectangular cells such that the optimal solution for each cell is $O(1/\varepsilon^3)$.

Lemma 7.9. *In time $n^{O(1)}$ we can compute a set of horizontal line segments of total cost $O(\varepsilon) \cdot \mathsf{OPT}$ that, together with the vertical grid lines, divides the input instance into rectangular cells containing independent subproblems, each with an optimal solution of cost $O(1/\varepsilon^3)$.*

Proof. We will consider each vertical strip of width at most $1/\varepsilon^2$ separately as they constitute independent subproblems. Consider one such strip. Since all rectangles are contained in $[0, n] \times [0, n]$, we consider only the portion of the strip contained in $[0, n] \times [0, n]$. We sweep a horizontal line from bottom to top, and at each discrete y-coordinate compute a constant approximate solution (using a known $O(1)$-approximation algorithm [40] for HV-Stabbing) for the subproblem of the rectangles that are completely contained in the area of the strip below the line down to the bottom of the strip, or the end of a previous cell, whichever is closer. (Note here that we are only computing these solutions, and not adding all of them to our final answer.) At the smallest y-coordinate y_0, at which the cost of the solution exceeds $1/\varepsilon^3$, we take the line $[0, 1/\varepsilon^2] \times y_0$ as a part of our solution and discard the rectangles already stabbed by it. Note that if we had moved the line up by another ε/n, the cost of the solution of the considered area could have increased by at most $1/\varepsilon^2$ (by also considering the segment $[0, 1/\varepsilon^2] \times y_0$ as part of the solution of the considered area), therefore the cost of segments from OPT in the considered area is between $1/(\alpha\varepsilon^3)$ and $1/\varepsilon^3 + 1/\varepsilon^2$, which is $\Omega(1/\varepsilon^3)$.

Thus, every time we perform a step as described above, we add a segment of length $1/\varepsilon^2$ to the solution, and create a subproblem that has an optimal solution of cost $\Omega(1/\varepsilon^3)$. Hence the cost of the added segment is at most $O(\varepsilon^{-2}/\varepsilon^{-3}) = O(\varepsilon)$ times the cost of the solution to the subproblem that it creates. Since an optimal solution to all such subproblems can be at most the cost of the original instance, the cost of the horizontal segments that we have added is at most $O(\varepsilon) \cdot \mathsf{OPT}$. □

Now, in any optimal solution to a subproblem contained in a cell as described above, there can be at most $O(1/(\delta\varepsilon^3))$ *long segments*, i.e., segments of length at least δ. Since there are only a polynomial number of discrete candidate segments we can guess the set of long segments in the solution in polynomial time (the exact analysis is shown later).

7.4.1.2 Computing Short Segments in the Solution

For each guess made for the set of long segments in the solution, we compute a $(1 + \varepsilon)$-approximation for the *short segments*, i.e., segments of length at most δ in the solution using Theorem 7.1.

Note that any rectangle with both height and width at least δ will have to be stabbed by a segment of length at least δ. However, since all such long segments in the solution have already been guessed, each remaining segment can stab a δ-large rectangle only along its shorter dimension. So the rectangles in the instance that have not yet been stabbed can be partitioned into *two* independent instances, one which has rectangles with width less than δ (will be stabbed horizontally) and one which has rectangles with height less than δ (will be stabbed vertically).

We apply Theorem 7.1 on each of these independent instances and combine the solutions to get a $(1+\varepsilon)$-approximate solution for the remaining problem.

Theorem 7.2. *For HV-Stabbing with δ-large rectangles, there is a $(1+\varepsilon)$-approximation algorithm with a running time of $(n/\varepsilon)^{O(1/(\delta\varepsilon^3))}$.*

Proof. First, we analyze the running time. In the first phase of the algorithm, we divide the given instance into $n/(1/\varepsilon^2) = n\varepsilon^2$ strips, and run the constant-factor $n^{O(1)}$ approximation algorithm [40] for each discrete y-coordinate. This requires a running time of $n\varepsilon^2 \times n/(\varepsilon/n) \times n^{O(1)} = \varepsilon \cdot n^{O(1)}$. Now in a strip there are at most $(n/(\varepsilon/n)) \times ((1/\varepsilon^2)/(\varepsilon/n)) = n^3/\varepsilon^4$ possible discrete endpoints for segments, which means there are less than $\binom{n^3/\varepsilon^4}{2}$, i.e., $O(n^6/\varepsilon^8)$ possible candidate segments in a strip. Hence, we can enumerate over all of their subsets of size $O(1/(\delta\varepsilon^3))$ in $(n/\varepsilon)^{O(1/(\delta\varepsilon^3))}$ time. Following this we have two invocations of Theorem 7.1 to solve the independent subproblems which would together take $(n/\varepsilon)^{O(1/\varepsilon^3)}$. This yields an overall running time of $(n/\varepsilon)^{O(1/(\delta\varepsilon^3))}$.

Now to show correctness, let us partition the OPT set into three sets, OPT_δ consisting of segments of length at least δ, OPT_x consisting of horizontal segments shorter than δ, and OPT_y consisting of vertical segments shorter than δ. We can correctly guess OPT_δ (up to a factor of $1+O(\varepsilon)$ by Lemma 7.7 and Lemma 7.9), and hence identify the rectangles that are stabbed by OPT_x but not by OPT_δ (since these are the remaining rectangles which have width at most δ) and by OPT_y but not by OPT_δ. Now by Theorem 7.1 we can compute $(1+\varepsilon)$-approximations for these two sets, and hence get a solution of cost

$$\mathsf{SOL} \leq (1+O(\varepsilon))\mathsf{OPT}_\delta + (1+\varepsilon)\mathsf{OPT}_x + (1+O(\varepsilon))\mathsf{OPT}_y \leq (1+O(\varepsilon))\mathsf{OPT}.$$

Now by appropriately adjusting the value of ε, we can obtain the claimed result. □

7.4.2 Generalized Minimum Manhattan Network

In the Generalized Minimum Manhattan Network problem (GMMN), we are given a set R of n unordered terminal pairs, and the goal is to find a minimum-length rectilinear network such that every pair in R is connected by a Manhattan path, which is a path consisting of axis-parallel segments whose total length is the Manhattan distance of the pair under consideration.

For the special case of 2D-GMMN, where all terminals lie on a 2-dimensional plane, Das, Fleszar, Kobourov, Spoerhase, Veeramoni, and Wolff [55] gave an $O(\log n)$-approximate solution using a $(6+\varepsilon)$-approximate solution for x-separated 2D-GMMN instances. An instance of GMMN is called x-separated if there is a vertical line that intersects every rectangle formed

by a pair of terminals as the diagonally opposite corners of the rectangle. Let $\mathsf{OPT}_{\text{ver}}$ and $\mathsf{OPT}_{\text{hor}}$ refer to the cost of the vertical and horizontal segments respectively in an optimal solution. The algorithm solves an x-separated instance by giving a solution of the form $N = A_{\text{up}} \cup A_{\text{down}} \cup S$, where S is a set of horizontal segments stabbing all the rectangles in the instance and $A_{\text{up}} \cup A_{\text{down}}$ is a set of segments of cost bounded by $(2 + 2\varepsilon) \cdot (\mathsf{OPT} + \mathsf{OPT}_{\text{ver}})$. Using a 4-approximation algorithm for finding the set of segments S, horizontally stabbing the rectangles, a bound of $(6 + \varepsilon) \cdot \mathsf{OPT}$ is obtained for the cost of segments in N. But instead of the 4-approximation algorithm we can instead use the PTAS from Corollary 7.3 to obtain a better bound of $(4 + \varepsilon) \cdot \mathsf{OPT}$ on the cost as follows,

$$\begin{aligned} c(N) &= c(A_{\text{up}}) + c(A_{\text{down}}) + c(S) \\ &\leq (2 + 2\varepsilon) \cdot (\mathsf{OPT} + \mathsf{OPT}_{\text{ver}}) + (1 + \varepsilon) \cdot \mathsf{OPT}_{\text{hor}} \\ &\leq (2 + 2\varepsilon) \cdot \mathsf{OPT} + (2 + 2\varepsilon) \cdot (\mathsf{OPT}_{\text{ver}} + \mathsf{OPT}_{\text{hor}}) \\ &= (4 + \varepsilon') \cdot \mathsf{OPT}. \end{aligned}$$

7.5 PTAS for k-shapes with Bounded Width Variance

In this section, we present a Polynomial-Time Approximation Scheme (PTAS) for the k-Stabbing problem restricted to instances where the widths of the constituent rectangles of each k-shape are relatively similar. Specifically, we consider the case where k is a constant and, for each input k-shape K, and for any two of its constituent rectangles $R_i, R_j \subseteq K$, it holds that $\delta w(R_j) \leq w(R_i) \leq w(R_j)/\delta$ for a given constant $\delta \in (0, 1]$.

This algorithm generalizes the PTAS for rectangles presented in Section 7.3. The core difficulty in moving from rectangles to k-shapes is that a single k-shape might be stabbed by segments of vastly different lengths. However, the bounded width constraint allows us to approximate these shapes with bounding boxes without losing too much in the approximation factor during specific recursive steps.

7.5.1 Dynamic Program

We again employ a dynamic programming approach based on a hierarchical decomposition of the plane. We define a DP table containing cells $\text{DP}(R, \mathcal{S})$ for each combination of:

- A closed rectangle $R \subseteq [0, \alpha n] \times [0, (k + 1)n]$ with discrete coordinates.
- A set $\mathcal{S}$ of at most $3\varepsilon^{-3}$ discrete horizontal line segments that intersect R.

The value stored in $\mathrm{DP}(R, \mathcal{S})$ represents the minimum-cost set of additional segments required to stab all k-shapes contained in R that are *not* already stabbed by $\mathcal{S}$.

Operations. For a cell $\mathrm{DP}(R, \mathcal{S})$, we calculate the solution as follows:

1. **Base Case:** If $\mathcal{S}$ stabs all k-shapes inside R, return $\emptyset$.
2. **Trivial Operation:** If there exists a segment $\ell \in \mathcal{S}$ that cuts R into two connected components R_1 and R_2, we divide the problem. Let $\mathcal{S}_1$ and $\mathcal{S}_2$ be the subsets of $\mathcal{S}$ intersecting R_1 and R_2 respectively. We set:
$$\mathrm{DP}(R, \mathcal{S}) = \mathrm{DP}(R_1, \mathcal{S}_1) \cup \mathrm{DP}(R_2, \mathcal{S}_2) \cup \{\ell\}.$$
3. **Recursive Step:** If no trivial operation is possible, we define the solution as the minimum-cost over the following two operations:
   - **Add Operation:** We guess a small set of segments. We iterate over all sets $\mathcal{S}'$ of discrete segments in R such that $|\mathcal{S}| + |\mathcal{S}'| \leq 3\varepsilon^{-3}$. The candidate solution is $\mathcal{S}' \cup \mathrm{DP}(R, \mathcal{S} \cup \mathcal{S}')$.
   - **Line Operation:** We iterate over all vertical lines ℓ intersecting the interior of R. Let $\mathcal{K}_\ell$ be the set of k-shapes in R intersected by ℓ. We approximate the shapes in $\mathcal{K}_\ell$ by their bounding boxes. Let $\mathcal{R}_\ell$ be the set of these bounding boxes. We solve the Rectangle Stabbing problem on $\mathcal{R}_\ell$ using the PTAS from Section 7.3 to obtain a set of segments $\mathcal{S}_\ell$. Let R_1, R_2 be the components of $R \setminus \ell$. The candidate solution is:
   $$\mathcal{S}_\ell \cup \mathrm{DP}(R_1, \mathcal{S} \cap R_1) \cup \mathrm{DP}(R_2, \mathcal{S} \cap R_2).$$

The final solution is obtained from $\mathrm{DP}([0, \alpha n] \times [0, (k+1)n], \emptyset)$.

7.5.2 Analysis

We now analyze the running time and the approximation guarantee.

Lemma 7.10 (Runtime)**.** *The running time of the dynamic program is* $(kn/\varepsilon)^{O(1/\varepsilon^3)}$.

Proof. We know that the DP always picks the sequence of operations that gives a result of minimum cost. Since there exists a sequence of operations that produces a solution of cost $(1+O(\varepsilon))\mathsf{OPT}$ (as shown in the subsequent analysis), the DP returns a solution of cost at most that. To analyze the running time, we first bound the number of possible states in the DP table. Recall that a subproblem is defined by a rectangular cell R and a set of segments $\mathcal{S}$.

Counting Discrete Points: The algorithm operates on a discretized grid. The number of discrete x-coordinates is at most $\frac{\alpha n}{\varepsilon^d}$. The number of discrete y-coordinates is at most $(k+1)n$. Thus, the total number of discrete points in the grid is bounded by:

$$N_{\text{pts}} = \frac{\alpha n}{\varepsilon^d} \times (k+1)n \leq \frac{\alpha n \cdot n}{\varepsilon^3} \times (k+1)n = O(\alpha k \varepsilon^{-3} n^3),$$

where we used the definition $\varepsilon^d \geq \varepsilon^3/n$.

Counting Rectangles (R): A rectangle R is defined by two discrete points (its top-left and bottom-right corners). Therefore, the number of possible rectangles is at most:

$$N_{\text{rect}} \leq \binom{N_{\text{pts}}}{2} = O(k^2 n^6 \varepsilon^{-6}).$$

Counting Segments ($\mathcal{S}$): A discrete horizontal line segment is defined by two x-coordinates (start and end) and one y-coordinate. The total number of possible valid discrete segments is:

$$N_{\text{seg}} \leq \binom{\alpha n/\varepsilon^d}{2} \times (k+1)n \leq O\left(\frac{n^4}{\varepsilon^6} \cdot kn\right) = O(kn^5 \varepsilon^{-6}).$$

The set $\mathcal{S}$ in a DP cell contains at most $3\varepsilon^{-3}$ such segments. The number of such subsets is bounded by:

$$N_{\text{sets}} \leq \sum_{i=0}^{3\varepsilon^{-3}} \binom{N_{\text{seg}}}{i} \leq (N_{\text{seg}})^{3\varepsilon^{-3}} = (kn/\varepsilon)^{O(1/\varepsilon^3)}.$$

Total Runtime: The number of DP cells is $N_{\text{rect}} \times N_{\text{sets}}$, which is dominated by $(kn/\varepsilon)^{O(1/\varepsilon^3)}$. For each cell, we compute the cost of candidate solutions:

- The *trivial operation* takes constant time relative to the size of $\mathcal{S}$.
- The *line operation* iterates over all vertical discrete lines intersecting R. There are at most $O(n^2/\varepsilon^3)$ such lines. For each line, we solve a Rectangle Stabbing instance on bounding boxes, which takes polynomial time.
- The *add operation* iterates over all possible sets $\mathcal{S}'$ such that $|\mathcal{S}| + |\mathcal{S}'| \leq 3\varepsilon^{-3}$. The number of such sets is bounded similarly to N_{sets}, i.e., $(kn/\varepsilon)^{O(1/\varepsilon^3)}$.

The computational cost per cell is dominated by the number of choices in the Add Operation. Therefore, the total running time is the number of cells multiplied by the transitions per cell:

$$(kn/\varepsilon)^{O(1/\varepsilon^3)} \times (kn/\varepsilon)^{O(1/\varepsilon^3)} = (kn/\varepsilon)^{O(1/\varepsilon^3)}. \qquad \square$$

To prove the approximation ratio, we construct a specific solution (a "DP-decision tree") that fits within our DP structure and has cost close to optimal. This construction relies on a hierarchical grid shifted by a random offset r.

Hierarchical Grid and Alignment. We use a hierarchical grid shifted by a random offset $r \in \{0, \varepsilon^d, 2\varepsilon^d, \dots, \alpha\varepsilon^{-2}\}$. For each level $j \in \mathbb{N}_0$, we define a vertical grid line $\{r + t \cdot \alpha\varepsilon^{j-2}\} \times \mathbb{R}$ for each $t \in \mathbb{Z}$. We say a segment $\ell \in \mathsf{OPT}$ is of *level* j if its length is in $(\alpha\varepsilon^j, \alpha\varepsilon^{j-1}]$. A segment of level j is *well-aligned* if its endpoints lie on grid lines of level $j+3$ and have discrete y-coordinates.

Lemma 7.11 (Alignment)**.** *By losing a factor of $1 + O(\varepsilon)$ in the approximation ratio, we can assume that each line segment $\ell \in \mathsf{OPT}$ is well-aligned.*

Proof. For a segment $\ell_i \in \mathsf{OPT}$ with length in $(\alpha\varepsilon^j, \alpha\varepsilon^{j-1}]$, we extend it by at most $\alpha\varepsilon^{j+1}$ on either side to align with grid lines of level $j+3$. The new length is bounded:

$$|\ell_i'| \le |\ell_i| + 2\alpha\varepsilon^{j+1} \le |\ell_i| \left(1 + \frac{2\alpha\varepsilon^{j+1}}{\alpha\varepsilon^j}\right) = |\ell_i|(1 + 2\varepsilon).$$

Summing over all segments yields a total cost of $(1 + 2\varepsilon)\mathsf{OPT}$. □

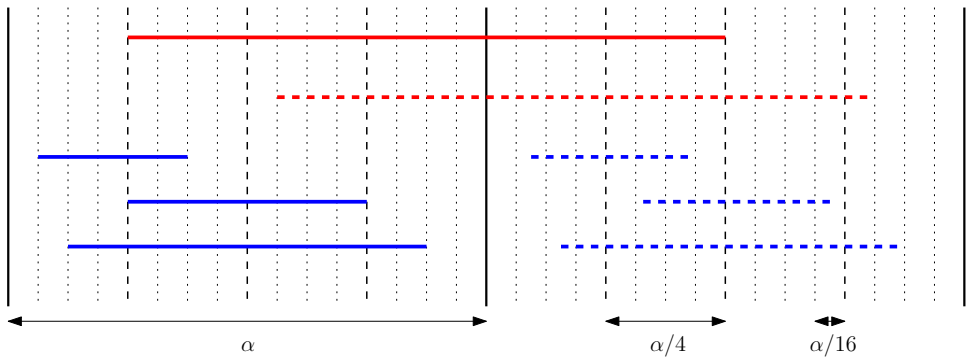


Figure 7.6: For $\varepsilon = 1/4$, the figure shows vertical grid lines of level $j = 2, 3, 4$ (solid, dashed and dotted lines respectively). Horizontal segments of level $j = 0, 1$ (red and blue respectively) are shown where the solid segments are well-aligned, and the dashed ones are not.

Canonical Sequence of Operations. To show that the DP finds a valid solution, we simulate the execution using a canonical sequence based on the hierarchical grid.

1. **Level 0 Operations:** We first perform *Line Operations* on all vertical grid lines of level $j = 0$. This partitions the instance into strips of width $\alpha\varepsilon^{-2}$. Within these strips, we perform *Add Operations* to introduce horizontal segments that partition the strips further

(picking every ε^{-3}-th segment from OPT sorted by y-coordinate). This triggers *Trivial Operations*. Finally, we perform *Add Operations* for all remaining segments of Level 0 in OPT.

2. **Level j Operations:** Proceeding for $j = 1, 2, \ldots$:
   - Perform *Line Operations* on vertical grid lines of level j.
   - Perform any valid *Trivial Operations*.
   - Perform *Add Operations* to partition vertical strips into smaller subproblems.
   - Perform *Add Operations* for all segments from OPT of level j.

Lemma 7.12 (Validity)**.** *The canonical sequence of operations ensures that for every visited subproblem $(R, \mathcal{S})$, the size of the set $|\mathcal{S}|$ never exceeds $3\varepsilon^{-3}$.*

Proof. Consider a subproblem $(R, \mathcal{S})$. Let the last segment added to $\mathcal{S}$ be of level j. An Add Operation of level j is strictly preceded by Line Operations of level j. By the alignment property (Lemma 7.11), any segment of level $j-3$ or higher (larger length) that was previously in $\mathcal{S}$ would have been removed by a Trivial Operation because it would fully span the grid cells generated by level j lines (as its endpoints lie on level $(j-3)+3 = j$ lines). Therefore, $\mathcal{S}$ only contains segments from levels $j, j-1$, and $j-2$. By construction, we add at most ε^{-3} segments per level. Thus, $|\mathcal{S}| \leq 3\varepsilon^{-3}$. □

Finally, we bound the cost of the Line Operations, which use the approximation for bounding boxes.

Lemma 7.13 (Error Bound)**.** *There exists a discrete offset r such that the DP solution has cost $(1 + O(\varepsilon))\mathsf{OPT}$.*

Proof. In the described sequence of operations, some add operations are applied on segments from OPT (or their parts), and hence the cost across all such add operations is at most cost of OPT. Similarly, all trivial operations are applied on segments that were 'added' before, and hence their cost is also already accounted for. So we are left with analyzing the cost of stabbing the rectangles which are intersected by the lines along which we apply the line operations, and the add operations which are not applied on a segment from OPT (that is, the ones that are used to partition a vertical strip). We claim that for a discretized random offset $r \in \{0, \varepsilon^d, 2\varepsilon^d, \ldots, \alpha\varepsilon^{-2}\}$, the expected cost is $O(\varepsilon \cdot \mathsf{OPT})$, which would give us the required result.

Let us first consider any add operation of level j that is applied to a horizontal line ℓ (that is not in OPT). We do such an operation only after accounting for ε^{-3} segments from OPT of level j, i.e., segments of cost at least $\varepsilon^{-3} \cdot \alpha\varepsilon^j = \alpha\varepsilon^{j-3}$. Since a segment of width $\alpha\varepsilon^{j-2}$ (width of the strip) is sufficient to stab all the k-shapes stabbed by ℓ, we see that this horizontal segment only takes ε times the cost of the segments in OPT that we have already accounted for. We charge the cost of adding this segment to the solution, to the ε^{-3} segments from OPT that were counted before adding it. Since any segment in OPT gets charged only once, we can infer that the cost of such add operations is at most $2\varepsilon \cdot \mathsf{OPT}$.

Now, let us consider the *line operations* applied to vertical grid lines. Consider a grid line ℓ of level j. We wish to bound the cost of stabbing all the rectangles in $\mathcal{R}_\ell$ intersected by grid lines, over all levels j. Let $\mathsf{OPT}_{\mathcal{R}_\ell}$ be the set of minimum cost that stabs all rectangles in $\mathcal{R}_\ell$, and let $\mathsf{OPT}_{\mathcal{K}_\ell}$ be the set of segments from OPT that stab the corresponding k-shapes in $\mathcal{K}_\ell$. From Lemma 7.2, we know that $\mathsf{OPT}_{\mathcal{R}_\ell} \leq k/\delta \cdot \mathsf{OPT}_{\mathcal{K}_\ell}$. So instead of bounding the cost of stabbing $\mathcal{R}_\ell$, we instead focus on bounding the cost of stabbing $\mathcal{K}_\ell$.

A horizontal line segment $\ell' \in \mathsf{OPT}$ of some level $j' \geq j$ stabs a k-shape in $\mathcal{K}_\ell$ only if the distance between ℓ and ℓ' is at most $\alpha\varepsilon^{j-1}$. So we need to bound the cost, over all levels j, of line segments of level j in OPT (call this set OPT_j) intersected or close to grid lines of level j. For a horizontal segment $\ell' \in \mathsf{OPT}_j$, let $I_{\ell'}$ be the indicator variable representing the event that ℓ' is within distance $\alpha\varepsilon^{j-1}$ of ℓ. Now, since any two consecutive grid lines of level j are separated by $\alpha\varepsilon^{j-2}$, there are $(\alpha\varepsilon^{j-2})/\varepsilon^d$ possible shifts for these grid lines due to our offset, and each of these shifts has the same probability. Similarly, $(2\alpha\varepsilon^{j-1}/\varepsilon^d) + 1$ of these offsets would allow ℓ' to be within $\alpha\varepsilon^{j-1}$ distance of ℓ. So, if we take a random discrete offset $r \in \{0, \varepsilon^d, 2\varepsilon^d, \ldots, \varepsilon^{-2}\}$, we have that

$$\mathbb{E}[I_{\ell'}] \leq \frac{(2\alpha\varepsilon^{j-1}/\varepsilon^d) + 1}{\alpha\varepsilon^{j-2}/\varepsilon^d} = 2\varepsilon + \frac{\varepsilon^{2+(d-j)}}{\alpha} \leq 3\varepsilon. \qquad \text{(since } j \in \{0, 1, \ldots, d-1\}\text{)}$$

With the expectation computed above, we can upper bound the expected cost of segments in $\mathsf{OPT}_{\mathcal{K}_\ell}$ as:

$$\begin{aligned}
\mathbb{E}\left[\sum_j \sum_{\ell \in \mathsf{OPT}_j} I_\ell \cdot |\ell|\right] &= \sum_j \sum_{\ell \in \mathsf{OPT}_j} \mathbb{E}\left[I_\ell \cdot |\ell|\right] \\
&= \sum_j \sum_{\ell \in \mathsf{OPT}_j} |\ell| \cdot \mathbb{E}[I_\ell] \\
&\leq \sum_j \sum_{\ell \in \mathsf{OPT}_j} |\ell| \cdot (3\varepsilon)
\end{aligned}$$

$$= 3\varepsilon \cdot \mathsf{OPT}$$

Since $\mathsf{OPT}_{\mathcal{R}_\ell} \leq k/\delta \cdot \mathsf{OPT}_{\mathcal{K}_\ell}$, we get that $\mathbb{E}[\mathsf{OPT}_{\mathcal{R}_\ell}] \leq 3k\varepsilon/\delta \cdot \mathsf{OPT}$. And since we use a PTAS (which, let's say has an approximation factor of $(1+\varepsilon')$) for computing the cost of $\mathsf{OPT}_{\mathcal{R}_\ell}$, the solution returned by our algorithm takes an additional cost of $\frac{3k\varepsilon(1+\varepsilon')}{\delta} \cdot \mathsf{OPT}$. □

Combining these lemmas, we arrive at the main result of this section.

Theorem 7.3. *For any constants $k \in \mathbb{N}$ and $\delta \in (0, 1]$, there is a PTAS for the k-Stabbing problem restricted to instances where the widths of rectangles within each k-shape differ by at most a factor of $1/\delta$. The running time is $(kn/\varepsilon)^{O(1/\varepsilon^3)}$.*

7.6 Lower Bound for the General Case

In the previous sections, we established that k-Stabbing admits efficient approximation schemes when the input shapes satisfy specific geometric constraints. In this section, we study the general case where these constraints are lifted.

We show that the problem becomes significantly harder. Specifically, we prove that for $k \geq 3$, the problem is APX-hard. Furthermore, for arbitrary rectilinear polygons (where k is part of the input), the problem is as hard as general Set Cover, implying an $\Omega(\log n)$ lower bound on the approximation ratio.

7.6.1 APX-Hardness for 3-Shapes

In contrast to the cases studied in Section 7.5, we show that the general case of the stabbing problem does *not* admit a $(1+\varepsilon)$-approximation algorithm, even for only slightly more general types of instances. Formally, we prove that stabbing is APX-hard, already if each input polygon is a 3-shape.

Theorem 7.4. *The k-Stabbing problem is APX-hard for $k \geq 3$.*

Before proving this hardness, it is worth noting why the case of $k = 2$ (e.g., L-shapes) remains tractable. Any 2-shape satisfies the hourglass property; hence, stabbing is unlikely to be APX-hard for this class of objects since we have a QPTAS for this case [96].

Proposition 7.3. *Every 2-shape satisfies the hourglass condition.*

Proof. Recall Definition 7.2. If $k = 2$, the sequence of widths consists of only two elements. There is no index $i \in \{2, \ldots, k-1\}$ that can violate the monotonicity required on either side of a minimum. Thus, the hourglass property is trivially satisfied. □

We now prove Theorem 7.4 via an L-reduction from the Vertex Cover problem on cubic graphs to 3-Stabbing. Note that it is NP-hard to approximate Vertex Cover with a strictly better approximation factor than $\sqrt{2}$ [97]. We will obtain the same lower bound for stabbing.

Reduction Construction. Consider a given instance $G = (V, E)$ of Vertex Cover. Recall that in Vertex Cover, we are required to select a subset $S \subseteq V$ of smallest size such that for each $e \in E$ one of its endpoints is in S. We construct an instance of 3-Stabbing corresponding to G as follows. Assume that $V = \{v_1, \ldots, v_n\}$.

1. **Vertex Gadgets:** For each $v_i \in V$ construct a 1×1 square s_i, such that they are all arranged in a column separated by 1 unit distance each (see Figure 7.7). Formally, for each $v_i \in V$ the top-left corner of the square s_i has the coordinates $(0, 2i - 1)$. Note that the squares $s_1, \ldots, s_n$ do *not* belong to our input shapes, but they only help us to construct the latter.

2. **Edge Gadgets:** For each edge $\{v_i, v_j\} \in E$ (assume $i < j$), we define a 3-shape $r_{i,j}$ as the union of the three rectangles:

   - The square s_i.
   - The wide rectangle $[0, n + 1] \times [2i - 1, 2j - 2]$.
   - The square s_j.

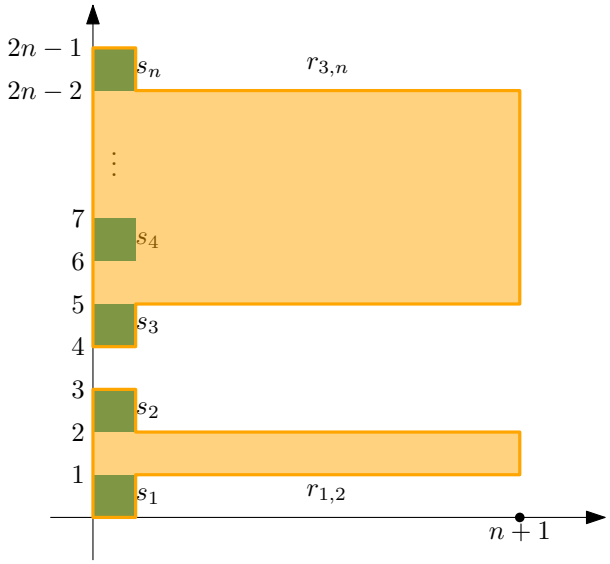


Figure 7.7: Construction of k-stabbing instance in our reduction from Vertex Cover. The squares s_i correspond to vertices, and the 3-shapes correspond to edges connecting them.

Note that none of the resulting shapes satisfies the hourglass property, and also for none of them are the widths of the three constituent rectangles within a constant factor. The width of the widest rectangle of each constructed 3-shape is greater than n, but there is always a feasible solution with cost n that simply stabs the square s_i for each vertex $v_i \in V$. Thus, in any given solution to the stabbing instance, we can assume w.l.o.g. that no 3-shape is stabbed across its widest rectangle.

Lemma 7.14. *The* Vertex Cover *instance has a solution of size γ if and only if the corresponding 3-*Stabbing *instance has a solution of cost γ.*

Proof. ($\Rightarrow$) Let $S \subseteq V$ be a vertex cover of size γ. We construct a set of segments by selecting, for each $v_i \in S$, the top edge of square s_i (a segment of length 1). The total cost is γ. Consider any 3-shape $r_{i,j}$ corresponding to edge $\{v_i, v_j\}$. Since S covers the edge, either $v_i \in S$ or $v_j \in S$. Consequently, either s_i is stabbed or s_j is stabbed. In either case, $r_{i,j}$ is stabbed.

($\Leftarrow$) Let $\mathcal{S}$ be a solution to the stabbing instance with cost γ. First, we normalize $\mathcal{S}$.

- If any segment has length $\geq n+1$, it costs more than the trivial solution of stabbing all squares s_i (cost n). We can safely assume $\gamma \leq n$. Thus, no segments stab the wide connectors.
- If a segment has length $\ell \in (1, n+1)$, it cannot stab more than one shape efficiently (due to the vertical separation). We can shorten it to length 1 without losing coverage of the relevant square s_i.
- Segments of length < 1 cannot stab any shape (width of s_i is 1).

Thus, we can assume $\mathcal{S}$ consists only of segments of length 1, each stabbing some square s_i. Let $C = \{v_i \mid s_i \text{ is stabbed}\}$. Since every 3-shape $r_{i,j}$ is stabbed, and the wide connector is not stabbed, at least one of s_i or s_j must be stabbed. This implies $v_i \in C$ or $v_j \in C$ for every edge. Thus, C is a vertex cover of size at most γ. □

This linear reduction preserves the approximation hardness. Since Vertex Cover is APX-hard, 3-Stabbing is APX-hard.

7.6.2 Logarithmic Hardness for General k-Shapes

We further show that k-Stabbing for arbitrary k-shapes cannot be approximated with a ratio of $o(\log n)$, unless $\mathsf{P} = \mathsf{NP}$. In fact, we show that the problem is as hard as general instances of Set Cover.

Theorem 7.5. *The k-Stabbing problem does not admit an $o(\log n)$-approximation algorithm, unless* $\mathsf{P} = \mathsf{NP}$.

We reduce from the Hitting Set problem, which is known to be equivalent to Set Cover. In Hitting Set, we are given a set of elements $P = \{p_1, p_2, \ldots, p_n\}$, and a family of their subsets $\mathcal{F}$. The aim is to compute a minimum-size subset $H \subseteq P$ such that every set in $\mathcal{F}$ contains a point in H.

Reduction Construction. Our construction here is similar to the reduction from Vertex Cover above.

- For each element $p_i \in P$, we construct a unit square s_i with its top-left corner being located at $(0, 2i - 1)$.
- For each set $S_i = \{p_{i_1}, p_{i_2}, \ldots, p_{i_f}\} \in \mathcal{F}$ we construct a k-shape (where $k = f$) which is a stack of the rectangles corresponding to the elements in the set, connected by wide bridges. Specifically, the shape is the union of:

$$s_{i_1}, \quad [0, n+1] \times [2i_1 - 1, 2i_2 - 2], \quad s_{i_2}, \quad \ldots, \quad [0, n+1] \times [2i_{f-1} - 1, 2i_f - 2], \quad s_{i_f}.$$

See Figure 7.8 for an illustration.

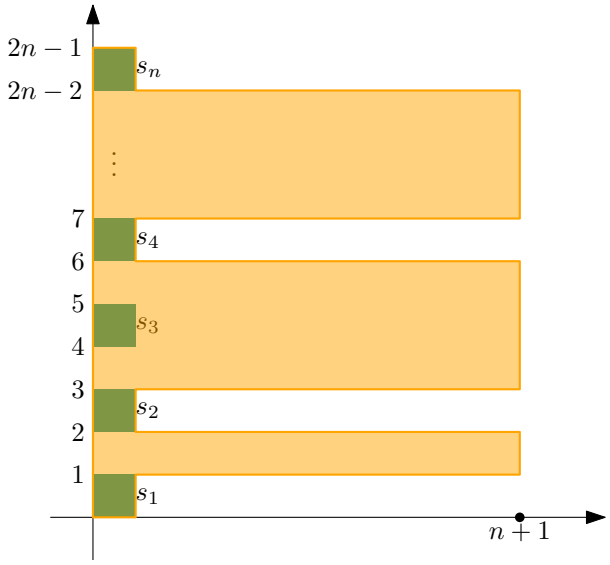


Figure 7.8: The k-shape constructed for set $S = \{v_1, v_2, v_4, v_n\} \in \mathcal{F}$ in the Hitting Set reduction.

Again, these constructed k-shapes satisfy neither the hourglass property nor the condition that the widths of their rectangles are in a bounded range. As before, for each given solution,

we can assume w.l.o.g. that no shape is stabbed across one of its wide rectangles (i.e., of width $n+1$).

Lemma 7.15. *For each $\gamma \in \mathbb{N}$, the given instance of* Hitting Set *has a solution of size γ if and only if the constructed k-*Stabbing *instance has a solution of cost γ.*

Proof. ($\Rightarrow$) Given a solution $H = \{v_1, v_2, \ldots, v_\gamma\}$ to the Hitting Set instance, construct a solution to the stabbing instance as follows: for each $v_i \in H$, stab the corresponding s_i along its top edge by a segment of length one. Clearly the cost of this set of segments is γ. Now we notice that every k-shape r_{S_i} corresponds to a set $S_i \in \mathcal{F}$. Since this set has been hit by a vertex, say $v_i \in H$, r_{S_i} is also stabbed by the segment that stabs s_i. We know that every set of the family $\mathcal{F}$ is covered by some element in H, and hence every k-shape in the instance is also stabbed in the solution we constructed.

($\Leftarrow$) Similar to the argument in proof of Lemma 7.14, we can modify the solution without increasing its cost so that every segment is of length one. Now we construct a Hitting Set solution by picking the elements v_i that correspond to any square s_i that has been stabbed by the given (or modified) stabbing solution. Note that every k-shape is stabbed by the given solution, and hence an element from each set in $\mathcal{F}$ has been picked by us. This shows that the selected set is in fact a valid hitting set, and is of size at most γ. □

Since Set Cover cannot be approximated better than $(1-o(1))\ln n$, the same lower bound applies to k-Stabbing.

Chapter 8

Lower Bounds for Batched MaxRS

8.1 Introduction

In the previous chapters of this thesis, we focused on *minimization* problems: selecting the smallest number of links to augment connectivity (Chapters 4 and 5) or the minimum weight set of geometric objects to cover a set of points (Chapters 6 and 7). In this final technical chapter, we shift our perspective to the dual maximization problem. Instead of covering all requirements with minimum cost, we consider the setting where we have a fixed budget (a single geometric shape) and wish to maximize the "profit" (the weight of the covered points).

This brings us to the Maximum Range Sum (MaxRS) problem. The input consists of a set P of n weighted points in $\mathbb{R}^d$ and a geometric range shape Q (typically an axis-aligned rectangle or a Euclidean ball). The goal is to determine a placement of Q in $\mathbb{R}^d$ that maximizes the total weight of the points in $P \cap Q$. In the unweighted setting, this is equivalent to finding a placement that covers the maximum number of points. As noted in the preliminaries (Section 2.4.3), this problem is intimately tied to Geometric Set Cover via duality: finding a location for a shape Q to cover the maximum number of points is equivalent to finding a point p in the dual arrangement that is covered by the maximum number of dual shapes.

MaxRS is a fundamental primitive in computational geometry and spatial databases, with applications ranging from finding "hotspots" in trajectory data [136] to facility location problems. While the problem is well-understood for single queries, this chapter investigates the complexity of processing massive batches of queries.

8.1.1 The Batched MaxRS Problem

In the Batched MaxRS problem, in addition to the weighted point set P, we are given a set of m geometric ranges $\mathcal{Q} = \{Q_1, Q_2, \ldots, Q_m\}$. The objective is to solve the MaxRS problem

for *each* $Q_i \in \mathcal{Q}$. For example, in the 1-dimensional setting, we are given m interval lengths, and for each length, we must find a placement that maximizes the weight of covered points.

Studying the batched complexity is crucial not only for high-throughput applications but also for understanding the theoretical limits of the single-query version. Typically, in geometric optimization, we aim to preprocess a dataset of size n into a data structure that can answer queries in $O(\text{polylog}n)$ time. If such a data structure existed for MaxRS, we could solve the Batched MaxRS problem in $\tilde{O}(n+m)$ time by simply querying the structure m times. Conversely, establishing a lower bound of $\Omega(mn)$ for the batched problem provides strong evidence that no such efficient data structure exists for weighted points, effectively ruling out sublinear time queries.

8.1.2 Our Results and Techniques

We establish conditional lower bounds for Batched MaxRS based on fine-grained complexity assumptions. Specifically, we relate the hardness of geometric optimization to the hardness of the $(\min,+)$-Convolution problem.

- Batched MaxRS Lower Bound: We prove that for a set of n weighted points and m query interval lengths in $\mathbb{R}^1$, any algorithm solving Batched MaxRS requires $\Omega(mn)$ time, conditioned on the $O(n^{2-\epsilon})$ hardness of the $(\min,+)$-Convolution problem. This suggests that the naive approach of solving each query independently is essentially optimal for weighted instances.

- Batched Smallest k-Enclosing Interval: We also consider a related problem: finding the smallest interval containing at least k points. In the batched version, we must solve this for *all* $k \in \{1,\ldots,n\}$. We establish a similar conditional lower bound of $\Omega(n^2)$ for this problem.

These results are established via a chain of reductions from $(\min,+)$-Convolution to a "Positive" variant of the problem, and finally to geometric placement problems.

8.1.3 Related Work

The MaxRS problem has a rich history in both computational geometry and spatial database communities. The problem was first initiated by Imai, and Asano [89], who presented an optimal $O(n \log n)$ time algorithm for the 2D rectangle case (later refined by Nandy and Bhattacharya [116]). The extension to d dimensions was handled by Chan [35]. For circular ranges (disks), Chazelle and Lee [41] presented an $O(n^2)$ algorithm, which was later shown to be optimal assuming 3SUM-hardness [10].

In the database community, MaxRS has been extensively studied under various models, including I/O-efficient algorithms [47], spatial data streams [8], and probabilistic data [105]. Parallel efforts have focused on designing $(1-\epsilon)$-approximation algorithms using random sampling [5, 125] and ϵ-nets [58].

Dynamic and Colored Variants. The lower bounds presented in this chapter are part of a broader study on MaxRS [84], which also investigates algorithmic upper bounds for two other variants:

- **Dynamic MaxRS:** Where points are inserted or deleted, and the optimal hotspot must be maintained efficiently. We obtained a randomized $(\frac{1}{2}-\epsilon)$-approximation with polylogarithmic update time for d-balls.
- **Colored MaxRS:** Where points are colored, and the goal is to maximize the number of *distinct* colors covered. We developed $(1-\epsilon)$-approximation algorithms for colored disks in $\mathbb{R}^2$ with near-linear running time.

While these algorithmic results are significant, this thesis focuses exclusively on the lower bounds for the batched setting. We mention these variants here to provide a complete picture of the research project.

Organization. The remainder of this chapter is organized as follows. Section 8.2 outlines the necessary reductions between variants of the convolution problem that serve as the foundation for our hardness proofs. Section 8.3 presents the construction and proof of the $\Omega(mn)$ lower bound for Batched MaxRS. Finally, Section 8.4 details the lower bound for the Batched Smallest k-Enclosing Interval problem.

8.2 Reductions for Convolution Variants

We first establish the hardness of specific variants of the convolution problem. This is done to highlight the geometric construction and reduction in the next section, and single out the algebraic details here. The reduction follows the path:

$(\min,+)$-Convolution $\rightarrow$ $(\min,+,M)$-Convolution $\rightarrow$ Positive $(\max,+,M)$-Convolution.

8.2.1 From (min,+)-Convolution to (min,+,M)-Convolution

The $(\min,+,M)$-Convolution problem. Given integer sequences $D=(D_0,D_1,\ldots,D_{n-1})$, $E=(E_0,E_1,\ldots,E_{n-1})$, and a set of m distinct indices $M=\{k_0,k_1,\ldots,k_{m-1}\}\subseteq\{0,1,\ldots,n-$

1}, the $(\min, +, M)$-Convolution problem asks to compute the sequence $F_M = (F_{k_s})_{s=0}^{m-1}$ where for each $k_s \in M$,

$$F_{k_s} = \min_{i+j=k_s} (D_i + E_j).$$

Reduction: Let input sequences A, B be given for an instance of $(\min, +)$-Convolution, where we must compute $C_k = \min_{i+j=k}(A_i + B_j)$ for all $k \in K_{\text{all}} = \{0, 1, \ldots, n-1\}$. We solve this using an oracle for the $(\min, +, M)$-Convolution problem.

We partition the set K_{all} into $p = \lceil n/m \rceil$ disjoint subsets $M_1, M_2, \ldots, M_p$. Each subset M_s contains at most m indices from K_{all}. For instance, we can define $M_s = \{k \in K_{\text{all}} \mid (s-1)m \leq k < sm\}$. For each $s \in \{1, \ldots, p\}$, we invoke the $(\min, +, M)$-Convolution oracle with inputs A, B, and M_s. The union of the results from these p calls yields all the required C_k values.

The partitioning of the indices and the combination of the results take $O(n)$ time. Therefore, an $o(nm)$ time algorithm for $(\min, +, M)$-Convolution would imply an $O(n) + o(nm) \times (n/m) = o(n^2)$ time algorithm for the original problem. Since $(\min, +)$-Convolution is conjectured to require $\Omega(n^2)$ time, the $(\min, +, M)$-Convolution problem has a conditional lower bound of $\Omega(nm)$.

8.2.2 From (min,+,M)-Convolution to (max,+,M)-Convolution

The $(\max, +, M)$-Convolution problem. Given integer sequences A, B and indices M as above, the $(\max, +, M)$-Convolution problem computes the sequence $C_M = (C_{k_s})_{s=0}^{m-1}$ where for each $k_s \in M$,

$$C_{k_s} = \max_{i+j=k_s} (A_i + B_j).$$

Reduction: To reduce $(\min, +, M)$-Convolution to its maximization counterpart, we negate the inputs. Given sequences D, E and target indices M, we construct A' and B' such that $A'_i = -D_i$ and $B'_j = -E_j$. We then invoke an oracle for $(\max, +, M)$-Convolution. The output C'_k satisfies:

$$C'_k = \max_{i+j=k}(A'_i + B'_j) = \max_{i+j=k}(-D_i - E_j) = -\left(\min_{i+j=k}(D_i + E_j)\right) = -C_k.$$

Thus, we recover the desired result via $C_k = -C'_k$. This linear-time reduction preserves the $\Omega(nm)$ hardness.

8.2.3 From (max,+,M)-Convolution to Positive (max,+,M)-Convolution

The Positive $(\max, +, M)$-Convolution problem. Given integer sequences A, B and indices M as above, additionally satisfying that A and B are non-negative, the Positive $(\max, +, M)$-Convolution problem computes the sequence $C_M = (C_{k_s})_{s=0}^{m-1}$ where for each $k_s \in M$,

$$C_{k_s} = \max_{i+j=k_s} (A_i + B_j).$$

We note here that though the problem admits non-negative integer sequences, we call the problem the *Positive* $(\max, +, M)$-Convolution problem for ease of exposition.

Reduction: Given an instance of $(\max, +, M)$-Convolution with potentially negative values in A and B, we shift the values to ensure non-negativity. Let $\Delta = \min((\min_i A_i), (\min_j B_j))$. If $\Delta < 0$, we define new sequences $A'_i = A_i - \Delta$ and $B'_j = B_j - \Delta$, which are guaranteed to be non-negative.

Invoking the Positive oracle on A', B' yields results C'_k. For each $k \in M$, the oracle computes:

$$C'_k = \max_{i+j=k}(A'_i + B'_j) = \max_{i+j=k}(A_i + B_j) - 2\Delta.$$

We recover the original solution as $C_k = C'_k + 2\Delta$. Computing Δ, shifting the sequences, and recovering the answers takes $O(n + m)$ time. This linear-time reduction implies that Positive $(\max, +, M)$-Convolution also requires $\Omega(nm)$ time.

8.3 Lower Bound for Batched MaxRS

We now present the reduction from Positive $(\max, +, M)$-Convolution to Batched MaxRS, establishing the $\Omega(mn)$ lower bound. The full series of reductions is illustrated in Figure 8.1.

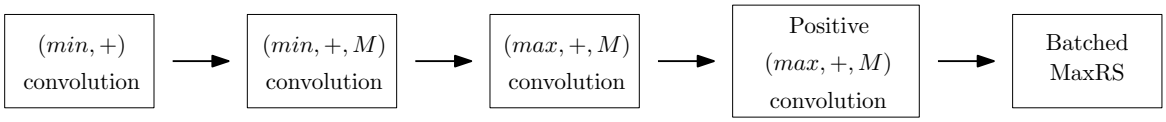


Figure 8.1: Series of reductions leading to conditional hardness of Batched MaxRS.

Problem Instance Construction: Let the Positive $(\max, +, M)$-Convolution instance be defined by non-negative integer sequences A, B and indices $M = \{k_0, \ldots, k_{m-1}\}$. We aim to compute $C_{k_s} = \max_{i+j=k_s}(A_i + B_j)$. We construct a Batched MaxRS instance with weighted points $p = (x_p, w_p)$ located at x-coordinate x_p, and with weight w_p, as follows:

1. **Point set construction:** We create a set P of $4n$ weighted points.
   - For each A_i where $i \in \{0, \ldots, n-1\}$, create a point $p_i^A = (i, A_i)$, and its "guard" point $p_i'^A = (i - 0.5, -A_i)$.
   - Let $X_{\text{offset}} = 2n - 1$. For each B_j where $j \in \{0, \ldots, n-1\}$, create a point $p_j^B = (X_{\text{offset}} - j, B_j)$ and its guard point $p_j'^B = (X_{\text{offset}} - j + 0.5, -B_j)$.

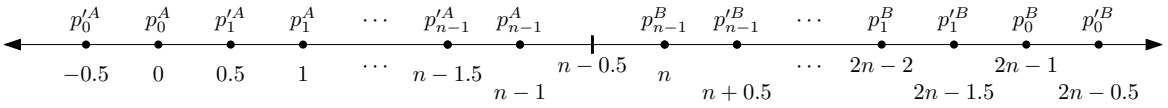


Figure 8.2: Position of constructed (weighted) points on real line.

   Observe that the points corresponding to A lie in the interval $[0, n-1]$ and those corresponding to B lie in the interval $[n, 2n-1]$, ensuring the two sets are spatially disjoint.

2. **Query interval lengths:** For each index $k_s \in M$, we define a query interval length $L_s = X_{\text{offset}} - k_s$. Since $0 \le k_s \le n-1$, we have $L_s \ge n$.

An oracle for Batched MaxRS takes the point set P and the interval lengths $\{L_s\}_{k_s \in M}$, returning the maximum weight W_s^* for each length.

Lemma 8.1. *For each $k_s \in M$, the maximum weight W_s^* found by the Batched MaxRS oracle for an interval of length $L_s = X_{offset} - k_s$ satisfies $W_s^* = \max_{i+j=k_s}(A_i + B_j)$.*

To prove this, let $w(I)$ denote the total weight covered by an interval I. We first characterize the weight of intervals aligned to the integer grid.

Claim 8.1. *The weight covered by an interval $I_{i,j} = [i, X_{offset} - j]$ for some $i, j \in \{0, \ldots, n-1\}$ is $w(I_{i,j}) = A_i + B_j$.*

Proof. The interval $I_{i,j}$ starts at coordinate i, including p_i^A but excluding its guard $p_i'^A$ (at $i - 0.5$). Similarly, it ends at $X_{\text{offset}} - j$, including p_j^B but excluding its guard $p_j'^B$ (at $X_{\text{offset}} - j + 0.5$). For all indices $i < i' \le n-1$, $I_{i,j}$ covers both $p_{i'}^A$ and $p_{i'}'^A$, whose weights sum to zero. Similarly, for all $0 \le j' < j$, it covers both $p_{j'}^B$ and $p_{j'}'^B$. Thus, the only non-cancelled weights are A_i and B_j. □

Proof of Lemma 8.1. Since A, B are non-negative, the target value $C_{k_s}^* \ge 0$. By Claim 8.1, placing the interval at i such that $i + j = k_s$ yields a weight of $A_i + B_j$. Thus, the maximum weight $W_s^* \ge C_{k_s}^*$.

We now show that any optimal interval $I^* = [x^*, x^* + L_s]$ must align with the integer grid construction:

1. If $x^* + L_s < n - 0.5$ (I^* covers no B-points): The interval covers points from $p_0'^A$ up to some p_a^A or $p_a'^A$. This results in either zero weight (if paired) or negative weight (if the last guard $p_a'^A$ is included without p_a^A).

2. If $x^* \geq n - 0.5$ (I^* covers no A-points): Similarly, the interval covers a sequence of B-points ending at $p_0'^B$. This results in either zero or negative weight.

3. If $x^* < -0.5$ (I^* covers all A-points): Every pair $(p_a^A, p_a'^A)$ is covered, summing to zero. At most one B-point p_b^B might be unpaired (i.e., p_b^B is covered, but $p_b'^B$ is not), so the total weight is either 0 or B_b.

4. Similarly, if $x^* + L_s > X_{\text{offset}} + 0.5$ (I^* covers all B-points), at most one point p_a^A remains unpaired, and the total weight is either 0 or A_a.

5. **The interesting case:** I^* partially covers both sets. Consider an integer-aligned interval $I_{i,j}$ with length L_s (implying $i + j = k_s$). Shifting this interval left by $\delta \in (0, 0.5]$ causes it to lose p_j^B (positive weight) and potentially gain $p_i'^A$ (negative weight). Shifting right by δ causes it to lose p_i^A and potentially gain $p_j'^B$. Both deviations strictly reduce the weight.

Thus, an optimal interval must be of the form I_{i^*,j^*} where $i^* + j^* = k_s$, and the oracle returns exactly $\max(A_{i^*} + B_{j^*})$. □

Constructing the point set P and the query interval lengths takes $O(n + m)$ time. Because this geometric reduction takes linear time, it confirms that solving Batched MaxRS is at least as hard as Positive $(\max, +, M)$-Convolution, proving the following theorem.

Theorem 8.1. *Assuming that* $(\min, +)$*-Convolution on sequences of length* n *requires* $\Omega(n^2)$ *time, the Batched MaxRS problem on* n *points and* m *query interval lengths requires* $\Omega(mn)$ *time.*

8.4 Lower Bound for Batched Smallest k-Enclosing Interval

We now address the Batched Smallest k-Enclosing Interval (BSEI) problem. Given n points on the real line, we wish to find the length of the smallest interval covering at least k points, for all $k \in \{1, \ldots, n\}$. We establish a quadratic lower bound via reduction from Monotone $(\min, +)$-Convolution.

8.4.1 Reduction from (min,+)-Convolution to Monotone (min,+)-Convolution

Definition 8.1 (Monotone (min, +)-Convolution)**.** *Given integer sequences* $D = (D_0, \ldots, D_{n-1})$ *and* $E = (E_0, \ldots, E_{n-1})$ *such that* $D_0 > D_1 > \cdots > D_{n-1}$ *and* $E_0 > E_1 > \cdots > E_{n-1}$*, for each* k *compute*

$$F_k = \min_{i+j=k}(D_i + E_j).$$

Given an arbitrary instance A, B of (min, +)-Convolution, we can convert it to a monotone instance by adding a large linear term. Let $\Delta = 1 + \max_i \max((A_i - A_{i-1}), (B_i - B_{i-1}))$. We construct $D_i = A_i - i \cdot \Delta$ and $E_i = B_i - i \cdot \Delta$. These new sequences are strictly decreasing. Solving the monotone instance yields F_k,

$$F_k = \min_{i+j=k}(D_i + E_j) = \min_{i+j=k}((A_i - i \cdot \Delta) + (B_j - j \cdot \Delta)) = C_k - k \cdot \Delta,$$

from which we recover the original solution as $C_k = F_k + k \cdot \Delta$. Computing Δ, constructing the sequences D and E, and recovering the values of C_k all take $O(n)$ time. Therefore, this linear-time reduction preserves the $\Omega(n^2)$ hardness.

8.4.2 Reduction from Monotone (min,+)-Convolution to BSEI

Given monotone sequences D, E, we construct a point set P of size $2n$. For each $i \in \{0, \ldots, n-1\}$, we define points:

$$P_i = -D_i + (D_{n-1} - 1)$$

$$P_{n+i} = E_{(n-1)-i} + (1 - E_{n-1})$$

Note that the offset terms (namely $(D_{n-1} - 1)$ and $(1 - E_{n-1})$) ensure $P_i < 0$ and $P_{n+i} > 0$. See Figure 8.3 for an illustration.

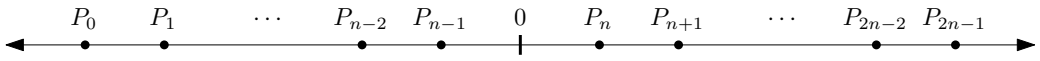


Figure 8.3: Values of newly constructed sequence as points on the real line.

An algorithm for BSEI returns a sequence $G_1, \ldots, G_{2n}$, where G_k for $k \in \{1, 2, \ldots, 2n\}$ is the minimum length of an interval covering k points. Consider the interval between the i-th point and the $(\ell+i)$-th point in our constructed set. The distance is $P_{\ell+i} - P_i$, and this interval

covers exactly $\ell + 1$ points. Substituting our construction into the expression for F_k:

$$\begin{aligned} F_k &= \min_{i \in \{0,\ldots,k\}} (E_{k-i} + D_i) \\ &= \min_{i \in \{0,\ldots,k\}} \left(E_{(n-1)-(n-1-k+i)} + D_i\right) \\ &= (D_{n-1} - 1) - (1 - E_{n-1}) + \min_{i \in \{0,\ldots,k\}} \left(P_{n+(n-1-k+i)} - P_i\right) \\ &= G_{2n-k} + D_{n-1} + E_{n-1} - 2. \end{aligned}$$

Constructing the $2n$ points and later recovering the values of F_k takes $O(n)$ time. Thus, this linear-time reduction allows us to recover F_k from the BSEI solution G_{2n-k} (and consequently the solution to the original convolution problem). This finishes the proof of Theorem 8.2.

Theorem 8.2. *Assuming that* $(\min, +)$*-*Convolution *on sequences of length* n *requires* $\Omega(n^2)$ *time, the* Batched Smallest k*-*Enclosing Interval *problem on* n *points requires* $\Omega(n^2)$ *time.*

Chapter 9

Conclusion and Future Directions

In this thesis, we have investigated several variants of covering problems within the domains of network design and geometric optimization. Although these problems operate on different structures of graph and geometry, they are unified by the fundamental challenge of selecting a minimum-cost collection of resources to satisfy evolving requirements. This commonality is reflected in our technical approach; for instance, the interval flattening procedure served as a versatile tool that was successfully adapted to both online network augmentation and dynamic geometric coverage. This shared technique was used to provide strong theoretical guarantees across different problems, and different models of computation.

9.1 Summary of Contributions

9.1.1 Part I: Connectivity Augmentation

In the first part of the thesis (Chapters 4 and 5), we focused on making networks resilient to failures in an *online* setting.

- **Online CAP and WCAP:** We presented the first deterministic algorithms for the online Connectivity Augmentation Problem (CAP) and its weighted variant (WCAP) that achieve polylogarithmic competitive ratios independent of the connectivity requirement k. By reducing general graphs to cactus graphs and leveraging the structure of Steiner forests, we obtained an $O(\log n)$ competitive ratio for CAP and $O(\log^2 n)$ for WCAP.
- **Online TAP:** We stripped away the complexity of general graphs to focus on the online Tree Augmentation Problem (TAP). We provided simple, combinatorial $O(\log n)$-competitive algorithms for online TAP and WTAP. Furthermore, we proved an $\Omega(\log n)$

lower bound for online TAP even under the random-order arrival model, establishing that the hardness of online connectivity is inherent to the problem structure, not just the adversarial arrival order.

9.1.2 Part II: Geometric Optimization

In the second part (Chapters 6 to 8), we shifted focus to covering problems induced by geometric objects.

- **Dynamic Geometric Set Cover:** In Chapter 6, we broke the barrier of weight-dependence in dynamic Geometric Set Cover. By introducing a hierarchical flattening technique and utilizing segment trees, we achieved an $O(\log^d n)$-approximation with polylogarithmic update times for hyperrectangles, independent of the ratio of maximum to minimum weights W.
- **Horizontal Rectangle Stabbing:** In Chapter 7, we moved to the static setting and showed that geometry allows us to break the logarithmic barrier of general Set Cover. We developed a Polynomial-Time Approximation Scheme (PTAS) for the Rectangle Stabbing problem using a hierarchical grid decomposition and dynamic programming.
- **Hardness of Batched MaxRS:** Finally, in Chapter 8, we explored the limits of efficiency for the dual maximization problem. We established that the Batched Maximum Range Sum problem admits a conditional lower bound of $\Omega(mn)$, suggesting that for weighted instances, no algorithm can significantly outperform the naive approach of solving each query independently.

9.2 Future Research Directions

While this thesis advances the state-of-the-art in several covering problems, many questions remain open. We outline a few specific directions for future research below.

9.2.1 Online Network Design

In Chapters 4 and 5, we presented deterministic algorithms for online CAP and WCAP. However, several gaps remain between the upper and lower bounds in this domain.

- **Tightening the Bound for Weighted CAP:** Our algorithm for the weighted variant (WCAP) achieves a competitive ratio of $O(\log^2 n)$, whereas the unweighted case achieves $O(\log n)$. A natural open question is whether it is possible to achieve an $O(\log n)$-competitive algorithm for online WCAP, matching the lower bound for the unweighted case.

- **Lower Bounds for Online SNDP:** Online CAP is a special case of the online Survivable Network Design Problem (SNDP). While we established an $\Omega(\log n)$ lower bound for TAP (and thus CAP), the general online SNDP is much richer. Improving the lower bounds for online SNDP remains an important theoretical goal.

9.2.2 Geometric Set Cover and Geometric Hitting Set

In Chapter 6, we focused on the dynamic maintenance of geometric set covers. Both the dynamic and the strictly online (irrevocable) settings offer rich avenues for future work.

- **Improved Approximation for Dynamic Rectangles:** For dynamic weighted Set Cover on d-dimensional hyperrectangles, we achieved an $O(\log^d n)$ approximation. For the specific case of 2D rectangles ($d = 2$), is it possible to improve this to $O(\log n)$ with polylogarithmic update time?

- **Fully Dynamic Geometric Set Cover:** Our work focused on the setting where points are dynamic but the geometric sets are fixed. A major challenge is the *fully dynamic* setting, where both points and geometric objects can be inserted or deleted. Obtaining algorithms with sublinear update time and polylogarithmic approximation factors for rectangles in this setting remains open.

- **Online Geometric Covering:** Moving from dynamic to *online* algorithms (where decisions are irrevocable), several gaps persist. For example, can we design an online algorithm for Set Cover on 3D cubes with a competitive ratio $o(\log^2 n)$? Similarly, for the weighted case of online Set Cover on unit squares, can we surpass the $O(\log^2 n)$ barrier?

9.2.3 Rectangle Stabbing

In Chapter 7, we resolved the complexity of horizontal Rectangle Stabbing by providing a PTAS, and provided a $(2+\varepsilon)$-approximation for Horizontal-Vertical Stabbing. However, it remains unknown whether the latter problem admits a PTAS or if it is APX-hard. Resolving the complexity of HV-Stabbing is the primary open problem in this area.

9.2.4 Maximum Range Sum Queries

In Chapter 8, we established conditional lower bounds for the batched weighted problem.

- **Unweighted Batched MaxRS:** Our $\Omega(mn)$ lower bound relies on point weights to embed the $(\min, +)$-convolution hardness. For the unweighted Batched MaxRS

problem in $\mathbb{R}$, the best known algorithm takes $O(mn \log n)$ time. It remains open whether a similar quadratic lower bound exists for the unweighted case, or if it can be solved in $\tilde{O}(n+m)$ time.

- **Batched and Static Hardness in Higher Dimensions:** For Batched MaxRS with disks in $\mathbb{R}^2$, exact algorithms require $O(mn^2)$ time. Can we prove a matching lower bound? Furthermore, for the static single-query MaxRS for d-balls, can we prove a conditional lower bound matching the $\tilde{O}(n^d)$ upper bound?

Bibliography

[1] Amir Abboud, Raghavendra Addanki, Fabrizio Grandoni, Debmalya Panigrahi, and Barna Saha. Dynamic set cover: improved algorithms and lower bounds. In *Proc. STOC*, pages 114–125, 2019. 9, 67, 69

[2] Mikkel Abrahamsen, Anna Adamaszek, and Tillmann Miltzow. The art gallery problem is $\exists\mathbb{R}$-complete. *Journal of the ACM*, 69(1):4:1–4:70, 2022. 12

[3] David Adjiashvili. Beating approximation factor two for weighted tree augmentation with bounded costs. *ACM Transactions on Algorithms*, 15(2):19:1–19:26, 2019. 26

[4] Pankaj K. Agarwal, Hsien-Chih Chang, Subhash Suri, Allen Xiao, and Jie Xue. Dynamic geometric set cover and hitting set. In *Proc. SoCG*, pages 2:1–2:15, 2020. 67, 69

[5] Pankaj K. Agarwal, Torben Hagerup, Rahul Ray, Micha Sharir, Michiel H. M. Smid, and Emo Welzl. Translating a planar object to maximize point containment. In *Proc. ESA*, pages 42–53, 2002. 114

[6] Pankaj K. Agarwal and Jiangwei Pan. Near-linear algorithms for geometric hitting sets and set covers. *Discrete & Computational Geometry*, 63(2):460–482, 2020. 66

[7] Noga Alon, Baruch Awerbuch, and Yossi Azar. The online set cover problem. In *Proc. STOC*, pages 100–105, 2003. 70

[8] Daichi Amagata and Takahiro Hara. A general framework for MaxRS and MaxCRS monitoring in spatial data streams. *ACM Transactions on Spatial Algorithms and Systems*, 3(1):1–34, 2017. 114

[9] Boris Aronov, Esther Ezra, and Micha Sharir. Small-size ϵ-nets for axis-parallel rectangles and boxes. *SIAM Journal on Computing*, 39(7):3248–3282, 2010. 66, 70

[10] Boris Aronov and Sariel Har-Peled. On approximating the depth and related problems. *SIAM Journal on Computing*, 38(3):899–921, 2008. 113

[11] Sepehr Assadi and Shay Solomon. Fully dynamic set cover via hypergraph maximal matching: an optimal approximation through a local approach. In *Proc. ESA*, page 8, 2021. 69

[12] Étienne Bamas, Marina Drygala, and Ola Svensson. A simple LP-based approximation algorithm for the matching augmentation problem. In *Proc. IPCO*, pages 57–69, 2022. 28

[13] Étienne Bamas, Andreas Maggiori, and Ola Svensson. The primal-dual method for learning augmented algorithms. In *Proc. NeurIPS*, 2020. 9

[14] Nikhil Bansal, Alberto Caprara, and Maxim Sviridenko. A new approximation method for set covering problems, with applications to multidimensional bin packing. *SIAM Journal on Computing*, 39(4):1256–1278, 2009. 13

[15] Manu Basavaraju, Fedor V. Fomin, Petr A. Golovach, Pranabendu Misra, M. S. Ramanujan, and Saket Saurabh. Parameterized algorithms to preserve connectivity. In *Proc. ICALP*, pages 800–811, 2014. 27, 46

[16] Piotr Berman and Chris Coulston. On-line algorithms for Steiner tree problems. In *Proc. STOC*, pages 344–353, 1997. 28

[17] Sayan Bhattacharya, Ruoxu Cen, and Debmalya Panigrahi. Fully dynamic set cover: Worst-case recourse and update time. *arXiv preprint arXiv:2511.08485*, 2025. 9

[18] Sayan Bhattacharya, Ruoxu Cen, and Debmalya Panigrahi. Fully dynamic set cover: Worst-case recourse and update time. In *Proc. STOC*, 2026. 68, 69, 72

[19] Sayan Bhattacharya, Monika Henzinger, and Giuseppe F. Italiano. Deterministic fully dynamic data structures for vertex cover and matching. *SIAM Journal on Computing*, 47(3):859–887, 2018. 69

[20] Sayan Bhattacharya, Monika Henzinger, and Giuseppe F. Italiano. Dynamic algorithms via the primal-dual method. *Information and Computation*, 261:219–239, 2018. 69

[21] Sayan Bhattacharya, Monika Henzinger, and Danupon Nanongkai. A new deterministic algorithm for dynamic set cover. In *Proc. FOCS*, pages 406–423, 2019. 69

[22] Sayan Bhattacharya, Monika Henzinger, Danupon Nanongkai, and Xiaowei Wu. Dynamic set cover: improved amortized and worst-case update time. In *Proc. SODA*, pages 2537–2549, 2021. 69

[23] Sujoy Bhore, Anupam Gupta, and Amit Kumar. Improved online hitting set algorithms for structured and geometric set systems. In *Proc. SoCG*, 2026. 70

[24] Allan Borodin and Ran El-Yaniv. *Online computation and competitive analysis*. Cambridge University Press, 1998. 8

[25] Sander Borst, Marek Eliás, and Moritz Venzin. Stronger adversaries grow cheaper forests: online node-weighted Steiner problems. In *Proc. SODA*, pages 3842–3864, 2025. 28, 29, 48

[26] Miguel Bosch-Calvo, Mohit Garg, Fabrizio Grandoni, Felix Hommelsheim, Afrouz Jabal Ameli, and Alexander Lindermayr. A 5/4-approximation for two-edge connectivity. In *Proc. STOC*, pages 653–664, 2025. 28

[27] Joan Boyar, Lene M. Favrholdt, Christian Kudahl, Kim S. Larsen, and Jesper W. Mikkelsen. Online algorithms with advice: A survey. *ACM Computing Surveys*, 50(2):19:1–19:34, 2017. 9

[28] David Bremner, Timothy M. Chan, Erik D. Demaine, Jeff Erickson, Ferran Hurtado, John Iacono, Stefan Langerman, Mihai Pătraşcu, and Perouz Taslakian. Necklaces, convolutions, and X+Y. *Algorithmica*, 69(2):294–314, 2014. 24

[29] Hervé Brönnimann and Michael T. Goodrich. Almost optimal set covers in finite VC-dimension. *Discrete & Computational Geometry*, 14(4):463–479, 1995. 12, 66

[30] Niv Buchbinder and Joseph Naor. The design of competitive online algorithms via a primal-dual approach. *Foundations and Trends in Theoretical Computer Science*, 3(2-3):93–263, 2009. 8

[31] Anton Bukov, Shay Solomon, and Tianyi Zhang. Nearly optimal dynamic set cover: breaking the quadratic-in-f time barrier. In *Proc. SODA*, pages 824–863, 2025. 69

[32] Jaroslaw Byrka, Fabrizio Grandoni, and Afrouz Jabal Ameli. Breaching the 2-approximation barrier for connectivity augmentation: a reduction to Steiner tree. *SIAM Journal on Computing*, 52(3):718–739, 2023. 26, 27, 46, 47

[33] Simon Caton and Christian Haas. Fairness in machine learning: A survey. *ACM Computing Surveys*, 56(7):166:1–166:38, 2024. 11

[34] Federica Cecchetto, Vera Traub, and Rico Zenklusen. Bridging the gap between tree and connectivity augmentation: unified and stronger approaches. In *Proc. STOC*, pages 370–383, 2021. 26

[35] Timothy M. Chan. A (slightly) faster algorithm for Klee's measure problem. *Computational Geometry*, 43(3):243–250, 2010. 113

[36] Timothy M. Chan and Elyot Grant. Exact algorithms and APX-hardness results for geometric packing and covering problems. *Computational Geometry*, 47(2):112–124, 2014. 70

[37] Timothy M. Chan, Elyot Grant, Jochen Könemann, and Malcolm Sharpe. Weighted capacitated, priority, and geometric set cover via improved quasi-uniform sampling. In *Proc. SODA*, pages 1576–1585, 2012. 70

[38] Timothy M. Chan and Qizheng He. More dynamic data structures for geometric set cover with sublinear update time. In *Proc. SoCG*, pages 25:1–25:14, 2021. 67

[39] Timothy M. Chan, Qizheng He, Subhash Suri, and Jie Xue. Dynamic geometric set cover, revisited. In *Proc. SODA*, pages 3496–3528, 2022. 4, 67

[40] Timothy M. Chan, Thomas C. van Dijk, Krzysztof Fleszar, Joachim Spoerhase, and Alexander Wolff. Stabbing rectangles by line segments - how decomposition reduces the shallow-cell complexity. In *Proc. ISAAC*, pages 61:1–61:13, 2018. 4, 78, 82, 84, 85, 86, 87, 93, 95, 98, 99, 100

[41] Bernard M. Chazelle and Der-Tsai Lee. On a circle placement problem. *Computing*, 36(1):1–16, 1986. 113

[42] Joseph Cheriyan, Robert Cummings, Jack Dippel, and Jasper Zhu. An improved approximation algorithm for the matching augmentation problem. *SIAM Journal on Discrete Mathematics*, 37(1):163–190, 2023. 28

[43] Joseph Cheriyan, Jack Dippel, Fabrizio Grandoni, Arindam Khan, and Vishnu V. Narayan. The matching augmentation problem: a 7/4-approximation algorithm. *Mathematical Programming*, 182(1):315–354, 2020. 28

[44] Joseph Cheriyan and Zhihan Gao. Approximating (unweighted) tree augmentation via lift-and-project, part I: stemless TAP. *Algorithmica*, 80(2):530–559, 2018. 26

[45] Joseph Cheriyan and Zhihan Gao. Approximating (unweighted) tree augmentation via lift-and-project, part II. *Algorithmica*, 80(2):608–651, 2018. 26

[46] Joseph Cheriyan, András Sebö, and Zoltán Szigeti. Improving on the 1.5-approximation of a smalles 2-edge connected spanning subgraph. *SIAM Journal on Discrete Mathematics*, 14(2):170–180, 2001. 28

[47] Dong-Wan Choi, Chin-Wan Chung, and Yufei Tao. Maximizing range sum in external memory. *ACM Transactions on Database Systems*, 39(3):1–44, 2014. 114

[48] Vasek Chvátal. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research*, 4(3):233–235, 1979. 66

[49] Kenneth L. Clarkson and Kasturi R. Varadarajan. Improved approximation algorithms for geometric set cover. *Discrete & Computational Geometry*, 37(1):43–58, 2007. 12, 70

[50] Ilan Reuven Cohen and Debmalya Panigrahi. A general framework for learning-augmented online allocation. In *Proc. ICALP*, volume 261, pages 43:1–43:21, 2023. 9

[51] Marek Cygan, Fedor V. Fomin, Lukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michal Pilipczuk, and Saket Saurabh. *Parameterized Algorithms*. Springer, 2015. 8

[52] Marek Cygan, Marcin Mucha, Karol Wegrzycki, and Michal Wlodarczyk. On problems equivalent to (min, +)-convolution. *ACM Transactions on Algorithms*, 15(1):1–25, 2019. 24

[53] Rajni Dabas, Samir Khuller, and Emilie Rivkin. Capacitated partition vertex cover and partition edge cover. *arXiv preprint arXiv:2512.17844*, 2025. 11

[54] Rajni Dabas, Samir Khuller, and Emilie Rivkin. Weighted partition vertex and edge cover. *arXiv preprint arXiv:2508.13055*, 2025. 11

[55] Aparna Das, Krzysztof Fleszar, Stephen G. Kobourov, Joachim Spoerhase, Sankar Veeramoni, and Alexander Wolff. Approximating the generalized minimum Manhattan network problem. *Algorithmica*, 80(4):1170–1190, 2018. 100

[56] Minati De, Saksham Jain, Sarat Varma Kallepalli, and Satyam Singh. Online piercing of geometric objects. In *Proc. FSTTCS*, pages 17:1–17:16, 2022. 70

[57] Minati De, Sambhav Khurana, and Satyam Singh. Online dominating set and coloring. In *Proc. COCOA*, pages 68–81, 2023. 70

[58] Mark de Berg, Sergio Cabello, and Sariel Har-Peled. Covering many or few points with unit disks. *Theory of Computing Systems*, 45(3):446–469, 2009. 114

[59] Mark de Berg, Otfried Cheong, Marc J. van Kreveld, and Mark H. Overmars. *Computational Geometry: Algorithms and Applications*. Springer, 3rd edition, 2008. 72

[60] Reinhard Diestel. *Graph Theory*. Springer-Verlag, Berlin, Heidelberg, 5th edition, 2017. 3

[61] E. A. Dinitz, A. V. Karzanov, and M. V. Lomonosov. On the structure of a family of minimal weighted cuts in a graph. *Studies in Discrete Optimization*, pages 290–306, 1976. 27, 29

[62] Irit Dinur and David Steurer. Analytical approach to parallel repetition. In *Proc. STOC*, pages 624–633, 2014. 8

[63] Friedrich Eisenbrand, Martina Gallato, Ola Svensson, and Moritz Venzin. A QPTAS for stabbing rectangles. *arXiv preprint arXiv:2107.06571*, 2021. 78, 79, 87

[64] Guy Even, Jon Feldman, Guy Kortsarz, and Zeev Nutov. A 1.8 approximation algorithm for augmenting edge-connectivity of a graph from 1 to 2. *ACM Transactions on Algorithms*, 5(2):21:1–21:17, 2009. 26

[65] Guy Even and Shakhar Smorodinsky. Hitting sets online and vertex ranking. In *Proc. ESA*, pages 347–357, 2011. 70

[66] Dan Feldman. Introduction to core-sets: an updated survey. *arXiv preprint arXiv:2011.09384*, 2020. 10

[67] Samuel Fiorini, Martin Groß, Jochen Könemann, and Laura Sanità. Approximating weighted tree augmentation via Chvátal-Gomory cuts. In *Proc. SODA*, pages 817–831, 2018. 26

[68] Fedor V. Fomin and Dieter Kratsch. *Exact Exponential Algorithms*. Springer, 2010. 8

[69] Rajiv Gandhi, Samir Khuller, and Aravind Srinivasan. Approximation algorithms for partial covering problems. *Journal of Algorithms*, 53(1):55–84, 2004. 11

[70] Mohit Garg, Fabrizio Grandoni, and Afrouz Jabal Ameli. Improved approximation for two-edge-connectivity. In *Proc. SODA*, pages 2368–2410, 2023. 28

[71] Mohit Garg, Felix Hommelsheim, and Nicole Megow. Matching augmentation via simultaneous contractions. In *Proc. ICALP*, pages 65:1–65:17, 2023. 28

[72] Naveen Garg, Santosh S. Vempala, and Aman Singla. Improved approximation algorithms for biconnected subgraphs via better lower bounding techniques. In *Proc. SODA*, pages 103–111, 1993. 28

[73] Paul C. Gilmore and Ralph E. Gomory. A linear programming approach to the cutting-stock problem. *Operations Research*, 9(6):849–859, 1961. 12

[74] Michel X. Goemans and David P. Williamson. A general approximation technique for constrained forest problems. *SIAM Journal on Computing*, 24(2):296–317, 1995. 11

[75] Fabrizio Grandoni, Afrouz Jabal Ameli, and Vera Traub. Breaching the 2-approximation barrier for the forest augmentation problem. In *Proc. STOC*, pages 1598–1611, 2022. 28

[76] Fabrizio Grandoni, Christos Kalaitzis, and Rico Zenklusen. Improved approximation for tree augmentation: saving by rewiring. In *Proc. STOC*, pages 632–645, 2018. 26

[77] Peter Greenhalgh. Big.LITTLE processing with arm cortex-A15 & cortex-A7. *ARM White paper*, 17, 2011. 77

[78] Jiong Guo, Rolf Niedermeier, and Sebastian Wernicke. Parameterized complexity of generalized vertex cover problems. In *Proc. WADS*, volume 3608, pages 36–48, 2005. 10

[79] Anupam Gupta and Jochen Könemann. Approximation algorithms for network design: A survey. *Surveys in Operations Research and Management Science*, 16(1):3–20, 2011. 12

[80] Anupam Gupta, Ravishankar Krishnaswamy, Amit Kumar, and Debmalya Panigrahi. Online and dynamic algorithms for set cover. In *Proc. STOC*, pages 537–550, 2017. 68, 69, 71, 72

[81] Anupam Gupta, Ravishankar Krishnaswamy, and R. Ravi. Online and stochastic survivable network design. *SIAM Journal on Computing*, 41(6):1649–1672, 2012. 26

[82] Anupam Gupta and Roie Levin. Fully-dynamic submodular cover with bounded recourse. In *Proc. FOCS*, pages 1147–1157, 2020. 69

[83] Anupam Gupta, Martin Pál, R. Ravi, and Amitabh Sinha. Boosted sampling: approximation algorithms for stochastic optimization. In *Proc. STOC*, pages 417–426, 2004. 10

[84] Rachana Gusain, Saladi Rahul, and Aditya Subramanian. A bouquet of results on maximum range sum: general techniques and hardness reductions. In *Proc. PODS*, 2026. 114

[85] MohammadTaghi Hajiaghayi, Vahid Liaghat, and Debmalya Panigrahi. Online node-weighted Steiner forest and extensions via disk paintings. *SIAM Journal on Computing*, 46(3):911–935, 2017. 29

[86] David G. Harris and N. S. Narayanaswamy. A faster algorithm for vertex cover parameterized by solution size. In *Proc. STACS 2024*, volume 289, pages 40:1–40:18, 2024. 8

[87] Felix Hommelsheim. Improved approximation algorithms for path and forest augmentation via a novel relaxation. *arXiv preprint arXiv:2505.15324*, 2025. 28

[88] Christoph Hunkenschröder, Santosh S. Vempala, and Adrian Vetta. A 4/3-approximation algorithm for the minimum 2-edge connected subgraph problem. *ACM Transactions on Algorithms*, 15(4):55:1–55:28, 2019. 28

[89] Hiroshi Imai and Takao Asano. Finding the connected components and a maximum clique of an intersection graph of rectangles in the plane. *Journal of Algorithms*, 4(4):310–323, 1983. 113

[90] Makoto Imase and Bernard M. Waxman. Dynamic Steiner tree problem. *SIAM Journal on Discrete Mathematics*, 4(3):369–384, 1991. 49

[91] Kamal Jain. A factor 2 approximation algorithm for the generalized Steiner network problem. *Combinatorica*, 21(1):39–60, 2001. 26

[92] Kamal Jain and Vijay V. Vazirani. Approximation algorithms for metric facility location and k-median problems using the primal-dual schema and lagrangian relaxation. *Journal of the ACM*, 48(2):274–296, 2001. 13

[93] Richard M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. 7

[94] Hervé Kerivin and Ali Ridha Mahjoub. Design of survivable networks: A survey. *Networks*, 46(1):1–21, 2005. 12

[95] Arindam Khan, Aditya Lonkar, Saladi Rahul, Aditya Subramanian, and Andreas Wiese. Online and dynamic algorithms for geometric set cover and hitting set. In *Proc. SoCG*, pages 46:1–46:17, 2023. 67, 68, 70

[96] Arindam Khan, Aditya Subramanian, Tobias Widmann, and Andreas Wiese. On approximation schemes for stabbing rectilinear polygons. In *Proc. FSTTCS*, volume 323, pages 27:1–27:18, 2024. 78, 107

[97] Subhash Khot and Oded Regev. Vertex cover might be hard to approximate to within $2-\epsilon$. *Journal of Computer and System Sciences*, 74(3):335–349, 2008. 108

[98] Samir Khuller and Uzi Vishkin. Biconnectivity approximations and graph carvings. In *Proc. STOC*, pages 759–770, 1992. 28

[99] Yusuke Kobayashi and Takashi Noguchi. An approximation algorithm for two-edge-connected subgraph problem via triangle-free two-edge-cover. In *Proc. ISAAC*, pages 49:1–49:10, 2023. 28

[100] Jochen Könemann, Ojas Parekh, and Danny Segev. A unified approach to approximating partial covering problems. *Algorithmica*, 59(4):489–509, 2011. 11

[101] Guy Kortsarz and Zeev Nutov. A simplified 1.5-approximation algorithm for augmenting edge-connectivity of a graph from 1 to 2. *ACM Transactions on Algorithms*, 12(2):23:1–23:20, 2016. 26

[102] Guy Kortsarz and Zeev Nutov. LP-relaxations for tree augmentation. *Discrete Applied Mathematics*, 239:94–105, 2018. 26

[103] Elias Koutsoupias. The k-server problem. *Computer Science Review*, 3(2):105–118, 2009. 13

[104] Shi Li. A 1.488 approximation algorithm for the uncapacitated facility location problem. *Information and Computation*, 222:45–58, 2013. 13

[105] Qiyu Liu, Xiang Lian, and Lei Chen. Probabilistic maximum range-sum queries on spatial database. In *Proc. SIGSPATIAL*, pages 159–168, 2019. 114

[106] Christine Markarian, Claude Fachkha, and Noura Yassine. Revisiting online algorithms: A survey of set cover solutions beyond competitive analysis. *IEEE Access*, 12:174723–174739, 2024. 8

[107] Andrew McGregor. Graph stream algorithms: a survey. *SIGMOD Record*, 43(1):9–20, 2014. 10

[108] Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan. A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6):115:1–115:35, 2022. 11

[109] Karl Menger. Zur allgemeinen kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927. 3

[110] Michael Mitzenmacher and Sergei Vassilvitskii. *Algorithms with Predictions*, page 646–662. Cambridge University Press, 2021. 9

[111] Nabil H. Mustafa, Rajiv Raman, and Saurabh Ray. Settling the APX-hardness status for geometric set cover. In *Proc. FOCS*, pages 541–550, 2014. 70

[112] Nabil H. Mustafa, Rajiv Raman, and Saurabh Ray. Quasi-polynomial time approximation scheme for weighted geometric set cover on pseudodisks and halfspaces. *SIAM Journal on Computing*, 44(6):1650–1669, 2015. 66

[113] Nabil H. Mustafa and Saurabh Ray. PTAS for geometric hitting set problems via local search. In *Proc. SoCG*, pages 17–22, 2009. 12, 66

[114] S. Muthukrishnan. Data streams: Algorithms and applications. *Foundations and Trends in Theoretical Computer Science*, 1(2), 2005. 9

[115] Hiroshi Nagamochi. An approximation for finding a smallest 2-edge-connected subgraph containing a specified spanning tree. *Discrete Applied Mathematics*, 126(1):83–113, 2003. 26

[116] Subhas C. Nandy and Bhargab B. Bhattacharya. A unified algorithm for finding maximum and minimum object enclosing rectangles and cuboids. *Computers & Mathematics with Applications*, 29(8):45–61, 1995. 113

[117] Joseph (Seffi) Naor, Seeun William Umboh, and David P. Williamson. Tight bounds for online weighted tree augmentation. *Algorithmica*, 84(2):304–324, 2022. 26, 41, 50, 51, 55, 57

[118] Joseph O'Rourke. *Art Gallery Theorems and Algorithms*. Oxford University Press, 1987. 12

[119] Nikhil Rukmabhatla, Rajshree Chabukswar, Sneha Gohad, and Michael Chynoweth. Intel performance hybrid architecture & software optimizations. *Intel White paper*, 2021. 77

[120] András Sebö and Jens Vygen. Shorter tours by nicer ears: 7/5-approximation for the graph-TSP, 3/2 for the path version, and 4/3 for two-edge-connected subgraphs. *Combinatorica*, 34(5):597–629, 2014. 28

[121] Daniel Dominic Sleator and Robert Endre Tarjan. A data structure for dynamic trees. *Journal of Computer and System Sciences*, 26(3):362–391, 1983. 56

[122] Shay Solomon and Amitai Uzrad. Dynamic $((1+\epsilon)$ ln $n)$-approximation algorithms for minimum set cover and dominating set. In *Proc. STOC*, pages 1187–1200, 2023. 69

[123] Shay Solomon and Amitai Uzrad. Dynamic set cover with worst-case recourse. *arXiv preprint arXiv:2511.07354*, 2025. 9

[124] Chaitanya Swamy and David B. Shmoys. Approximation algorithms for 2-stage stochastic optimization problems. *SIGACT News*, 37(1):33–46, 2006. 10

[125] Yufei Tao, Xiaocheng Hu, Dong-Wan Choi, and Chin-Wan Chung. Approximate MaxRS in spatial databases. In *Proc. VLDB*, volume 6, pages 1546–1557, 2013. 114

[126] Vera Traub and Rico Zenklusen. A better-than-2 approximation for weighted tree augmentation. In *Proc. FOCS*, pages 1–12, 2021. 26

[127] Vera Traub and Rico Zenklusen. Local search for weighted tree augmentation and Steiner tree. In *Proc. SODA*, pages 3253–3272, 2022. 26

[128] Vera Traub and Rico Zenklusen. A $(1.5+\epsilon)$-approximation algorithm for weighted connectivity augmentation. In *Proc. STOC*, pages 1820–1833, 2023. 26, 29

[129] Jorge Urrutia. Art gallery and illumination problems. In *Handbook of Computational Geometry*, pages 973–1027. North Holland / Elsevier, 2000. 12

[130] Kasturi R. Varadarajan. Weighted geometric set cover via quasi-uniform sampling. In *Proc. STOC*, pages 641–648, 2010. 66, 70

[131] Rahul Vaze. *Online Algorithms*. Cambridge University Press, 2023. 8

[132] Vijay V. Vazirani. *Approximation Algorithms*. Springer, 2001. 7, 16

[133] David P. Williamson and David B. Shmoys. *The Design of Approximation Algorithms*. Cambridge University Press, 2011. 7, 16

[134] Neal E. Young. Online paging and caching. In *Encyclopedia of Algorithms*, pages 1457–1461. Springer, 2016. 13

[135] Jian Zhang. A survey on streaming algorithms for massive graphs. In *Managing and Mining Graph Data*, volume 40, pages 393–420. Springer, 2010. 9

[136] Kaiqi Zhang, Hong Gao, Xixian Han, Jian Chen, and Jianzhong Li. Maximizing range sum in trajectory data. In *Proc. ICDE*, pages 755–766, 2022. 112